

```

pivotHighPrice = ta.pivohigh(legsInput, legsInput)

// Add a new `pivotPoint` object to the end of the array for each detected pivot.
if not na(pivotHighPrice)
    // A new pivot is found; create a new object of `pivotPoint` type, setting its `openTime` and `level` fields.
    newPivot = pivotPoint.new(time[legsInput], pivotHighPrice)
    // Add the new pivot object to the array.
    array.push(pivotHighArray, newPivot)

// On the last historical bar, draw pivot labels and connecting lines.
if barstate.islastconfirmedhistory
    var pivotPoint previousPoint = na
    for eachPivot in pivotHighArray
        // Display a label at the pivot point.
        label.new(eachPivot.openTime, eachPivot.level, str.tostring(eachPivot.level, format.mintick), xloc.bar
        // Create a line between pivots.
        if not na(previousPoint)
            // Only create a line starting at the loop's second iteration because lines connect two pivots.
            line.new(previousPoint.openTime, previousPoint.level, eachPivot.openTime, eachPivot.level, xloc = 
        // Save the pivot for use in the next iteration.
        previousPoint := eachPivot

```

Copying objects

In Pine, objects are assigned by reference. When an existing object is assigned to a new variable, both point to the same object.

In the example below, we create a `pivot1` object and set its `x` field to 1000. Then, we declare a `pivot2` variable containing the reference to the `pivot1` object, so both point to the same instance. Changing `pivot2.x` will thus also change `pivot1.x`, as both refer to the `x` field of the same object:

```

//@version=6
indicator("")
type pivotPoint
    int x
    float y
pivot1 = pivotPoint.new()
pivot1.x := 1000
pivot2 = pivot1
pivot2.x := 2000
// Both plot the value 2000.
plot(pivot1.x)
plot(pivot2.x)

```

To create a copy of an object that is independent of the original, we can use the built-in `copy()` method in this case.

In this example, we declare the `pivot2` variable referring to a copied instance of the `pivot1` object. Now, changing `pivot2.x` will not change `pivot1.x`, as it refers to the `x` field of a separate object:

```

//@version=6
indicator("")
type pivotPoint
    int x
    float y
pivot1 = pivotPoint.new()
pivot1.x := 1000
pivot2 = pivotPoint.copy(pivot1)
pivot2.x := 2000
// Plots 1000 and 2000.
plot(pivot1.x)
plot(pivot2.x)

```

It's important to note that the built-in `copy()` method produces a *shallow copy* of an object. If an object has fields with

special types (array, matrix, map, line, linefill, box, polyline, label, table, or chart.point), those fields in a shallow copy of the object will point to the same instances as the original.

In the following example, we have defined an `InfoLabel` type with a `label` as one of its fields. The script instantiates a shallow copy of the `parent` object, then calls a user-defined `set()` method to update the `info` and `lbl` fields of each object. Since the `lbl` field of both objects points to the same label instance, changes to this field in either object affect the other:

```
//@version=6
indicator("Shallow Copy")

type InfoLabel
  string info
  label lbl

method set(InfoLabel this, int x = na, int y = na, string info = na) =>
  if not na(x)
    this.lbl.set_x(x)
  if not na(y)
    this.lbl.set_y(y)
  if not na(info)
    this.info := info
    this.lbl.set_text(this.info)

var parent = InfoLabel.new("", label.new(0, 0))
var shallow = parent.copy()

parent.set(bar_index, 0, "Parent")
shallow.set(bar_index, 1, "Shallow Copy")
```

To produce a *deep copy* of an object with all of its special type fields pointing to independent instances, we must explicitly copy those fields as well.

In this example, we have defined a `deepCopy()` method that instantiates a new `InfoLabel` object with its `lbl` field pointing to a copy of the original's field. Changes to the `deep` copy's `lbl` field will not affect the `parent` object, as it points to a separate instance:

```
//@version=6
indicator("Deep Copy")

type InfoLabel
  string info
  label lbl

method set(InfoLabel this, int x = na, int y = na, string info = na) =>
  if not na(x)
    this.lbl.set_x(x)
  if not na(y)
    this.lbl.set_y(y)
  if not na(info)
    this.info := info
    this.lbl.set_text(this.info)

method deepCopy(InfoLabel this) =>
  InfoLabel.new(this.info, this.lbl.copy())

var parent = InfoLabel.new("", label.new(0, 0))
var deep = parent.deepCopy()

parent.set(bar_index, 0, "Parent")
deep.set(bar_index, 1, "Deep Copy")
```

Shadowing

To avoid potential conflicts in the eventuality where namespaces added to Pine Script™ in the future would collide with UDTs or object names in existing scripts; as a rule, UDTs and object names shadow the language’s namespaces. For example, a UDT or object can use the name of built-in types, such as `line` or `table`.

Only the language’s five primitive types cannot be used to name UDTs or objects: `int`, `float`, `string`, `bool`, and `color`.

[\[Previous\]](#)

[User-defined functions\]\(#user-defined-functions\)\[Next](#)

[Enums\]\(#enums\)](#) [User Manual/Language/Enums](#)

ADVANCED

Enums

Introduction

Pine Script™ Enums, otherwise known as *enumerations*, *enumerated types*, or enum types, are unique data types with all possible values (*members*) explicitly defined by the programmer. They provide a human-readable, expressive way to declare distinct sets of *predefined values* that variables, conditional expressions, and collections can accept, allowing more strict control over the values used in a script’s logic.

Declaring an enum

To declare an enum, use the `enum` keyword with the following syntax:

```
[export ]enum <enumName>    <field_1>[ = <title_1>]    <field_2>[ = <title_2>]    ...    <field_N>[ = <title_N>]
```

Each **field** in the enum represents a unique, *named member* (value) of the enum type. Users can specify optional “const string” **titles** for enum fields to add extra information about what their values represent. If the programmer does not specify a field’s title, its title is the “string” representation of its name. Enum inputs display enum field titles within their dropdown menus in a script’s “Settings/Inputs” tab. Scripts can also retrieve enum field titles using the `str.toString()` function, allowing their use in additional calculations. See this section below for more information.

While the above syntax may look similar to the syntax for declaring user-defined types (UDTs), it’s crucial to understand that enum types and UDTs serve different purposes. Scripts use UDTs to create objects with “series” fields that can hold values of *any* specified type. In contrast, enums are distinct groups of “simple” fields representing the specific, *predefined values* of the same *unique* type that variables, expressions, and collections can accept.

For example, this code block declares a `Signal` enum with three fields: `buy`, `sell`, and `neutral`. Each field represents a distinct member (possible value) of the `Signal` enum type:

```
//@enum          An enumeration of named values representing buy, sell, and neutral signal states.
//@field buy      Represents a "Buy signal" state.
//@field sell      Represents a "Sell signal" state.
//@field neutral   Represents a "neutral" state.
enum Signal
    buy           = "Buy signal"
    sell          = "Sell signal"
    neutral
```

Note that:

- The `Signal` identifier represents the enum’s name, which signifies the *unique type* the fields belong to.
- We used the `//@enum` and `//@field` annotations to document the meaning of the enum and its fields.
- Unlike the `buy` and `sell` fields, the `neutral` field does not include a specified title. As such, its title is the “string” representation of its *name* (“neutral”).

To retrieve a member of an enum, reference its field name using *dot notation* syntax, i.e.:

```
enumName.fieldName
```

As with other types, scripts can assign enum members to variables, function parameters, and UDT fields, allowing strict control over their allowed values.

For instance, this line of code declares a `mySignal` variable whose value is the `neutral` member of the `Signal` enum. Any value assigned to this variable later must also be of the same enum type:

```
mySignal = Signal.neutral
```

Note that the above line does not require declaring the variable's *type* as `Signal` because the compiler can automatically infer that information from the assigned value. If we use `na` as the initial value instead, we must use `Signal` as the type keyword to specify that `mySignal` will accept a `Signal` member:

```
Signal mySignal = na
```

Using enums

Scripts can compare enum members with the `==` and `!=` operators and use them in conditional structures, allowing the convenient creation of logical patterns with a reduced risk of unintended values or operations.

The following example declares an `OscType` enum with three fields representing different oscillator choices: `rsi`, `mfi`, and `cci`. The `calcOscillator()` function uses `OscType` members within a switch structure to determine which oscillator it calculates. The script calls this function using the value from an enum input as the `selection` argument and plots the resulting oscillator:

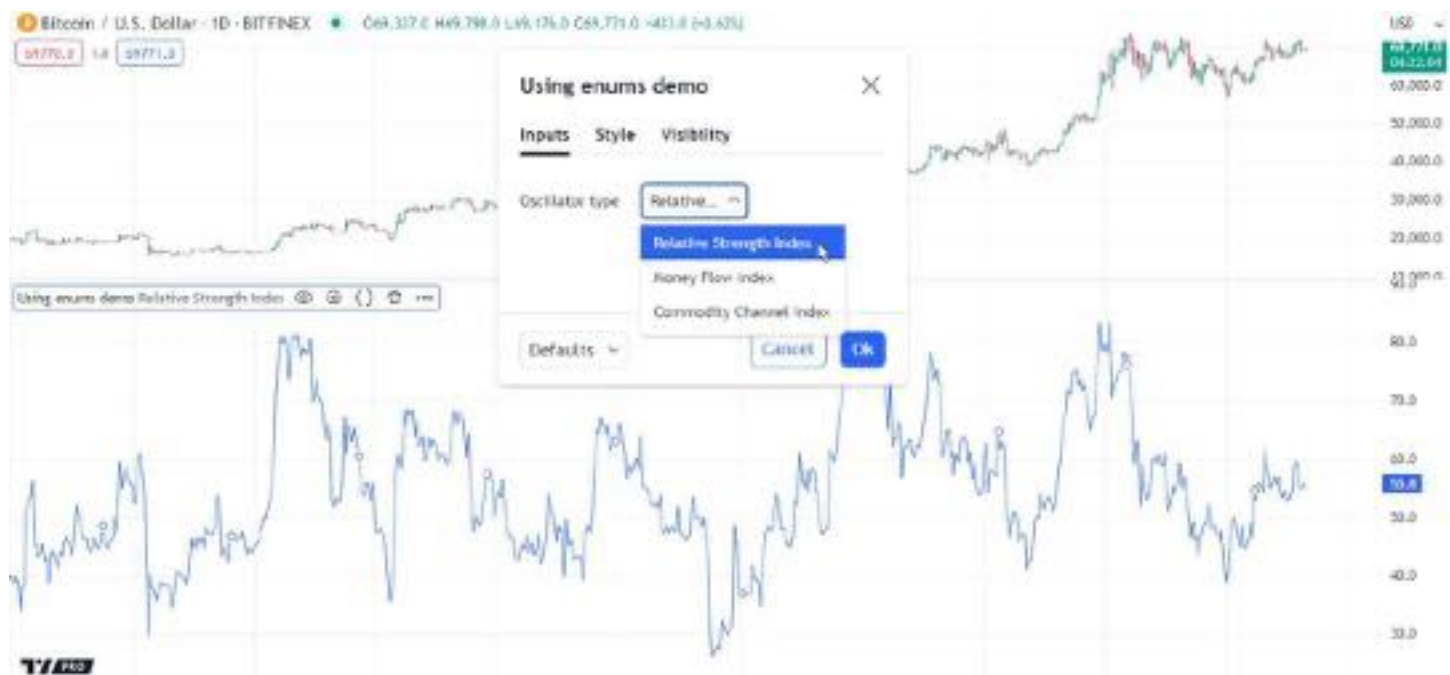


Figure 37: image

```
//@version=6
indicator("Using enums demo")

//@enum An enumeration of oscillator choices.
enum OscType
    rsi = "Relative Strength Index"
    mfi = "Money Flow Index"
    cci = "Commodity Channel Index"

//@variable An enumerator (member) of the `OscType` enum.
OscType oscInput = input.enum(OscType.rsi, "Oscillator type")

//@function          Calculates one of three oscillators based on a specified `selection`.
//@param source       The series of values to process.
//@param length       The number of bars in the calculation.
```

```
//@param selection Determines which oscillator to calculate.
calcOscillator(float source, simple int length, OscType selection) =>
    result = switch selection
        OscType.rsi => ta.rsi(source, length)
        OscType.mfi => ta.mfi(source, length)
        OscType.cci => ta.cci(source, length)

// Plot the value of a `calcOscillator()` call with `oscInput` as the `selection`.
plot(calcOscillator(close, 20, oscInput))
```

Note that:

- The `selection` parameter of the `calcOscillator()` function can only take on one of *four* values: `OscType.rsi`, `OscType.mfi`, `OscType.cci`, or `na`.
- The “Oscillator type” input in the script’s “Settings/Inputs” tab displays all `OscType` field titles in its dropdown. See this section to learn more about enum inputs.

It’s crucial to note that each declared enum represents a *unique* type. Scripts **cannot** compare members of different enums or use such members in expressions requiring a specific enum type, even if the fields have identical names and titles.

In this example, we added an `OscType2` enum to the above script and changed the `oscInput` variable to use a member of that enum. The script now raises a *compilation error* because it can’t use a member of the `OscType2` enum as the `selection` argument in the `calcOscillator()` call:

```
//@version=6
indicator("Incompatible enums demo")

//@enum An enumeration of oscillator choices.
enum OscType
    rsi = "Relative Strength Index"
    mfi = "Money Flow Index"
    cci = "Commodity Channel Index"

//@enum An enumeration of oscillator choices. Its fields DO NOT represent the same values those in the `OscType`
enum OscType2
    rsi = "Relative Strength Index"
    mfi = "Money Flow Index"
    cci = "Commodity Channel Index"

//@variable An enumerator (member) of the `OscType2` enum.
OscType2 oscInput = input.enum(OscType2.rsi, "Oscillator type")

//@function      Calculates one of three oscillators based on a specified `selection`.
//@param source   The series of values to process.
//@param length   The number of bars in the calculation.
//@param selection Determines which oscillator to calculate.
calcOscillator(float source, simple int length, OscType selection) =>
    result = switch selection
        OscType.rsi => ta.rsi(source, length)
        OscType.mfi => ta.mfi(source, length)
        OscType.cci => ta.cci(source, length)

// Plot the value of a `calcOscillator()` call with `oscInput` as the `selection`.
// Raises a compilation error because only members of `OscType` are allowed.
plot(calcOscillator(close, 20, oscInput))
```

Utilizing field titles

The “string” titles of an enum’s fields allow programmers to add extra information to each member. These field titles appear within a dropdown in the script’s “Settings/Inputs” tab when calling the `input.enum()` function.

Scripts can also utilize enum field titles in their calculations and logic. Use the string conversion function (`str.toString()`) on an enum field to access its title.

The following example combines different enum field titles to construct a ticker ID for requesting data from another context. The script declares two enums, **Exchange** and **Pair**, whose respective fields represent *exchange* and *currency pair* names. It uses `input.enum()` to assign user-specified enum members to the `exchangeInput` and `pairInput` variables, then retrieves the “string” titles from those variables with `str.toString()` and concatenates them to form an “Exchange:Symbol” pair for use in a `request.security()` call:



Figure 38: image

```
//@version=6
indicator("Utilizing field titles demo")

//@enum An enumeration of cryptocurrency exchanges. All field titles are the same as the field names.
enum Exchange
    BINANCE
    BITSTAMP
    BITFINEX
    COINBASE
    KRAKEN

//@enum An enumeration of cryptocurrency pairs. All the field titles are the same as the field names.
enum Pair
    BTCUSD
    ETHUSD
    SOLUSD
    XRPUSD

//@variable An enumerator (member) of the `Exchange` enum.
Exchange exchangeInput = input.enum(Exchange.BINANCE, "Exchange")
//@variable An enumerator (member) of the `Pair` enum.
Pair pairInput = input.enum(Pair.BTCUSD, "Pair")

//@variable The exchange-symbol pair for the data request.
simple string symbol = str.toString(exchangeInput) + ":" + str.toString(pairInput)

// Plot the `close` value requested from the `symbol` context.
plot(request.security(symbol, timeframe.period, close), "Requested close", color.purple, 3)

Note that:
```

- None of the members of the `Exchange` or `Pair` enums have specified titles. Therefore, each field's title is the “string” representation of its name, as shown by the script's enum inputs.
- Calling the `str.toString()` function on an enum field is the **only** way to retrieve its title for additional calculations. The `str.format()` and `log.*()` functions *cannot* accept enum members. To use a field's title in a string formatting function, call `str.toString()` on the field first, then pass the resulting “string” to the function.

Collecting enum members

Pine Script™ collections (arrays, matrices, and maps) can store enum members, allowing strict control over the values they can contain. To declare a collection of enum members, include the enum's *name* in the collection's type template.

For example, this code block creates an empty array to hold members of the `FooBar` enum. The only values this array can allow as elements are `FooBar.foo`, `FooBar.bar`, `FooBar.baz`, and `na`:

```
//@variable An enumeration of miscellaneous named members.
enum FooBar
    foo
    bar
    baz
```

```
//@variable An array that can only contain the following values: `FooBar.foo`, `FooBar.bar`, `FooBar.baz`, `na`
array<FooBar> fooBarArray = array.new<FooBar>()
```

Enums are particularly helpful when working with maps, as unlike other *non-fundamental* types, scripts can declare maps with *keys* of an enum type, enabling strict control over all possible keys allowed in their key-value pairs.

The following example uses a map with enum keys and “int” values to track and count signal states across chart bars. The script's `Signal` enum contains five fields representing specific named states. The `signalCounters` map uses the `Signal` name as the *first keyword* in its type template to specify that it can only accept `Signal` members as keys.

The script uses a switch structure to calculate a `signalState` variable whose value is a member of the `Signal` enum, which it uses to determine the counter value to update in the `signalCounters` map. It constructs a “string” to represent the key-value pairs of the map and displays the result in a single-cell table on the last chart bar:



Figure 39: image

```
//@version=6
indicator("Collecting enum members demo", overlay = true)

//@enum An enumeration of named signal states.
enum Signal
    strongBuy = "Strong buy"
    buy       = "Buy"
    neutral   = "Neutral"
```

```

    sell      = "Sell"
    strongSell = "Strong sell"

//@variable The number of bars in the signal calculation.
int lengthInput = input.int(50, "Length", 2)

//@variable A map of `Signal.*` keys and "int" values counting the number of bars with each signal state.
//      Allowed keys: `Signal.strongBuy`, `Signal.buy`, `Signal.neutral`, `Signal.sell`, `Signal.strongSell`
var map<Signal, float> signalCounters = map.new<Signal, float>()

//@variable A single-cell table displaying the key-value pairs of the `signalCounters` map.
var table infoTable = table.new(position.top_right, 1, 1, chart.bg_color)

if barstate.isfirst
    // Put `Signal.*`-"int" pairs into the `signalCounters` map to establish insertion order.
    signalCounters.put(Signal.strongBuy, 0)
    signalCounters.put(Signal.buy, 0)
    signalCounters.put(Signal.neutral, 0)
    signalCounters.put(Signal.sell, 0)
    signalCounters.put(Signal.strongSell, 0)
    // Initialize the `infoTable` cell.
    infoTable.cell(0, 0, text_color = chart.bg_color, text_halign = text.align_left, text_size = size.large)

// Calculate the EMA and Percent rank of `source` data over `length` bars.
float ema = ta.ema(close, lengthInput)
float rank = ta.percentrank(close, lengthInput)

//@variable A `Signal` member representing the current signal state based on `ema` and `rank` values.
Signal signalState = switch
    close > ema => rank > 70 ? Signal.strongBuy : rank > 50 ? Signal.buy : Signal.neutral
    close < ema => rank < 30 ? Signal.strongSell : rank < 50 ? Signal.sell : Signal.neutral
    => Signal.neutral

// Add 1 to the value in the `signalCounters` map associated with the `signalState` key.
signalCounters.put(signalState, signalCounters.get(signalState) + 1)

// Update the `infoTable` cell's text using the keys and values from the `signalCounters` map on the last bar.
if barstate.islast
    string tableText = ""
    for [state, count] in signalCounters
        tableText += str.toString(state) + ": " + str.toString(count) + "\n"
    infoTable.cell_set_text(0, 0, str.trim(tableText))

```

Note that:

- The `signalCounters` map can contain up to *six* key-value pairs, as the `Signal` enum has *five* predefined values, plus a possible value of `na`, and maps cannot contain *repetitive* keys.
- The script declares the `signalCounters` variable using the `var` keyword, signifying that the assigned map instance persists across executions.
- On the first chart bar, the script uses five `map.put()` calls to establish the *insertion order* of keys in the `signalCounters` map. See this section of the Maps page for more information.
- To minimize *resource usage*, the script declares the `infoTable` and initializes its cell on the *first bar*, then updates the cell's text on the *latest bar*. See this section of the Profiling and optimization page to learn more.

Shadowing

In contrast to user-defined types (UDTs), which can have names that *shadow* some built-in types or namespaces, enum types require *unique* names that do **not** match any built-in types or namespaces.

For example, this code declares four enums named `Syminfo`, `syminfo`, `polyline`, and `ta`. The last three all cause a compilation error because their names match built-in namespaces:


```
//@version=6
indicator("Shadowing demo")

// Naming an enum "Syminfo" with a capital "S" works without an issue.
enum Syminfo
    abcd

// In contrast, the names "syminfo", "polyline", "ta", etc. cause a compilation error because they match
// built-in namespaces:
enum syminfo
    abcd

enum polyline
    abcd

enum ta
    abcd

[Previous
```

[Objects\]\(#objects\)\[Next](#)

[Methods\]\(#methods\)](#) [User Manual/Language/Methods](#)

ADVANCED

Methods

Introduction

Pine Script™ methods are specialized functions associated with values of specific built-in types, user-defined types, or enum types. They behave the same as regular functions in most regards while offering a shorter, more convenient syntax. Users can access methods using *dot notation* syntax on variables of the associated type, similar to accessing the fields of a Pine Script™ object.

Built-in methods

Pine Script™ includes built-in methods for all *special types*, including array, matrix, map, line, linefill, box, polyline, label, and table. These methods provide users with a more concise way to call specialized routines for these types within their scripts.

When using these special types, the expressions:

```
<namespace>.<functionName>([paramName =] <objectName>, ...)
```

and:

```
<objectName>.<functionName>(…)
```

are equivalent. For example, rather than using:

```
array.get(id, index)
```

to get the value from an array `id` at the specified `index`, we can simply use:

```
id.get(index)
```

to achieve the same effect. This notation eliminates the need for users to reference the function's namespace, as `get()` is a method of `id` in this context.

Written below is a practical example to demonstrate the usage of built-in methods in place of functions.

The following script computes Bollinger Bands over a specified number of prices sampled once every `n` bars. It calls `array.push()` and `array.shift()` to queue `sourceInput` values through the `sourceArray`, then `array.avg()` and `array.stdev()` to compute the `sampleMean` and `sampleDev`. The script then uses these values to calculate the `highBand` and `lowBand`, which it plots on the chart along with the `sampleMean`:



Figure 40: image

```
//@version=6
indicator("Custom Sample BB", overlay = true)

float sourceInput = input.source(close, "Source")
int samplesInput = input.int(20, "Samples")
int n = input.int(10, "Bars")
float multiplier = input.float(2.0, "StdDev")

var array<float> sourceArray = array.new<float>(samplesInput)
var float sampleMean = na
var float sampleDev = na

// Identify if `n` bars have passed.
if bar_index % n == 0
    // Update the queue.
    array.push(sourceArray, sourceInput)
    array.shift(sourceArray)
    // Update the mean and standard deviation values.
    sampleMean := array.avg(sourceArray)
    sampleDev := array.stdev(sourceArray) * multiplier

// Calculate bands.
float highBand = sampleMean + sampleDev
float lowBand = sampleMean - sampleDev

plot(sampleMean, "Basis", color.orange)
plot(highBand, "Upper", color.lime)
plot(lowBand, "Lower", color.red)
```

Let's rewrite this code to utilize methods rather than built-in functions. In this version, we have replaced all built-in array.* functions in the script with equivalent methods:

```
//@version=6
indicator("Custom Sample BB", overlay = true)

float sourceInput = input.source(close, "Source")
```

```

int    samplesInput = input.int(20, "Samples")
int    n            = input.int(10, "Bars")
float multiplier    = input.float(2.0, "StdDev")

var array<float> sourceArray = array.new<float>(samplesInput)
var float      sampleMean  = na
var float      sampleDev   = na

// Identify if `n` bars have passed.
if bar_index % n == 0
    // Update the queue.
    sourceArray.push(sourceInput)
    sourceArray.shift()
    // Update the mean and standard deviation values.
    sampleMean := sourceArray.avg()
    sampleDev  := sourceArray.stdev() * multiplier

// Calculate band values.
float highBand = sampleMean + sampleDev
float lowBand  = sampleMean - sampleDev

plot(sampleMean, "Basis", color.orange)
plot(highBand, "Upper", color.lime)
plot(lowBand, "Lower", color.red)

```

Note that:

- We call the array methods using `sourceArray.*` rather than referencing the array namespace.
- We do not include `sourceArray` as a parameter when we call the methods since they already reference the object.

User-defined methods

Pine Script™ allows users to define custom methods for use with objects of any built-in or user-defined type. Defining a method is essentially the same as defining a function, but with two key differences:

- The method keyword must be included before the function name.
- The type of the first parameter in the signature must be explicitly declared, as it represents the type of object that the method will be associated with.

```
[export] method <functionName>(<paramType> <paramName> [= <defaultValue>], ...) =>    <functionBlock>
```

Let's apply user-defined methods to our previous Bollinger Bands example to encapsulate operations from the global scope, which will simplify the code and promote reusability. See this portion from the example:

```

// Identify if `n` bars have passed.
if bar_index % n == 0
    // Update the queue.
    sourceArray.push(sourceInput)
    sourceArray.shift()
    // Update the mean and standard deviation values.
    sampleMean := sourceArray.avg()
    sampleDev  := sourceArray.stdev() * multiplier

// Calculate band values.
float highBand = sampleMean + sampleDev
float lowBand  = sampleMean - sampleDev

```

We will start by defining a simple method to queue values through an array in a single call.

This `maintainQueue()` method invokes the `push()` and `shift()` methods on a `srcArray` when `takeSample` is true and returns the object:

```

// @function      Maintains a queue of the size of `srcArray`.
//               It appends a `value` to the array and removes its oldest element at position zero.
// @param srcArray (array<float>) The array where the queue is maintained.

```

```
// @param value      (float) The new value to be added to the queue.
//                  The queue's oldest value is also removed, so its size is constant.
// @param takeSample (bool) A new `value` is only pushed into the queue if this is true.
// @returns          (array<float>) `srcArray` object.
method maintainQueue(array<float> srcArray, float value, bool takeSample = true) =>
    if takeSample
        srcArray.push(value)
        srcArray.shift()
    srcArray
```

Note that:

- Just as with user-defined functions, we use the `@functioncompiler` annotation to document method descriptions.

Now we can replace `sourceArray.push()` and `sourceArray.shift()` with `sourceArray.maintainQueue()` in our example:

```
// Identify if `n` bars have passed.
if bar_index % n == 0
    // Update the queue.
    sourceArray.maintainQueue(sourceInput)
    // Update the mean and standard deviation values.
    sampleMean := sourceArray.avg()
    sampleDev  := sourceArray.stdev() * multiplier

// Calculate band values.
float highBand = sampleMean + sampleDev
float lowBand  = sampleMean - sampleDev
```

From here, we will further simplify our code by defining a method that handles all Bollinger Band calculations within its scope.

This `calcBB()` method invokes the `avg()` and `stdev()` methods on a `srcArray` to update `mean` and `dev` values when `calculate` is true. The method uses these values to return a tuple containing the basis, upper band, and lower band values respectively:

```
// @function      Computes Bollinger Band values from an array of data.
// @param srcArray (array<float>) The array where the queue is maintained.
// @param multiplier (float) Standard deviation multiplier.
// @param calculate (bool) The method will only calculate new values when this is true.
// @returns        A tuple containing the basis, upper band, and lower band respectively.
method calcBB(array<float> srcArray, float mult, bool calculate = true) =>
    var float mean = na
    var float dev  = na
    if calculate
        // Compute the mean and standard deviation of the array.
        mean := srcArray.avg()
        dev  := srcArray.stdev() * mult
    [mean, mean + dev, mean - dev]
```

With this method, we can now remove Bollinger Band calculations from the global scope and improve code readability:

```
// Identify if `n` bars have passed.
bool newSample = bar_index % n == 0

// Update the queue and compute new BB values on each new sample.
[sampleMean, highBand, lowBand] = sourceArray.maintainQueue(sourceInput, newSample).calcBB(multiplier, newSample)
```

Note that:

- Rather than using an `if` block in the global scope, we have defined a `newSample` variable that is only true once every `n` bars. The `maintainQueue()` and `calcBB()` methods use this value for their respective `takeSample` and `calculate` parameters.
- Since the `maintainQueue()` method returns the object that it references, we're able to call `calcBB()` from the same line of code, as both methods apply to `array<float>` instances.

Here is how the full script example looks now that we've applied our user-defined methods:

```

//@version=6
indicator("Custom Sample BB", overlay = true)

float sourceInput = input.source(close, "Source")
int samplesInput = input.int(20, "Samples")
int n = input.int(10, "Bars")
float multiplier = input.float(2.0, "StdDev")

var array<float> sourceArray = array.new<float>(samplesInput)

// @function Maintains a queue of the size of `srcArray`.
// It appends a `value` to the array and removes its oldest element at position zero.
// @param srcArray (array<float>) The array where the queue is maintained.
// @param value (float) The new value to be added to the queue.
// The queue's oldest value is also removed, so its size is constant.
// @param takeSample (bool) A new `value` is only pushed into the queue if this is true.
// @returns (array<float>) `srcArray` object.
method maintainQueue(array<float> srcArray, float value, bool takeSample = true) =>
    if takeSample
        srcArray.push(value)
        srcArray.shift()
    srcArray

// @function Computes Bollinger Band values from an array of data.
// @param srcArray (array<float>) The array where the queue is maintained.
// @param multiplier (float) Standard deviation multiplier.
// @param calculate (bool) The method will only calculate new values when this is true.
// @returns A tuple containing the basis, upper band, and lower band respectively.
method calcBB(array<float> srcArray, float mult, bool calculate = true) =>
    var float mean = na
    var float dev = na
    if calculate
        // Compute the mean and standard deviation of the array.
        mean := srcArray.avg()
        dev := srcArray.stdev() * mult
    [mean, mean + dev, mean - dev]

// Identify if `n` bars have passed.
bool newSample = bar_index % n == 0

// Update the queue and compute new BB values on each new sample.
[sampleMean, highBand, lowBand] = sourceArray.maintainQueue(sourceInput, newSample).calcBB(multiplier, newSample)

plot(sampleMean, "Basis", color.orange)
plot(highBand, "Upper", color.lime)
plot(lowBand, "Lower", color.red)

```

Method overloading

User-defined methods can override and overload existing built-in and user-defined methods with the same identifier. This capability allows users to define multiple routines associated with different parameter signatures under the same method name.

As a simple example, suppose we want to define a method to identify a variable's type. Since we must explicitly specify the type of object associated with a user-defined method, we will need to define overloads for each type that we want it to recognize.

Below, we have defined a `getType()` method that returns a string representation of a variable's type with overloads for the five primitive types:

```

// @function Identifies an object's type.
// @param this Object to inspect.

```

```
// @returns    (string) A string representation of the type.
method getType(int this) =>
    na(this) ? "int(na)" : "int"

method getType(float this) =>
    na(this) ? "float(na)" : "float"

method getType(bool this) =>
    // "bool" values only have two states, `true` and `false`, but never `na`.
    "bool"

method getType(color this) =>
    na(this) ? "color(na)" : "color"

method getType(string this) =>
    na(this) ? "string(na)" : "string"
```

Now we can use these overloads to inspect some variables. This script uses `str.format()` to format the results from calling the `getType()` method on five different variables into a single **results** string, then displays the string in the **lbl** label using the built-in `set_text()` method:



Figure 41: image

```
//@version=6
indicator("Type Inspection")

// @function    Identifies an object's type.
// @param this  Object to inspect.
// @returns    (string) A string representation of the type.
method getType(int this) =>
    na(this) ? "int(na)" : "int"

method getType(float this) =>
    na(this) ? "float(na)" : "float"

method getType(bool this) =>
    na(this) ? "bool(na)" : "bool"

method getType(color this) =>
    na(this) ? "color(na)" : "color"

method getType(string this) =>
    na(this) ? "string(na)" : "string"

a = 1
b = 1.0
c = true
```

```

d = color.white
e = "1"

// Inspect variables and format results.
results = str.format(
    "a: {0}\nb: {1}\nc: {2}\nd: {3}\ne: {4}",
    a.getType(), b.getType(), c.getType(), d.getType(), e.getType()
)

var label lbl = label.new(0, 0)
lbl.set_x(bar_index)
lbl.set_text(results)

```

Note that:

- The underlying type of each variable determines which overload of `getType()` the compiler will use.
- The method will append “(na)” to the output string when a variable is `na` to demarcate that it is empty.

Advanced example

Let’s apply what we’ve learned to construct a script that estimates the cumulative distribution of elements in an array, meaning the fraction of elements in the array that are less than or equal to any given value.

There are many ways in which we could choose to tackle this objective. For this example, we will start by defining a method to replace elements of an array, which will help us count the occurrences of elements within a range of values.

Written below is an overload of the built-in `fill()` method for `array<float>` instances. This overload replaces elements in a `srcArray` within the range between the `lowerBound` and `upperBound` with an `innerValue`, and replaces all elements outside the range with an `outerValue`:

```

// @function      Replaces elements in a `srcArray` between `lowerBound` and `upperBound` with an `innerValue`
//                and replaces elements outside the range with an `outerValue`.
// @param srcArray (array<float>) Array to modify.
// @param innerValue (float) Value to replace elements within the range with.
// @param outerValue (float) Value to replace elements outside the range with.
// @param lowerBound (float) Lowest value to replace with `innerValue`.
// @param upperBound (float) Highest value to replace with `innerValue`.
// @returns        (array<float>) `srcArray` object.
method fill(array<float> srcArray, float innerValue, float outerValue, float lowerBound, float upperBound) =>
    for [i, element] in srcArray
        if (element >= lowerBound or na(lowerBound)) and (element <= upperBound or na(upperBound))
            srcArray.set(i, innerValue)
        else
            srcArray.set(i, outerValue)
    srcArray

```

With this method, we can filter an array by value ranges to produce an array of occurrences. For example, the expression:

```
srcArray.copy().fill(1.0, 0.0, min, val)
```

copies the `srcArray` object, replaces all elements between `min` and `val` with 1.0, then replaces all elements above `val` with 0.0. From here, it’s easy to estimate the output of the cumulative distribution function at the `val`, as it’s simply the average of the resulting array:

```
srcArray.copy().fill(1.0, 0.0, min, val).avg()
```

Note that:

- The compiler will only use this `fill()` overload instead of the built-in when the user provides `innerValue`, `outerValue`, `lowerBound`, and `upperBound` arguments in the call.
- If either `lowerBound` or `upperBound` is `na`, its value is ignored while filtering the fill range.
- We are able to call `copy()`, `fill()`, and `avg()` successively on the same line of code because the first two methods return an `array<float>` instance.

We can now use this to define a method that will calculate our empirical distribution values. The following `eCDF()` method estimates a number of evenly spaced ascending `steps` from the cumulative distribution function of a `srcArray` and pushes

the results into a `cdfArray`:

```
// @function      Estimates the empirical CDF of a `srcArray`.
// @param srcArray (array<float>) Array to calculate on.
// @param steps    (int) Number of steps in the estimation.
// @returns       (array<float>) Array of estimated CDF ratios.
method eCDF(array<float> srcArray, int steps) =>
    float min = srcArray.min()
    float rng = srcArray.range() / steps
    array<float> cdfArray = array.new<float>()
    // Add averages of `srcArray` filtered by value region to the `cdfArray`.
    float val = min
    for i = 1 to steps
        val += rng
        cdfArray.push(srcArray.copy().fill(1.0, 0.0, min, val).avg())
    cdfArray
```

Lastly, to ensure that our `eCDF()` method functions properly for arrays containing small and large values, we will define a method to normalize our arrays.

This `featureScale()` method uses array `min()` and `range()` methods to produce a rescaled copy of a `srcArray`. We will use this to normalize our arrays prior to invoking the `eCDF()` method:

```
// @function      Rescales the elements within a `srcArray` to the interval [0, 1].
// @param srcArray (array<float>) Array to normalize.
// @returns       (array<float>) Normalized copy of the `srcArray`.
method featureScale(array<float> srcArray) =>
    float min = srcArray.min()
    float rng = srcArray.range()
    array<float> scaledArray = array.new<float>()
    // Push normalized `element` values into the `scaledArray`.
    for element in srcArray
        scaledArray.push((element - min) / rng)
    scaledArray
```

Note that:

- This method does not include special handling for divide by zero conditions. If `rng` is 0, the value of the array element will be `na`.

The full example below queues a `sourceArray` of size `length` with `sourceInput` values using our previous `maintainQueue()` method, normalizes the array's elements using the `featureScale()` method, then calls the `eCDF()` method to get an array of estimates for `n` evenly spaced steps on the distribution. The script then calls a user-defined `makeLabel()` function to display the estimates and prices in a label on the right side of the chart:

```
//@version=6
indicator("Empirical Distribution", overlay = true)

float sourceInput = input.source(close, "Source")
int length        = input.int(20, "Length")
int n             = input.int(20, "Steps")

// @function      Maintains a queue of the size of `srcArray`.
//                It appends a `value` to the array and removes its oldest element at position zero.
// @param srcArray (array<float>) The array where the queue is maintained.
// @param value    (float) The new value to be added to the queue.
//                The queue's oldest value is also removed, so its size is constant.
// @param takeSample (bool) A new `value` is only pushed into the queue if this is true.
// @returns       (array<float>) `srcArray` object.
method maintainQueue(array<float> srcArray, float value, bool takeSample = true) =>
    if takeSample
        srcArray.push(value)
        srcArray.shift()
    srcArray
```



```

method featureScale(array<float> srcArray) =>
    float min = srcArray.min()
    float rng = srcArray.range()
    array<float> scaledArray = array.new<float>()
    // Push normalized `element` values into the `scaledArray`.
    for element in srcArray
        scaledArray.push((element - min) / rng)
    scaledArray

// @function      Draws a label containing eCDF estimates in the format "{price}: {percent}%"
// @param srcArray (array<float>) Array of source values.
// @param cdfArray (array<float>) Array of CDF estimates.
// @returns       (void)
makeLabel(array<float> srcArray, array<float> cdfArray) =>
    float max      = srcArray.max()
    float rng      = srcArray.range() / cdfArray.size()
    string results = ""
    var label lbl = label.new(0, 0, "", style = label.style_label_left, text_font_family = font.family_monosp)
    // Add percentage strings to `results` starting from the `max`.
    cdfArray.reverse()
    for [i, element] in cdfArray
        results += str.format("{0}: {1}%\n", max - i * rng, element * 100)
    // Update `lbl` attributes.
    lbl.set_xy(bar_index + 1, srcArray.avg())
    lbl.set_text(results)

var array<float> sourceArray = array.new<float>(length)

// Add background color for the last `length` bars.
bgcolor(bar_index > last_bar_index - length ? color.new(color.orange, 80) : na)

// Queue `sourceArray`, feature scale, then estimate the distribution over `n` steps.
array<float> distArray = sourceArray.maintainQueue(sourceInput).featureScale().eCDF(n)
// Draw label.
makeLabel(sourceArray, distArray)

```

[Previous

Enums](#enums)[Next

Arrays](#arrays) User Manual/Language/Arrays

ADVANCED

Arrays

Introduction

Pine Script™ Arrays are one-dimensional collections that can hold multiple value references. Think of them as a better way to handle cases where one would otherwise need to explicitly declare a set of similar variables (e.g., `price00`, `price01`, `price02`, ...).

All elements in an array must be of the same built-in type, user-defined type, or enum type.

Scripts reference arrays using array IDs similar to the IDs of lines, labels, and other *special types*. Pine Script™ does not use an indexing operator to reference individual array elements. Instead, functions including `array.get()` and `array.set()` read and write the values of array elements.

Scripts reference the elements of an array using an *index*, which starts at 0 and extends to the number of elements in the array minus one. Arrays in Pine Script™ can have a dynamic size that varies across bars, as one can change the number of elements in an array on each iteration of a script. Scripts can contain multiple array instances. The size of arrays is limited to 100,000 elements.

Declaring arrays

Pine Script™ uses the following syntax to declare arrays:

```
[var/varip ] [array<type>/<type[]> ] <identifier> = <expression>
```

Where **<type>** is a type template for the array that declares the type of values it will contain, and the **<expression>** returns either an array of the specified type or **na**.

When declaring a variable as an array, we can use the array keyword followed by a type template. Alternatively, we can use the **type** name followed by the **[]** modifier (not to be confused with the **[]** *history-referencing operator*).

Since Pine always uses type-specific functions to create arrays, the **array<type>/type[]** part of the declaration is redundant, except when declaring an array variable assigned to **na**. Even when not required, explicitly declaring the array type helps clearly state the intention to readers.

This line of code declares an array variable named **prices** that points to **na**. In this case, we must specify the type to declare that the variable can reference arrays containing “float” values:

```
array<float> prices = na
```

We can also write the above example in this form:

```
float[] prices = na
```

When declaring an array and the **<expression>** is not **na**, use one of the following functions: **array.new(size, initial_value)**, **array.from()**, or **array.copy()**. For **array.new<type>(size, initial_value)** functions, the arguments of the **size** and **initial_value** parameters can be “series” to allow dynamic sizing and initialization of array elements. The following example creates an array containing zero “float” elements, and this time, the array ID returned by the **array.new()** function call is assigned to **prices**:

```
prices = array.new<float>(0)
```

The **initial_value** parameter of **array.new*** functions allows users to set all elements in the array to a specified value. If no argument is provided for **initial_value**, the array is filled with **na** values.

This line declares an array ID named **prices** pointing to an array containing two elements, each assigned to the bar’s **close** value:

```
prices = array.new<float>(2, close)
```

To create an array and initialize its elements with different values, use **array.from()**. This function infers the array’s size and the type of elements it will hold from the arguments in the function call. As with **array.new*** functions, it accepts “series” arguments. All values supplied to the function must be of the same type.

For example, all three of these lines of code will create identical “bool” arrays with the same two elements:

```
statesArray = array.from(close > open, high != close)
bool[] statesArray = array.from(close > open, high != close)
array<bool> statesArray = array.from(close > open, high != close)
```

Using var and varip keywords

Users can utilize **var** and **varip** keywords to instruct a script to declare an array variable only once on the first iteration of the script on the first chart bar. Array variables declared using these keywords point to the same array instances until explicitly reassigned, allowing an array and its element references to persist across bars.

When declaring an array variable using these keywords and pushing a new value to the end of the referenced array on each bar, the array will grow by one on each bar and be of size **bar_index + 1** (**bar_index** starts at zero) by the time the script executes on the last bar, as this code demonstrates:

```
//@version=6
indicator("Using `var`")
//@variable An array that expands its size by 1 on each bar.
var a = array.new<float>(0)
array.push(a, close)

if barstate.islast
    //@variable A string containing the size of `a` and the current `bar_index` value.
    string labelText = "Array size: " + str.tostring(a.size()) + "\nbar_index: " + str.tostring(bar_index)
```

```
// Display the `labelText`.
label.new(bar_index, 0, labelText, size = size.large)
```

The same code without the `var` keyword would re-declare the array on each bar. In this case, after execution of the `array.push()` call, the `a.size()` call would return a value of 1.

Reading and writing array elements

Scripts can write values to existing individual array elements using `array.set(id, index, value)`, and read using `array.get(id, index)`. When using these functions, it is imperative that the `index` in the function call is always less than or equal to the array's size (because array indices start at zero). To get the size of an array, use the `array.size(id)` function.

The following example uses the `set()` method to populate a `fillColors` array with instances of one base color using different transparency levels. It then uses `array.get()` to retrieve one of the colors from the array based on the location of the bar with the highest price within the last `lookbackInput` bars:



Figure 43: image

```
//@version=6
indicator("Distance from high", "", true)
lookbackInput = input.int(100)
FILL_COLOR = color.green
// Declare array and set its values on the first bar only.
var fillColors = array.new<color>(5)
if barstate.isfirst
    // Initialize the array elements with progressively lighter shades of the fill color.
    fillColors.set(0, color.new(FILL_COLOR, 70))
    fillColors.set(1, color.new(FILL_COLOR, 75))
    fillColors.set(2, color.new(FILL_COLOR, 80))
    fillColors.set(3, color.new(FILL_COLOR, 85))
    fillColors.set(4, color.new(FILL_COLOR, 90))

// Find the offset to highest high. Change its sign because the function returns a negative value.
lastHiBar = - ta.highestbars(high, lookbackInput)
// Convert the offset to an array index, capping it to 4 to avoid a runtime error.
// The index used by `array.get()` will be the equivalent of `floor(fillNo)`.
fillNo = math.min(lastHiBar / (lookbackInput / 5), 4)
// Set background to a progressively lighter fill with increasing distance from location of highest high.
bgcolor(array.get(fillColors, fillNo))
// Plot key values to the Data Window for debugging.
plotchar(lastHiBar, "lastHiBar", "", location.top, size = size.tiny)
plotchar(fillNo, "fillNo", "", location.top, size = size.tiny)
```

Another technique for initializing the elements in an array is to create an *empty array* (an array with no elements), then

use `array.push()` to append **new** elements to the end of the array, increasing the size of the array by one on each call. The following code is functionally identical to the initialization section from the preceding script:

```
// Declare array and set its values on the first bar only.
var fillColors = array.new<color>(0)
if barstate.isfirst
    // Initialize the array elements with progressively lighter shades of the fill color.
    array.push(fillColors, color.new(FILL_COLOR, 70))
    array.push(fillColors, color.new(FILL_COLOR, 75))
    array.push(fillColors, color.new(FILL_COLOR, 80))
    array.push(fillColors, color.new(FILL_COLOR, 85))
    array.push(fillColors, color.new(FILL_COLOR, 90))
```

This code is equivalent to the one above, but it uses `array.unshift()` to insert new elements at the *beginning* of the `fillColors` array:

```
// Declare array and set its values on the first bar only.
var fillColors = array.new<color>(0)
if barstate.isfirst
    // Initialize the array elements with progressively lighter shades of the fill color.
    array.unshift(fillColors, color.new(FILL_COLOR, 90))
    array.unshift(fillColors, color.new(FILL_COLOR, 85))
    array.unshift(fillColors, color.new(FILL_COLOR, 80))
    array.unshift(fillColors, color.new(FILL_COLOR, 75))
    array.unshift(fillColors, color.new(FILL_COLOR, 70))
```

We can also use `array.from()` to create the same `fillColors` array with a single function call:

```
//@version=6
indicator("Using `var`")
FILL_COLOR = color.green
var array<color> fillColors = array.from(
    color.new(FILL_COLOR, 70),
    color.new(FILL_COLOR, 75),
    color.new(FILL_COLOR, 80),
    color.new(FILL_COLOR, 85),
    color.new(FILL_COLOR, 90)
)
// Cycle background through the array's colors.
bgcolor(array.get(fillColors, bar_index % (fillColors.size())))
```

The `array.fill(id, value, index_from, index_to)` function points all array elements, or the elements within the `index_from` to `index_to` range, to a specified value. Without the last two optional parameters, the function fills the whole array, so:

```
a = array.new<float>(10, close)
```

and:

```
a = array.new<float>(10)
a.fill(close)
```

are equivalent, but:

```
a = array.new<float>(10)
a.fill(close, 1, 3)
```

only fills the second and third elements (at index 1 and 2) of the array with `close`. Note how `array.fill()`'s last parameter, `index_to`, must be one greater than the last index the function will fill. The remaining elements will hold **na** values, as the `array.new()` function call does not contain an `initial_value` argument.

Looping through array elements

When looping through an array's element indices and the array's size is unknown, one can use the `array.size()` function to get the maximum index value. For example:

```
//@version=6
indicator("Protected `for` loop", overlay = true)
```

```
//@variable An array of `close` prices from the 1-minute timeframe.
array<float> a = request.security_lower_tf(syminfo.tickerid, "1", close)

//@variable A string representation of the elements in `a`.
string labelText = ""
for i = 0 to (array.size(a) == 0 ? na : array.size(a) - 1)
    labelText += str.tostring(array.get(a, i)) + "\n"

label.new(bar_index, high, text = labelText)
```

Note that:

- We use the `request.security_lower_tf()` function which returns an array of close prices at the 1 minute timeframe.
- This code example will throw an error if you use it on a chart timeframe smaller than 1 minute.
- for loops do not execute if the `to` expression is `na`. Note that the `to` value is only evaluated once upon entry.

An alternative method to loop through an array is to use a `for...in` loop. This approach is a variation of the standard `for` loop that can iterate over the value references and indices in an array. Here is an example of how we can write the code example from above using a `for...in` loop:

```
//@version=6
indicator("`for...in` loop", overlay = true)
//@variable An array of `close` prices from the 1-minute timeframe.
array<float> a = request.security_lower_tf(syminfo.tickerid, "1", close)

//@variable A string representation of the elements in `a`.
string labelText = ""
for price in a
    labelText += str.tostring(price) + "\n"

label.new(bar_index, high, text = labelText)
```

Note that:

- `for...in` loops can return a tuple containing each index and corresponding element. For example, `for [i, price] in a` returns the `i` index and `price` value for each element in `a`.

A while loop statement can also be used:

```
//@version=6
indicator("`while` loop", overlay = true)
array<float> a = request.security_lower_tf(syminfo.tickerid, "1", close)

string labelText = ""
int i = 0
while i < array.size(a)
    labelText += str.tostring(array.get(a, i)) + "\n"
    i += 1

label.new(bar_index, high, text = labelText)
```

Scope

Users can declare arrays within the global scope of a script, as well as the local scopes of functions, methods, and conditional structures. Unlike some of the other built-in types, namely *fundamental* types, scripts can modify globally-assigned arrays from within local scopes, allowing users to implement global variables that any function in the script can directly interact with. We use the functionality here to calculate progressively lower or higher price levels:

```
//@version=6
indicator("Bands", "", true)
//@variable The distance ratio between plotted price levels.
factorInput = 1 + (input.float(-2., "Step %") / 100)
//@variable A single-value array holding the lowest `ohlc4` value within a 50 bar window from 10 bars back.
level = array.new<float>(1, ta.lowest(ohlc4, 50)[10])
```



Figure 44: image

```
nextLevel(val) =>
    newLevel = level.get(0) * val
    // Write new level to the global `level` array so we can use it as the base in the next function call.
    level.set(0, newLevel)
    newLevel

plot(nextLevel(1))
plot(nextLevel(factorInput))
plot(nextLevel(factorInput))
plot(nextLevel(factorInput))
```

History referencing

Pine Script™'s history-referencing operator `[]` can access the history of array variables, allowing scripts to interact with past array instances previously assigned to a variable.

To illustrate this, let's create a simple example to show how one can fetch the previous bar's `close` value in two equivalent ways. This script uses the `[]` operator to get the array instance assigned to `a` on the previous bar, then uses the `get()` method to retrieve the value of the first element (`previousClose1`). For `previousClose2`, we use the history-referencing operator on the `close` variable directly to retrieve the value. As we see from the plots, `previousClose1` and `previousClose2` both return the same value:



Figure 45: image

```
//@version=6
indicator("History referencing")

//@variable A single-value array declared on each bar.
a = array.new<float>(1)
// Set the value of the only element in `a` to `close`.
array.set(a, 0, close)
```

```
//@variable The array instance assigned to `a` on the previous bar.
previous = a[1]

previousClose1 = na(previous) ? na : previous.get(0)
previousClose2 = close[1]

plot(previousClose1, "previousClose1", color.gray, 6)
plot(previousClose2, "previousClose2", color.white, 2)
```

Inserting and removing array elements

Inserting

The following three functions can insert new elements into an array.

`array.unshift()` inserts a new element at the beginning of an array (index 0) and increases the index values of any existing elements by one.

`array.insert()` inserts a new element at the specified **index** and increases the index of existing elements at or after the **index** by one.

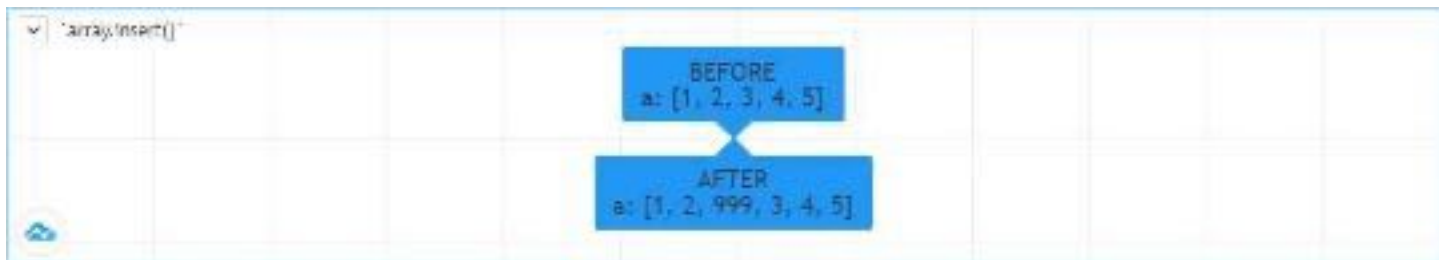


Figure 46: image

```
//@version=6
indicator("`array.insert()`")
a = array.new<float>(5, 0)
for i = 0 to 4
    array.set(a, i, i + 1)
if barstate.islast
    label.new(bar_index, 0, "BEFORE\na: " + str.tostring(a), size = size.large)
    array.insert(a, 2, 999)
    label.new(bar_index, 0, "AFTER\na: " + str.tostring(a), style = label.style_label_up, size = size.large)

array.push() adds a new element at the end of an array.
```

Removing

These four functions remove elements from an array. The first three also return the value of the removed element.

`array.remove()` removes the element at the specified **index** and returns that element's value.

`array.shift()` removes the first element from an array and returns its value.

`array.pop()` removes the last element of an array and returns its value.

`array.clear()` removes all elements from an array. Note that clearing an array won't delete any objects its elements referenced. See the example below that illustrates how this works:

```
//@version=6
indicator("`array.clear()` example", overlay = true)

// Create a label array and add a label to the array on each new bar.
var a = array.new<label>()
label lbl = label.new(bar_index, high, "Text", color = color.red)
array.push(a, lbl)
```



```

var table t = table.new(position.top_right, 1, 1)
// Clear the array on the last bar. This doesn't remove the labels from the chart.
if barstate.islast
    array.clear(a)
    table.cell(t, 0, 0, "Array elements count: " + str.tostring(array.size(a)), bgcolor = color.yellow)

```

Using an array as a stack

Stacks are LIFO (last in, first out) constructions. They behave somewhat like a vertical pile of books to which books can only be added or removed one at a time, always from the top. Pine Script™ arrays can be used as a stack, in which case we use the `array.push()` and `array.pop()` functions to add and remove elements at the end of the array.

`array.push(prices, close)` will add a new element to the end of the `prices` array, increasing the array's size by one.

`array.pop(prices)` will remove the end element from the `prices` array, return its value and decrease the array's size by one.

See how the functions are used here to track successive lows in rallies:



Figure 47: image

```

//@version=6
indicator("Lows from new highs", "", true)
var lows = array.new<float>(0)
flushLows = false

//@function Removes the last element from the `id` stack when `cond` is `true`.
array_pop(id, cond) => cond and array.size(id) > 0 ? array.pop(id) : float(na)

if ta.rising(high, 1)
    // Rising highs; push a new low on the stack.
    lows.push(low)
    // Force the return type of this `if` block to be the same as that of the next block.
    bool(na)
else if lows.size() >= 4 or low < array.min(lows)
    // We have at least 4 lows or price has breached the lowest low;
    // sort lows and set flag indicating we will plot and flush the levels.
    array.sort(lows, order.ascending)
    flushLows := true

// If needed, plot and flush lows.
lowLevel = array_pop(lows, flushLows)
plot(lowLevel, "Low 1", low > lowLevel ? color.silver : color.purple, 2, plot.style_linebr)
lowLevel := array_pop(lows, flushLows)
plot(lowLevel, "Low 2", low > lowLevel ? color.silver : color.purple, 3, plot.style_linebr)

```

```

lowLevel := array_pop(lows, flushLows)
plot(lowLevel, "Low 3", low > lowLevel ? color.silver : color.purple, 4, plot.style_linebr)
lowLevel := array_pop(lows, flushLows)
plot(lowLevel, "Low 4", low > lowLevel ? color.silver : color.purple, 5, plot.style_linebr)

if flushLows
    // Clear remaining levels after the last 4 have been plotted.
    lows.clear()

```

Using an array as a queue

Queues are FIFO (first in, first out) constructions. They behave somewhat like cars arriving at a red light. New cars are queued at the end of the line, and the first car to leave will be the first one that arrived to the red light.

In the following code example, we let users decide through the script's inputs how many labels they want to have on their chart. We use that quantity to determine the size of the array of labels we then create, initializing the array's elements to `na`.

When a new pivot is detected, we create a label for it, saving the label's ID in the `pLabel` variable. We then queue the ID of that label by using `array.push()` to append the new label's ID to the end of the array, making our array size one greater than the maximum number of labels to keep on the chart.

Lastly, we de-queue the oldest label by removing the array's first element using `array.shift()` and deleting the label referenced by that array element's value. As we have now de-queued an element from our queue, the array contains `pivotCountInput` elements once again. Note that on the dataset's first bars we will be deleting `na` label IDs until the maximum number of labels has been created, but this does not cause runtime errors. Let's look at our code:



Figure 48: image

```

//@version=6
MAX_LABELS = 100
indicator("Show Last n High Pivots", "", true, max_labels_count = MAX_LABELS)

pivotCountInput = input.int(5, "How many pivots to show", minval = 0, maxval = MAX_LABELS)
pivotLegsInput = input.int(3, "Pivot legs", minval = 1, maxval = 5)

// Create an array containing the user-selected max count of label IDs.
var labelIds = array.new<label>(pivotCountInput)

pHi = ta.pivohigh(pivotLegsInput, pivotLegsInput)
if not na(pHi)
    // New pivot found; plot its label `pivotLegsInput` bars behind the current `bar_index`.
    pLabel = label.new(bar_index - pivotLegsInput, pHi, str.tostring(pHi, format.mintick), textcolor = color.w)
    // Queue the new label's ID by appending it to the end of the array.
    array.push(labelIds, pLabel)

```

```
// De-queue the oldest label ID from the queue and delete the corresponding label.
label.delete(array.shift(labelIds))
```

Negative indexing

The `array.get()`, `array.set()`, `array.insert()`, and `array.remove()` functions support *negative indexing*, which references elements starting from the end of the array. An index of `-1` refers to the last element in the array, an index of `-2` refers to the second to last element, and so on.

When using a *positive* index, functions traverse the array *forwards* from the beginning of the array (*first to last* element). The first element's index is `0`, and the last element's index is `array.size() - 1`. When using a *negative* index, functions traverse the array *backwards* from the end of the array (*last to first* element). The last element's index is `-1`, and the first element's index is `-array.size()`:

```
array<string> myArray = array.from("first", "second", "third", "fourth", "last")
```

```
// Positive indexing: Indexes forwards from the beginning of the array.
```

```
myArray.get(0)           // Returns "first" element
myArray.get(myArray.size() - 1) // Returns "last" element
myArray.get(4)           // Returns "last" element
```

```
// Negative indexing: Indexes backwards from the end of the array.
```

```
myArray.get(-1)          // Returns "last" element
myArray.get(-myArray.size()) // Returns "first" element
myArray.get(-5)          // Returns "first" element
```

Like positive indexing, negative indexing is bound by the size of the array. For example, functions operating on an array of 5 elements only accept indices of `0` to `4` (first to last element) or `-1` to `-5` (last to first element). Any other indices are out of bounds and will raise a runtime error.

We can use negative indices to retrieve, update, add, and remove array elements. This simple script creates an “int” `countingArray` and calls the `array.get()`, `array.set()`, `array.insert()`, and `array.remove()` functions to perform various array operations using negative indices. It displays each array operation and its corresponding result in a table:

Negative indexing demo 

ARRAY OPERATION	RESULT
Initial `countingArray`	[10, 20, 30, 40, 50, 60]
`countingArray.get(0)`	10
`countingArray.get(-1)`	60
`countingArray.get(-countingArray.size())`	10
`countingArray.set(-2, 99)`	[10, 20, 30, 40, 99, 60]
`countingArray.insert(-5, 878)`	[10, 878, 20, 30, 40, 99, 60]
`countingArray.remove(-3)`	[10, 878, 20, 30, 99, 60]

 TradingView

Figure 49: image

```
//@version=6
```

```
indicator("Negative indexing demo", overlay = false)
```

```
//@variable A `table` that displays various array operations and their results.
```

```
var table displayTable = table.new(
    position.middle_center, 2, 15, bgcolor = color.white,
    frame_color = color.black, frame_width = 1, border_width = 1
)
```

```
//@function Initializes a `displayTable` row to output a "string" of an `arrayOperation` and the `operationRes
```

```

displayRow(int rowID, string arrayOperation, operationResult) =>
  //@variable Is white if the `rowID` is even, light blue otherwise. Used to set alternating table row color
  color rowColor = rowID % 2 == 0 ? color.white : color.rgb(33, 149, 243, 75)
  // Display the `arrayOperation` in the row's first cell.
  displayTable.cell(0, rowID, arrayOperation, text_color = color.black,
    text_halign = text.align_left, bgcolor = rowColor, text_font_family = font.family_monospace
  )
  // Display the `operationResult` in the row's second cell.
  displayTable.cell(1, rowID, str.toString(operationResult), text_color = color.black,
    text_halign = text.align_right, bgcolor = rowColor
  )

if barstate.islastconfirmedhistory
  //@variable Array of "int" numbers. Holds six multiples of 10, counting from 10 to 60.
  array<int> countingArray = array.from(10, 20, 30, 40, 50, 60)

  // Initialize the table's header cells.
  displayTable.cell(0, 0, "ARRAY OPERATION")
  displayTable.cell(1, 0, "RESULT")

  // Display the initial `countingArray` values.
  displayTable.cell(0, 1, "Initial `countingArray`",
    text_color = color.black, text_halign = text.align_center, bgcolor = color.yellow)
  displayTable.cell(1, 1, str.toString(countingArray),
    text_color = color.black, text_halign = text.align_right, bgcolor = color.yellow)

  // Retrieve array elements using negative indices in `array.get()`.
  displayRow(2, "`countingArray.get(0)`", countingArray.get(0))
  displayRow(3, "`countingArray.get(-1)`", countingArray.get(-1))
  displayRow(4, "`countingArray.get(-countingArray.size())`", countingArray.get(-countingArray.size()))

  // Update array elements using negative indices in `array.set()` and `array.insert()`.
  countingArray.set(-2, 99)
  displayRow(5, "`countingArray.set(-2, 99)`", countingArray)

  countingArray.insert(-5, 878)
  displayRow(6, "`countingArray.insert(-5, 878)`", countingArray)

  // Remove array elements using negative indices in `array.remove()`.
  countingArray.remove(-3)
  displayRow(7, "`countingArray.remove(-3)`", countingArray)

```

Note that not all array operations can use negative indices. For example, search functions like `array.indexof()` and `array.binary_search()` return the *positive* index of an element if it's found in the array. If the value is not found, the functions return `-1`. However, this returned value is **not** a negative index, and using it as one would incorrectly reference the last array element. If a script needs to use a search function's returned index in subsequent array operations, it must appropriately differentiate between this `-1` result and other valid indices.

Calculations on arrays

While series variables can be viewed as a horizontal set of values stretching back in time, Pine Script™'s one-dimensional arrays can be viewed as vertical structures residing on each bar. As an array's set of elements is not a time series, Pine Script™'s usual mathematical functions are not allowed on them. Special-purpose functions must be used to operate on all of an array's values. The available functions are: `array.abs()`, `array.avg()`, `array.covariance()`, `array.min()`, `array.max()`, `array.median()`, `array.mode()`, `array.percentile_linear_interpolation()`, `array.percentile_nearest_rank()`, `array.percentrank()`, `array.range()`, `array.standardize()`, `array.stdev()`, `array.sum()`, `array.variance()`.

Note that contrary to the usual mathematical functions in Pine Script™, those used on arrays do not return `na` when some of the values they calculate on have `na` values. There are a few exceptions to this rule:

- When all array elements have `na` value or the array contains no elements, `na` is returned. `array.standardize()` however, will return an empty array.

- `array.mode()` will return `na` when no mode is found.

Manipulating arrays

Concatenation

Two arrays can be merged — or concatenated — using `array.concat()`. When arrays are concatenated, the second array is appended to the end of the first, so the first array is modified while the second one remains intact. The function returns the array ID of the first array:

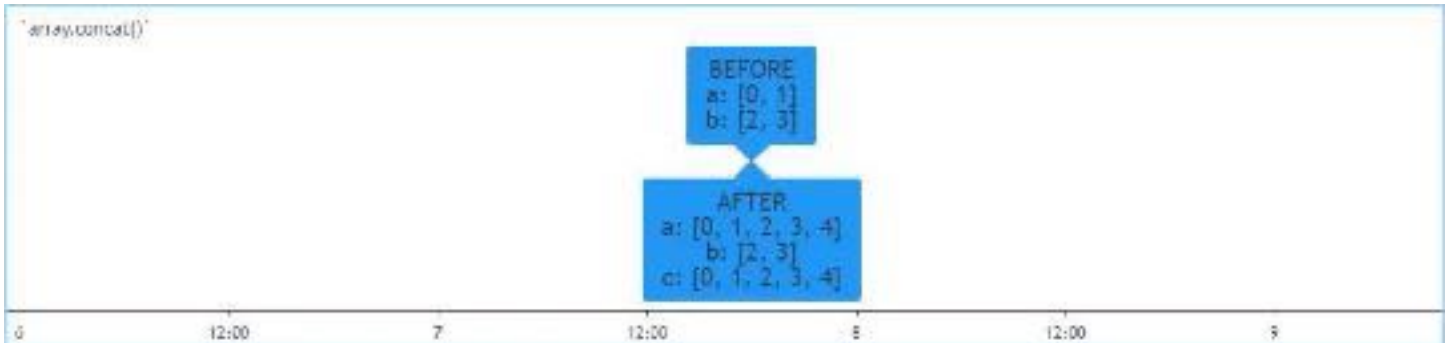


Figure 50: image

```

//@version=6
indicator("`array.concat()`")
a = array.new<float>(0)
b = array.new<float>(0)
array.push(a, 0)
array.push(a, 1)
array.push(b, 2)
array.push(b, 3)
if barstate.islast
    label.new(bar_index, 0, "BEFORE\na: " + str.tostring(a) + "\nb: " + str.tostring(b), size = size.large)
    c = array.concat(a, b)
    array.push(c, 4)
    label.new(bar_index, 0, "AFTER\na: " + str.tostring(a) + "\nb: " + str.tostring(b) + "\nc: " + str.tostring(c))

```

Copying

You can copy an array using `array.copy()`. Here we copy the array `a` to a new array named `_b`:



Figure 51: image

```

//@version=6
indicator("`array.copy()`")
a = array.new<float>(0)
array.push(a, 0)
array.push(a, 1)
if barstate.islast
    b = array.copy(a)
    array.push(b, 2)

```

```
label.new(bar_index, 0, "a: " + str.tostring(a) + "\nb: " + str.tostring(b), size = size.large)
```

Note that simply using `_b = a` in the previous example would not have copied the array, but only its ID. From thereon, both variables would point to the same array, so using either one would affect the same array.

Joining

Use `array.join()` to concatenate all of the elements in the array into a string and separate these elements with the specified separator:

```
//@version=6
indicator("")
v1 = array.new<string>(10, "test")
v2 = array.new<string>(10, "test")
array.push(v2, "test1")
v3 = array.new_float(5, 5)
v4 = array.new_int(5, 5)
l1 = label.new(bar_index, close, array.join(v1))
l2 = label.new(bar_index, close, array.join(v2, ","))
l3 = label.new(bar_index, close, array.join(v3, ","))
l4 = label.new(bar_index, close, array.join(v4, ","))
```

Sorting

Arrays containing “int” or “float” elements can be sorted in either ascending or descending order using `array.sort()`. The `order` parameter is optional and defaults to `order.ascending`. As all `array.*()` function arguments, it is qualified as “series”, so can be determined at runtime, as is done here. Note that in the example, which array is sorted is also determined at runtime:



Figure 52: image

```
//@version=6
indicator("`array.sort()`")
a = array.new<float>(0)
b = array.new<float>(0)
array.push(a, 2)
array.push(a, 0)
```

```

array.push(a, 1)
array.push(b, 4)
array.push(b, 3)
array.push(b, 5)
if barstate.islast
    barUp = close > open
    array.sort(barUp ? a : b, barUp ? order.ascending : order.descending)
    label.new(bar_index, 0,
        "a " + (barUp ? "is sorted : " : "is not sorted: ") + str.tostring(a) + "\n\n" +
        "b " + (barUp ? "is not sorted: " : "is sorted : ") + str.tostring(b), size = size.large)

```

Another useful option for sorting arrays is to use the `array.sort_indices()` function, which takes a reference to the original array and returns an array containing the indices from the original array. Please note that this function won't modify the original array. The `order` parameter is optional and defaults to `order.ascending`.

Reversing

Use `array.reverse()` to reverse an array:

```

//@version=6
indicator("`array.reverse()`")
a = array.new<float>(0)
array.push(a, 0)
array.push(a, 1)
array.push(a, 2)
if barstate.islast
    array.reverse(a)
    label.new(bar_index, 0, "a: " + str.tostring(a))

```

Slicing

Slicing an array using `array.slice()` creates a shallow copy of a subset of the parent array. You determine the size of the subset to slice using the `index_from` and `index_to` parameters. The `index_to` argument must be one greater than the end of the subset you want to slice.

The shallow copy created by the slice acts like a window on the parent array's content. The indices used for the slice define the window's position and size over the parent array. If, as in the example below, a slice is created from the first three elements of an array (indices 0 to 2), then regardless of changes made to the parent array, and as long as it contains at least three elements, the shallow copy will always contain the parent array's first three elements.

Additionally, once the shallow copy is created, operations on the copy are mirrored on the parent array. Adding an element to the end of the shallow copy, as is done in the following example, will widen the window by one element and also insert that element in the parent array at index 3. In this example, to slice the subset from index 0 to index 2 of array `a`, we must use `sliceOfA = array.slice(a, 0, 3)`:

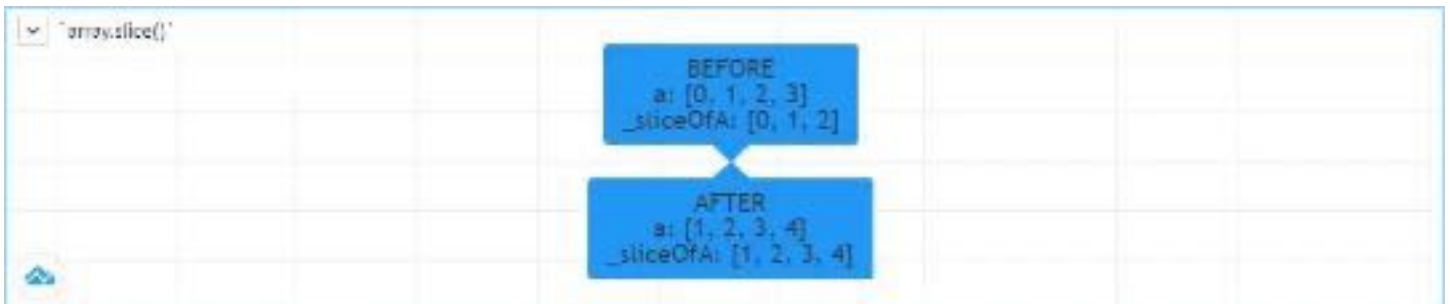


Figure 53: image

```

//@version=6
indicator("`array.slice()`")
a = array.new<float>(0)
array.push(a, 0)
array.push(a, 1)

```

```

array.push(a, 2)
array.push(a, 3)
if barstate.islast
    // Create a shadow of elements at index 1 and 2 from array `a`.
    sliceOfA = array.slice(a, 0, 3)
    label.new(bar_index, 0, "BEFORE\na: " + str.tostring(a) + "\nsliceOfA: " + str.tostring(sliceOfA))
    // Remove first element of parent array `a`.
    array.remove(a, 0)
    // Add a new element at the end of the shallow copy, thus also affecting the original array `a`.
    array.push(sliceOfA, 4)
    label.new(bar_index, 0, "AFTER\na: " + str.tostring(a) + "\nsliceOfA: " + str.tostring(sliceOfA), style =

```

Searching arrays

We can test if a value is part of an array with the `array.includes()` function, which returns true if the element is found. We can find the first occurrence of a value in an array by using the `array.indexof()` function. The first occurrence is the one with the lowest index. We can also find the last occurrence of a value with `array.lastindexof()`:

```

//@version=6
indicator("Searching in arrays")
valueInput = input.int(1)
a = array.new<float>(0)
array.push(a, 0)
array.push(a, 1)
array.push(a, 2)
array.push(a, 1)
if barstate.islast
    valueFound      = array.includes(a, valueInput)
    firstIndexFound = array.indexof(a, valueInput)
    lastIndexFound  = array.lastindexof(a, valueInput)
    label.new(bar_index, 0, "a: " + str.tostring(a) +
        "\nFirst " + str.tostring(valueInput) + (firstIndexFound != -1 ? " value was found at index: " + str.tostring(firstIndexFound) : " not found") +
        "\nLast  " + str.tostring(valueInput) + (lastIndexFound  != -1 ? " value was found at index: " + str.tostring(lastIndexFound) : " not found")

```

We can also perform a binary search on an array but note that performing a binary search on an array means that the array will first need to be sorted in ascending order only. The `array.binary_search()` function will return the value's index if it was found or -1 if it wasn't. If we want to always return an existing index from the array even if our chosen value wasn't found, then we can use one of the other binary search functions available. The `array.binary_search_leftmost()` function, which returns an index if the value was found or the first index to the left where the value would be found. The `array.binary_search_rightmost()` function is almost identical and returns an index if the value was found or the first index to the right where the value would be found.

Error handling

Malformed `array.*()` call syntax in Pine scripts will cause the usual **compiler** error messages to appear in Pine Editor's console, at the bottom of the window, when you save a script. Refer to the Pine Script™ v6 Reference Manual when in doubt regarding the exact syntax of function calls.

Scripts using arrays can also throw **runtime** errors, which appear as an exclamation mark next to the indicator's name on the chart. We discuss those runtime errors in this section.

Index xx is out of bounds. Array size is yy

This error is the most frequent one programmers encounter when using arrays. The error occurs when the script references a *nonexistent* array index. The "xx" value represents the out-of-bounds index the function tried to use, and "yy" represents the array's size. Recall that array indices start at zero — not one — and end at the array's size, minus one. For instance, the last valid index in a three-element array is 2.

To avoid this error, you must make provisions in your code logic to prevent using an index value outside the array's boundaries. This code example generates the error because the last `i` value in the loop's iterations is beyond the valid index range for the `a` array:

```

//@version=6

```



```

indicator("Out of bounds index")
a = array.new<float>(3)
for i = 1 to 3
    array.set(a, i, i)
plot(array.pop(a))

```

To resolve the error, last *i* value in the loop statement should be less than or equal to 2:

```

for i = 0 to 2

```

To iterate over all elements in an array of *unknown* size with a for loop, set the loop counter's final value to one less than the `array.size()` value:

```

//@version=6
indicator("Protected `for` loop")
sizeInput = input.int(0, "Array size", minval = 0, maxval = 100000)
a = array.new<float>(sizeInput)
for i = 0 to (array.size(a) == 0 ? na : array.size(a) - 1)
    array.set(a, i, i)
plot(array.pop(a))

```

When sizing arrays dynamically using a field in the script's *Settings/Inputs* tab, protect the boundaries of that value using `input.int()`'s `minval` and `maxval` parameters:

```

//@version=6
indicator("Protected array size")
sizeInput = input.int(10, "Array size", minval = 1, maxval = 100000)
a = array.new<float>(sizeInput)
for i = 0 to sizeInput - 1
    array.set(a, i, i)
plot(array.size(a))

```

See the Looping through array elements section of this page for more information.

Cannot call array methods when ID of array is 'na'

When an array ID is initialized to `na`, operations on it are not allowed, since no array exists. All that exists at that point is an array variable containing the `na` value rather than a valid array ID pointing to an existing array. Note that an array created with no elements in it, as you do when you use `a = array.new_int(0)`, has a valid ID nonetheless. This code will throw the error we are discussing:

```

//@version=6
indicator("Array methods on `na` array")
array<int> a = na
array.push(a, 111)
label.new(bar_index, 0, "a: " + str.tostring(a))

```

To avoid it, create an array with size zero using:

```

array<int> a = array.new_int(0)

```

or:

```

a = array.new_int(0)

```

Array is too large. Maximum size is 100000

This error will appear if your code attempts to declare an array with a size greater than 100,000. It will also occur if, while dynamically appending elements to an array, a new element would increase the array's size past the maximum.

Cannot create an array with a negative size

We haven't found any use for arrays of negative size yet, but if you ever do, we may allow them :)

Cannot use `shift()` if array is empty.

This error will occur if `array.shift()` is called to remove the first element of an empty array.

Cannot use pop() if array is empty.

This error will occur if `array.pop()` is called to remove the last element of an empty array.

Index ‘from’ should be less than index ‘to’

When two indices are used in functions such as `array.slice()`, the first index must always be smaller than the second one.

Slice is out of bounds of the parent array

This message occurs whenever the parent array’s size is modified in such a way that it makes the shallow copy created by a slice point outside the boundaries of the parent array. This code will reproduce it because after creating a slice from index 3 to 4 (the last two elements of our five-element parent array), we remove the parent’s first element, making its size four and its last index 3. From that moment on, the shallow copy which is still pointing to the “window” at the parent array’s indices 3 to 4, is pointing out of the parent array’s boundaries:

```
//@version=6
indicator("Slice out of bounds")
a = array.new<float>(5, 0)
b = array.slice(a, 3, 5)
array.remove(a, 0)
c = array.indexof(b, 2)
plot(c)
```

[\[Previous](#)

[Methods\]\(#methods\)\[Next](#)

[Matrices\]\(#matrices\) User Manual/Language/Matrices](#)

ADVANCED

Matrices

Introduction

Pine Script™ Matrices are collections that store value references in a rectangular format. They are the equivalent of two-dimensional array objects with functions and methods for inspection, modification, and specialized calculations. As with arrays, all matrix elements must be of the same type, user-defined type, or enum type.

Matrices reference their elements using *two* indices: one index for their *rows* and the other for their *columns*. Each index starts at 0 and extends to the number of rows/columns in the matrix minus one. Matrices in Pine can have dynamic numbers of rows and columns that vary across bars. The total number of elements within a matrix is the *product* of the number of rows and columns (e.g., a 5x5 matrix has a total of 25). Like arrays, the total number of elements in a matrix cannot exceed 100,000.

Declaring a matrix

Pine Script™ uses the following syntax for matrix declaration:

```
[var/varip ] [matrix<type> ]<identifier> = <expression>
```

Where `<type>` is a type template for the matrix that declares the type of values it will contain, and the `<expression>` returns either a matrix instance of the type or `na`.

When declaring a matrix variable as `na`, users must specify that the identifier will reference matrices of a specific type by including the matrix keyword followed by a type template.

This line declares a new `myMatrix` variable with a value of `na`. It explicitly declares the variable as `matrix<float>`, which tells the compiler that the variable can only accept matrix objects containing float values:

```
matrix<float> myMatrix = na
```

When a matrix variable is not assigned to `na`, the matrix keyword and its type template are optional, as the compiler will use the type information from the object the variable references.

Here, we declare a `myMatrix` variable referencing a new `matrix<float>` instance with two rows, two columns, and an `initial_value` of 0. The variable gets its type information from the new object in this case, so it doesn't require an explicit type declaration:

```
myMatrix = matrix.new<float>(2, 2, 0.0)
```

Using `var` and `varip` keywords

As with other variables, users can include the `var` or `varip` keywords to instruct a script to declare a matrix variable only once rather than on every bar. A matrix variable declared with this keyword will point to the same instance throughout the span of the chart unless the script explicitly assigns another matrix to it, allowing a matrix and its element references to persist between script iterations.

This script declares an `m` variable assigned to a matrix that holds a single row of two `int` elements using the `var` keyword. On every 20th bar, the script adds 1 to the first element on the first row of the `m` matrix. The `plot()` call displays this element on the chart. As we see from the plot, the value of `m.get(0, 0)` persists between bars, never returning to the initial value of 0:



Figure 54: image

```
//@version=6
indicator("var matrix demo")

//@variable A 1x2 rectangular matrix declared only at `bar_index == 0`, i.e., the first bar.
var m = matrix.new<int>(1, 2, 0)

//@variable Is `true` on every 20th bar.
bool update = bar_index % 20 == 0

if update
    int currentValue = m.get(0, 0) // Get the current value of the first row and column.
    m.set(0, 0, currentValue + 1) // Set the first row and column element value to `currentValue + 1`.

plot(m.get(0, 0), linewidth = 3) // Plot the value from the first row and column.
```

Reading and writing matrix elements

`matrix.get()` and `matrix.set()`

To retrieve the value from a matrix at a specified `row` and `column` index, use `matrix.get()`. This function locates the specified matrix element and returns its value. Similarly, to overwrite a specific element's value, use `matrix.set()` to assign the element at the specified `row` and `column` to a new `value`.

The example below defines a square matrix `m` with two rows and columns and an `initial_value` of 0 for all elements on the first bar. The script adds 1 to each element's value on different bars using the `m.get()` and `m.set()` methods. It updates the first row's first value once every 11 bars, the first row's second value once every seven bars, the second row's first value once every five bars, and the second row's second value once every three bars. The script plots each element's value on the chart:

```
//@version=6
indicator("Reading and writing elements demo")
```

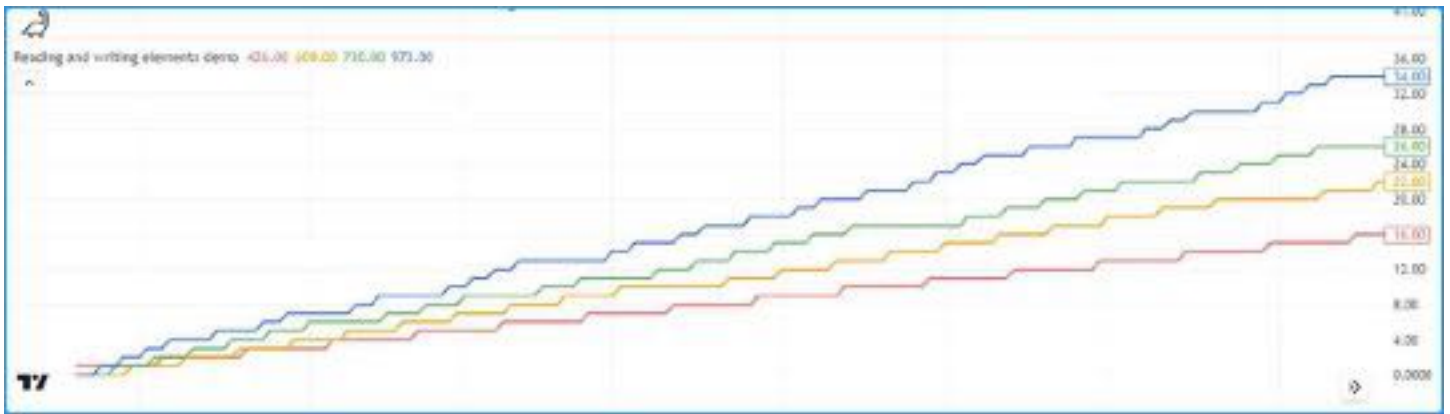


Figure 55: image

```
//@variable A 2x2 square matrix of `float` values.
var m = matrix.new<float>(2, 2, 0.0)

switch
    bar_index % 11 == 0 => m.set(0, 0, m.get(0, 0) + 1.0) // Adds 1 to the value at row 0, column 0 every 11th
    bar_index % 7 == 0 => m.set(0, 1, m.get(0, 1) + 1.0) // Adds 1 to the value at row 0, column 1 every 7th
    bar_index % 5 == 0 => m.set(1, 0, m.get(1, 0) + 1.0) // Adds 1 to the value at row 1, column 0 every 5th
    bar_index % 3 == 0 => m.set(1, 1, m.get(1, 1) + 1.0) // Adds 1 to the value at row 1, column 1 every 3rd

plot(m.get(0, 0), "Row 0, Column 0 Value", color.red, 2)
plot(m.get(0, 1), "Row 0, Column 1 Value", color.orange, 2)
plot(m.get(1, 0), "Row 1, Column 0 Value", color.green, 2)
plot(m.get(1, 1), "Row 1, Column 1 Value", color.blue, 2)

matrix.fill()
```

To overwrite all matrix elements with a specific value, use `matrix.fill()`. This function points all items in the entire matrix or within the `from_row/column` and `to_row/column` index range to the `value` specified in the call. For example, this snippet declares a 4x4 square matrix, then fills its elements with a random value:

```
myMatrix = matrix.new<float>(4, 4)
myMatrix.fill(math.random())
```

Note when using `matrix.fill()` with matrices containing special types (line, linefill, box, polyline, label, table, or `chart.point`) or UDTs, all replaced elements will point to the same object passed in the function call.

This script declares a matrix with four rows and columns of label references, which it fills with a new label object on the first bar. On each bar, the script sets the `x` attribute of the label referenced at row 0, column 0 to `bar_index`, and the `text` attribute of the one referenced at row 3, column 3 to the number of labels on the chart. Although the matrix can reference 16 (4x4) labels, each element points to the *same* instance, resulting in only one label on the chart that updates its `x` and `text` attributes on each bar:

```
//@version=6
indicator("Object matrix fill demo")

//@variable A 4x4 label matrix.
var matrix<label> m = matrix.new<label>(4, 4)

// Fill `m` with a new label object on the first bar.
if bar_index == 0
    m.fill(label.new(0, 0, textcolor = color.white, size = size.huge))

//@variable The number of label objects on the chart.
int numLabels = label.all.size()
```



Figure 56: image

```
// Set the `x` of the label from the first row and column to `bar_index`.
m.get(0, 0).set_x(bar_index)
// Set the `text` of the label at the last row and column to the number of labels.
m.get(3, 3).set_text(str.format("Total labels on the chart: {0}", numLabels))
```

Rows and columns

Retrieving

Matrices facilitate the retrieval of all values from a specific row or column via the `matrix.row()` and `matrix.col()` functions. These functions return the values as an array object sized according to the other dimension of the matrix, i.e., the size of a `matrix.row()` array equals the number of columns and the size of a `matrix.col()` array equals the number of rows.

The script below populates a 3x2 m matrix with the values 1 - 6 on the first chart bar. It calls the `m.row()` and `m.col()` methods to access the first row and column arrays from the matrix and displays them on the chart in a label along with the array sizes:



Figure 57: image

```
//@version=6
indicator("Retrieving rows and columns demo")

//@variable A 3x2 rectangular matrix.
var matrix<float> m = matrix.new<float>(3, 2)
```

```

if bar_index == 0
    m.set(0, 0, 1.0) // Set row 0, column 0 value to 1.
    m.set(0, 1, 2.0) // Set row 0, column 1 value to 2.
    m.set(1, 0, 3.0) // Set row 1, column 0 value to 3.
    m.set(1, 1, 4.0) // Set row 1, column 1 value to 4.
    m.set(2, 0, 5.0) // Set row 2, column 0 value to 5.
    m.set(2, 1, 6.0) // Set row 2, column 1 value to 6.

//@variable The first row of the matrix.
array<float> row0 = m.row(0)
//@variable The first column of the matrix.
array<float> column0 = m.col(0)

//@variable Displays the first row and column of the matrix and their sizes in a label.
var label debugLabel = label.new(0, 0, color = color.blue, textcolor = color.white, size = size.huge)
debugLabel.set_x(bar_index)
debugLabel.set_text(str.format("Row 0: {0}, Size: {1}\nCol 0: {2}, Size: {3}", row0, m.columns(), column0, m.r

```

Note that:

- To get the sizes of the arrays displayed in the label, we used the `rows()` and `columns()` methods rather than `array.size()` to demonstrate that the size of the `row0` array equals the number of columns and the size of the `column0` array equals the number of rows.

`matrix.row()` and `matrix.col()` copy the references in a row/column to a new array. Modifications to the arrays returned by these functions do not directly affect the elements or the shape of a matrix.

Here, we've modified the previous script to set the first element of `row0` to 10 via the `array.set()` method before displaying the label. This script also plots the value from row 0, column 0. As we see, the label shows that the first element of the `row0` array is 10. However, the plot shows that the corresponding matrix element still has a value of 1:

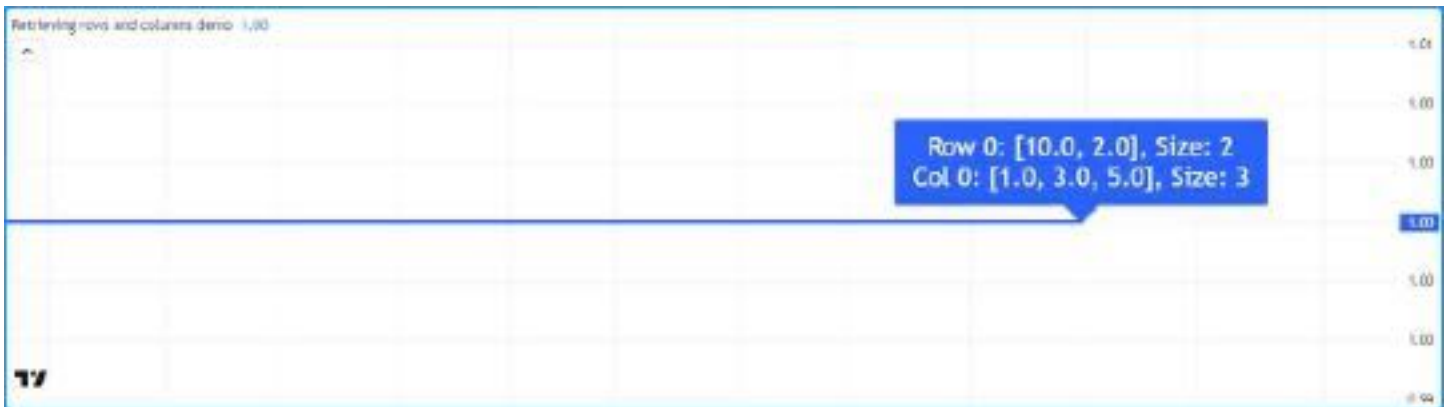


Figure 58: image

```

//@version=6
indicator("Retrieving rows and columns demo")

//@variable A 3x2 rectangular matrix.
var matrix<float> m = matrix.new<float>(3, 2)

if bar_index == 0
    m.set(0, 0, 1.0) // Set row 0, column 0 value to 1.
    m.set(0, 1, 2.0) // Set row 0, column 1 value to 2.
    m.set(1, 0, 3.0) // Set row 1, column 0 value to 3.
    m.set(1, 1, 4.0) // Set row 1, column 1 value to 4.
    m.set(2, 0, 5.0) // Set row 1, column 0 value to 5.
    m.set(2, 1, 6.0) // Set row 1, column 1 value to 6.

```

```

//@variable The first row of the matrix.
array<float> row0 = m.row(0)
//@variable The first column of the matrix.
array<float> column0 = m.col(0)

// Set the first `row` element to 10.
row0.set(0, 10)

//@variable Displays the first row and column of the matrix and their sizes in a label.
var label debugLabel = label.new(0, m.get(0, 0), color = color.blue, textcolor = color.white, size = size.huge)
debugLabel.set_x(bar_index)
debugLabel.set_text(str.format("Row 0: {0}, Size: {1}\nCol 0: {2}, Size: {3}", row0, m.columns(), column0, m.r

// Plot the first element of `m`.
plot(m.get(0, 0), linewidth = 3)

```

Although changes to an array returned by `matrix.row()` or `matrix.col()` do not directly affect a parent matrix, it's important to note the resulting array from a matrix containing UDTs or special types, including `line`, `linefill`, `box`, `polyline`, `label`, `table`, or `chart.point`, behaves as a *shallow copy* of a row/column, i.e., the elements within an array returned from these functions point to the same objects as the corresponding matrix elements.

This script contains a custom `myUDT` type containing a `value` field with an initial value of 0. It declares a 1x1 `m` matrix to hold a single `myUDT` instance on the first bar, then calls `m.row(0)` to copy the first row of the matrix as an array. On every chart bar, the script adds 1 to the `value` field of the first `row` array element. In this case, the `value` field of the matrix element increases on every bar as well since both elements reference the same object:

```

//@version=6
indicator("Row with reference types demo")

//@type A custom type that holds a float value.
type myUDT
    float value = 0.0

//@variable A 1x1 matrix of `myUDT` type.
var matrix<myUDT> m = matrix.new<myUDT>(1, 1, myUDT.new())
//@variable A shallow copy of the first row of `m`.
array<myUDT> row = m.row(0)
//@variable The first element of the `row`.
myUDT firstElement = row.get(0)

firstElement.value += 1.0 // Add 1 to the `value` field of `firstElement`. Also affects the element in the mat

plot(m.get(0, 0).value, linewidth = 3) // Plot the `value` of the `myUDT` object from the first row and column

```

Inserting

Scripts can add new rows and columns to a matrix via `matrix.add_row()` and `matrix.add_col()`. These functions insert the value references from an array into a matrix at the specified `row/column` index. If the `id` matrix is empty (has no rows or columns), the `array_id` in the call can be of any size. If a row/column exists at the specified index, the matrix increases the index value for the existing row/column and all after it by 1.

The script below declares an empty `m` matrix and inserts rows and columns using the `m.add_row()` and `m.add_col()` methods. It first inserts an array with three elements at row 0, turning `m` into a 1x3 matrix, then another at row 1, changing the shape to 2x3. After that, the script inserts another array at row 0, which changes the shape of `m` to 3x3 and shifts the index of all rows previously at index 0 and higher. It inserts another array at the last column index, changing the shape to 3x4. Finally, it adds an array with four values at the end row index.

The resulting matrix has four rows and columns and contains values 1-16 in ascending order. The script displays the rows of `m` after each row/column insertion with a user-defined `debugLabel()` function to visualize the process:

```

//@version=6
indicator("Rows and columns demo")

```



Figure 59: image

```
//@function Displays the rows of a matrix in a label with a note.
//@param    this The matrix to display.
//@param    barIndex The `bar_index` to display the label at.
//@param    bgColor The background color of the label.
//@param    textColor The color of the label's text.
//@param    note The text to display above the rows.
method debugLabel(
    matrix<float> this, int barIndex = bar_index, color bgColor = color.blue,
    color textColor = color.white, string note = ""
) =>
    labelText = note + "\n" + str.toString(this)
    if barstate.ishistory
        label.new(
            barIndex, 0, labelText, color = bgColor, style = label.style_label_center,
            textcolor = textColor, size = size.huge
        )

//Create an empty matrix.
var m = matrix.new<float>()

if bar_index == last_bar_index - 1
    debugLabel(m, bar_index - 30, note = "Empty matrix")

// Insert an array at row 0. `m` will now have 1 row and 3 columns.
m.add_row(0, array.from(5, 6, 7))
debugLabel(m, bar_index - 20, note = "New row at\nindex 0")

// Insert an array at row 1. `m` will now have 2 rows and 3 columns.
m.add_row(1, array.from(9, 10, 11))
debugLabel(m, bar_index - 10, note = "New row at\nindex 1")

// Insert another array at row 0. `m` will now have 3 rows and 3 columns.
// The values previously on row 0 will now be on row 1, and the values from row 1 will be on row 2.
m.add_row(0, array.from(1, 2, 3))
debugLabel(m, bar_index, note = "New row at\nindex 0")

// Insert an array at column 3. `m` will now have 3 rows and 4 columns.
m.add_col(3, array.from(4, 8, 12))
debugLabel(m, bar_index + 10, note = "New column at\nindex 3")

// Insert an array at row 3. `m` will now have 4 rows and 4 columns.
m.add_row(3, array.from(13, 14, 15, 16))
```



```
debugLabel(m, bar_index + 20, note = "New row at\nindex 3")
```

Removing

To remove a specific row or column from a matrix, use `matrix.remove_row()` and `matrix.remove_col()`. These functions remove the specified row/column and decrease the index values of all rows/columns after it by 1.

For this example, we've added these lines of code to our "Rows and columns demo" script from the section above:

```
// Removing example

// Remove the first row and last column from the matrix. `m` will now have 3 rows and 3 columns.
m.remove_row(0)
m.remove_col(3)
debugLabel(m, bar_index + 30, color.red, note = "Removed row 0\nand column 3")
```

This code removes the first row and the last column of the `m` matrix using the `m.remove_row()` and `m.remove_col()` methods and displays the rows in a label at `bar_index + 30`. As we can see, `m` has a 3x3 shape after executing this block, and the index values for all existing rows are reduced by 1:

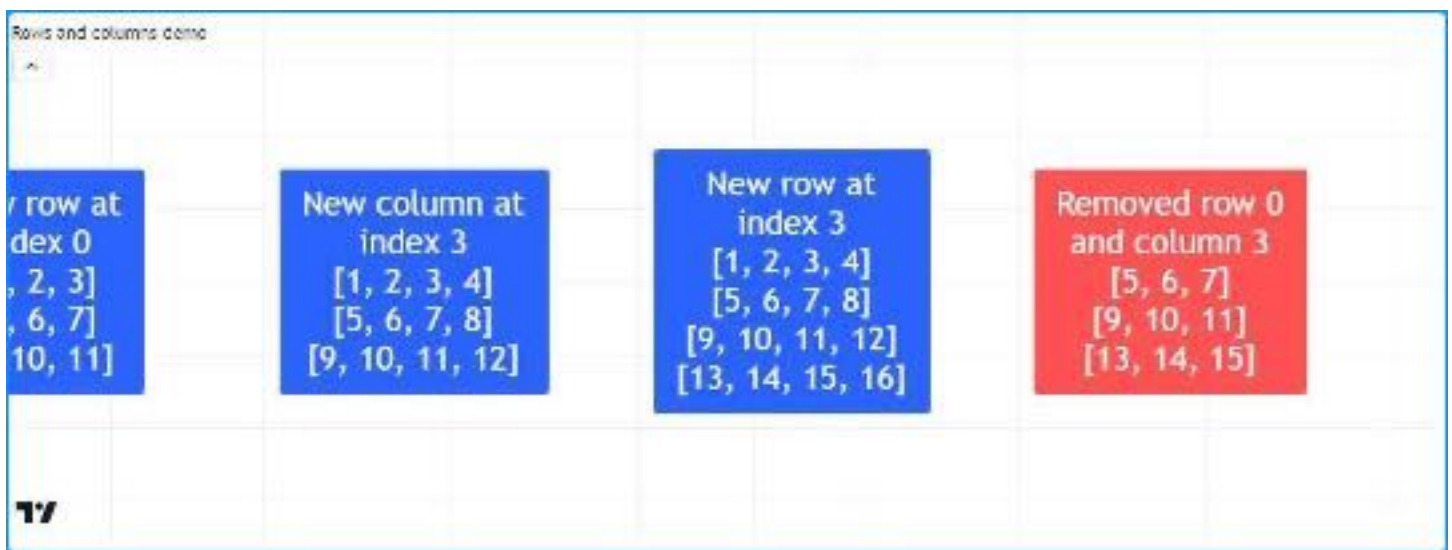


Figure 60: image

Swapping

To swap the rows and columns of a matrix without altering its dimensions, use `matrix.swap_rows()` and `matrix.swap_columns()`. These functions swap the locations of the elements at the `row1/column1` and `row2/column2` indices.

Let's add the following lines to the previous example, which swap the first and last rows of `m` and display the changes in a label at `bar_index + 40`:

```
// Swapping example

// Swap the first and last row. `m` retains the same dimensions.
m.swap_rows(0, 2)
debugLabel(m, bar_index + 40, color.purple, note = "Swapped rows 0\nand 2")
```

In the new label, we see the matrix has the same number of rows as before, and the first and last rows have traded places:

Replacing

It may be desirable in some cases to completely *replace* a row or column in a matrix. To do so, insert the new array at the desired `row/column` and remove the old elements previously at that index.

In the following code, we've defined a `replaceRow()` method that uses the `add_row()` method to insert the new `values` at the `row` index and uses the `remove_row()` method to remove the old row that moved to the `row + 1` index. This script uses

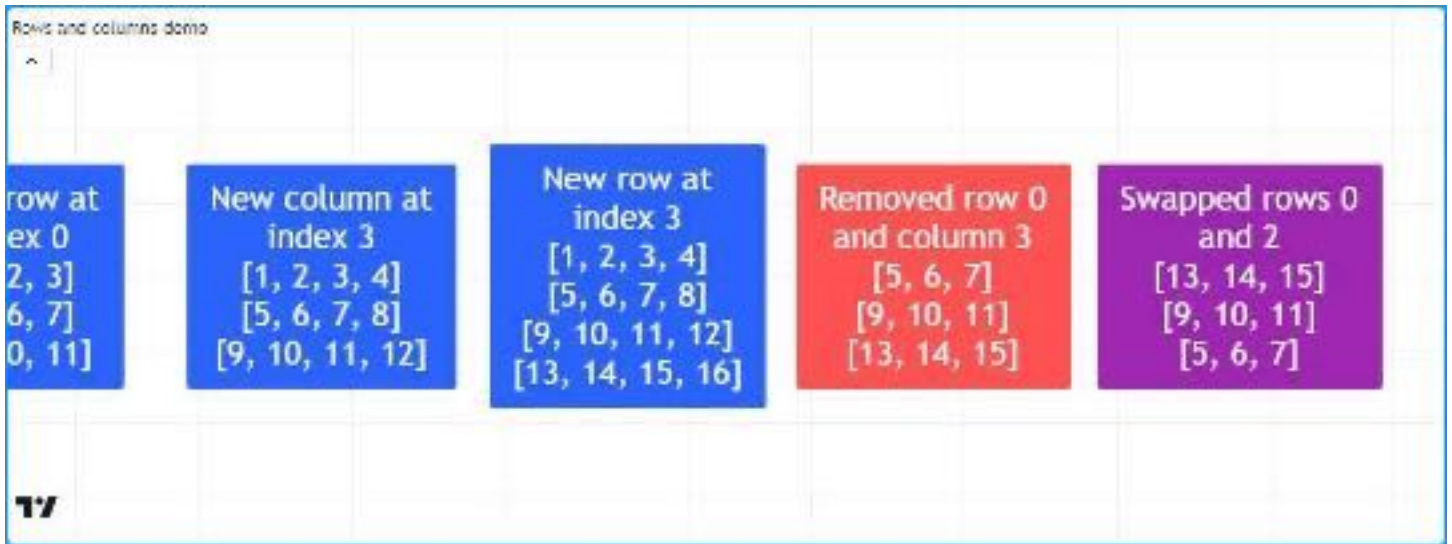


Figure 61: image

the `replaceRow()` method to fill the rows of a 3x3 matrix with the numbers 1-9. It draws a label on the chart before and after replacing the rows using the custom `debugLabel()` method:

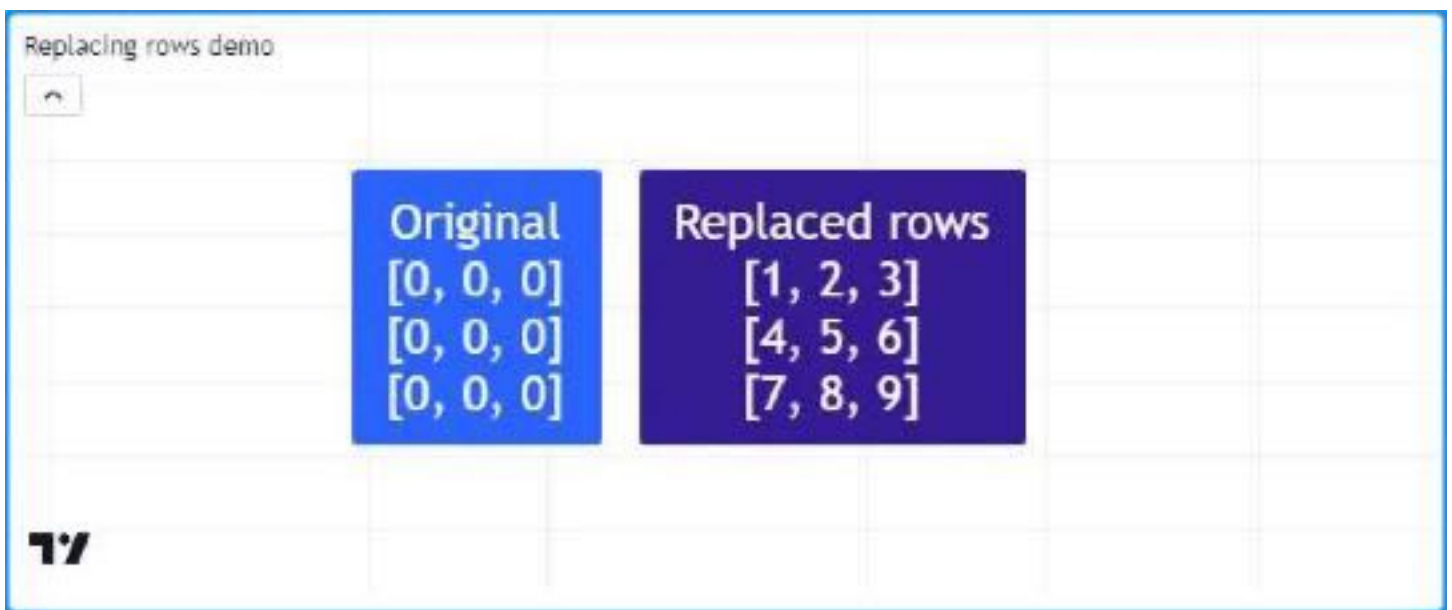


Figure 62: image

```
//@version=6
indicator("Replacing rows demo")

//@function Displays the rows of a matrix in a label with a note.
//@param this The matrix to display.
//@param barIndex The `bar_index` to display the label at.
//@param bgColor The background color of the label.
//@param textColor The color of the label's text.
//@param note The text to display above the rows.
method debugLabel(
    matrix<float> this, int barIndex = bar_index, color bgColor = color.blue,
    color textColor = color.white, string note = ""
) =>
    labelText = note + "\n" + str.toString(this)
    if barstate.ishistory
```

```

label.new(
    barIndex, 0, labelText, color = bgColor, style = label.style_label_center,
    textcolor = textColor, size = size.huge
)

//@function Replaces the `row` of `this` matrix with a new array of `values`.
//@param    row The row index to replace.
//@param    values The array of values to insert.
method replaceRow(matrix<float> this, int row, array<float> values) =>
    this.add_row(row, values) // Inserts a copy of the `values` array at the `row`.
    this.remove_row(row + 1) // Removes the old elements previously at the `row`.

//@variable A 3x3 matrix.
var matrix<float> m = matrix.new<float>(3, 3, 0.0)

if bar_index == last_bar_index - 1
    m.debugLabel(note = "Original")
    // Replace each row of `m`.
    m.replaceRow(0, array.from(1.0, 2.0, 3.0))
    m.replaceRow(1, array.from(4.0, 5.0, 6.0))
    m.replaceRow(2, array.from(7.0, 8.0, 9.0))
    m.debugLabel(bar_index + 10, note = "Replaced rows")

```

Looping through a matrix

for

When a script only needs to iterate over the row/column indices in a matrix, the most common method is to use for loops. For example, this line creates a loop with a `row` value that starts at 0 and increases by one until it reaches one less than the number of rows in the `m` matrix (i.e., the last row index):

```
for row = 0 to m.rows() - 1
```

To iterate over all index values in the `m` matrix, we can create a *nested* loop that iterates over each `column` index on each `row` value:

```
for row = 0 to m.rows() - 1
    for column = 0 to m.columns() - 1
```

Let's use this nested structure to create a method that visualizes matrix elements. In the script below, we've defined a `toTable()` method that displays the elements of a matrix within a table object. It iterates over each `row` index and over each `column` index on every row. Within the loop, it converts each element to a string to display in the corresponding table cell.

On the first bar, the script creates an empty `m` matrix, populates it with rows, and calls `m.toTable()` to display its elements:



Figure 63: image

```
//@version=6
```

```

indicator("for loop demo", "Matrix to table")

//@function Displays the elements of `this` matrix in a table.
//@param    this The matrix to display.
//@param    position The position of the table on the chart.
//@param    bgColor The background color of the table.
//@param    textColor The color of the text in each cell.
//@param    note A note string to display on the bottom row of the table.
//@returns  A new `table` object with cells corresponding to each element of `this` matrix.
method toTable(
    matrix<float> this, string position = position.middle_center,
    color bgColor = color.blue, color textColor = color.white,
    string note = na
) =>
    //@variable The number of rows in `this` matrix.
    int rows = this.rows()
    //@variable The number of columns in `this` matrix.
    int columns = this.columns()
    //@variable A table that displays the elements of `this` matrix with an optional `note` cell.
    table result = table.new(position, columns, rows + 1, bgColor)

    // Iterate over each row index of `this` matrix.
    for row = 0 to rows - 1
        // Iterate over each column index of `this` matrix on each `row`.
        for col = 0 to columns - 1
            //@variable The element from `this` matrix at the `row` and `col` index.
            float element = this.get(row, col)
            // Initialize the corresponding `result` cell with the `element` value.
            result.cell(col, row, str.toString(element), text_color = textColor, text_size = size.huge)

    // Initialize a merged cell on the bottom row if a `note` is provided.
    if not na(note)
        result.cell(0, rows, note, text_color = textColor, text_size = size.huge)
        result.merge_cells(0, rows, columns - 1, rows)

    result // Return the `result` table.

//@variable A 3x4 matrix of values.
var m = matrix.new<float>()

if bar_index == 0
    // Add rows to `m`.
    m.add_row(0, array.from(1, 2, 3))
    m.add_row(1, array.from(5, 6, 7))
    m.add_row(2, array.from(9, 10, 11))
    // Add a column to `m`.
    m.add_col(3, array.from(4, 8, 12))
    // Display the elements of `m` in a table.
    m.toTable()

```

for...in

When a script needs to iterate over and retrieve the rows of a matrix, using the `for...in` structure is often preferred over the standard `for` loop. This structure directly references the row arrays in a matrix, making it a more convenient option for such use cases. For example, this line creates a loop that returns a row array for each row in the `m` matrix:

```
for row in m
```

The following indicator calculates the moving average of OHLC data with an input `length` and displays the values on the chart. The custom `rowWiseAvg()` method loops through the rows of a matrix using a `for...in` structure to produce an array containing the `array.avg()` of each row.

On the first chart bar, the script creates a new `m` matrix with four rows and `length` columns, which it queues a new column of OHLC data into via the `m.add_col()` and `m.remove_col()` methods on each subsequent bar. It uses `m.rowWiseAvg()` to calculate the array of row-wise averages, then it plots the element values on the chart:



Figure 64: image

```
//@version=6
indicator("for...in loop demo", "Average OHLC", overlay = true)

//@variable The number of terms in the average.
int length = input.int(20, "Length", minval = 1)

//@function Calculates the average of each matrix row.
method rowWiseAvg(matrix<float> this) =>
    //@variable An array with elements corresponding to each row's average.
    array<float> result = array.new<float>()
    // Iterate over each `row` of `this` matrix.
    for row in this
        // Push the average of each `row` into the `result`.
        result.push(row.avg())
    result // Return the resulting array.

//@variable A 4x`length` matrix of values.
var matrix<float> m = matrix.new<float>(4, length)

// Add a new column containing OHLC values to the matrix.
m.add_col(m.columns(), array.from(open, high, low, close))
// Remove the first column.
m.remove_col(0)

//@variable An array containing averages of `open`, `high`, `low`, and `close` over `length` bars.
array<float> averages = m.rowWiseAvg()

plot(averages.get(0), "Average Open", color.blue, 2)
plot(averages.get(1), "Average High", color.green, 2)
plot(averages.get(2), "Average Low", color.red, 2)
```

```
plot(averages.get(3), "Average Close", color.orange, 2)
```

Note that:

- `for...in` loops can also reference the index value of each row. For example, `for [i, row] in m` creates a tuple containing the `i` row index and the corresponding `row` array from the `m` matrix on each loop iteration.

Copying a matrix

Shallow copies

Pine scripts can copy matrices via `matrix.copy()`. This function returns a *shallow copy* of a matrix that does not affect the shape of the original matrix or its references.

For example, this script assigns a new matrix to the `myMatrix` variable and adds two columns. It creates a new `myCopy` matrix from `myMatrix` using the `myMatrix.copy()` method, then adds a new row. It displays the rows of both matrices in labels via the user-defined `debugLabel()` function:

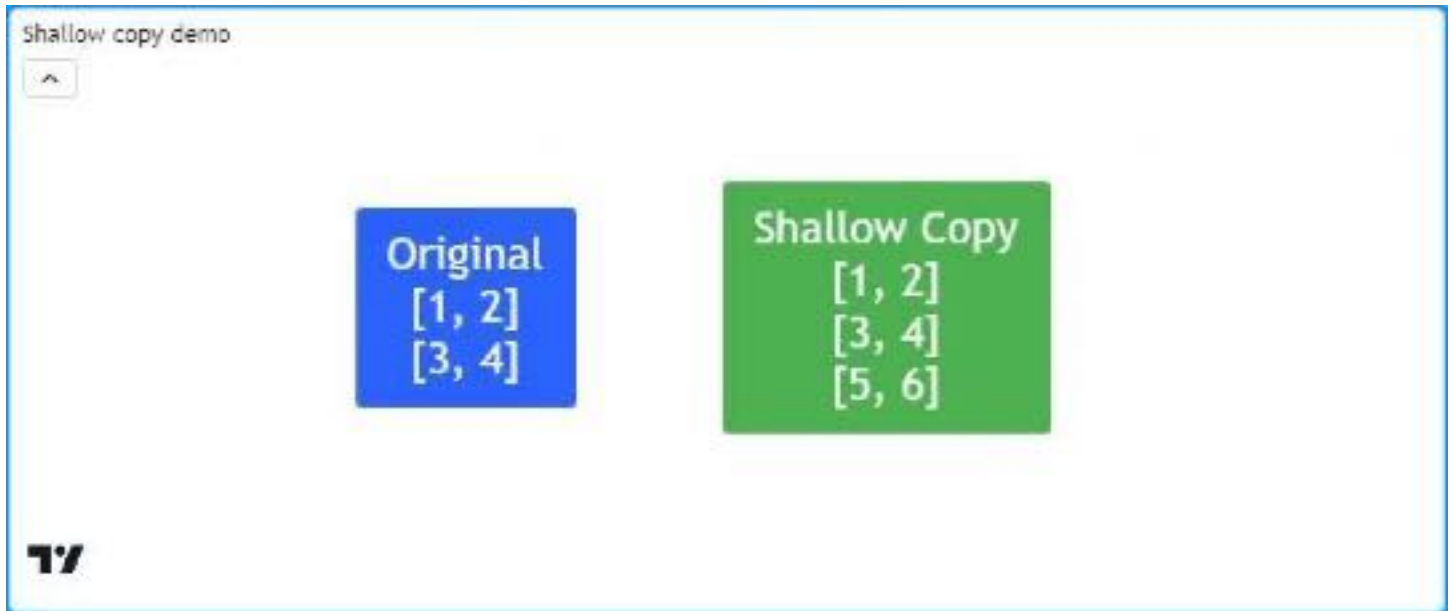


Figure 65: image

```
//@version=6
indicator("Shallow copy demo")

//@function Displays the rows of a matrix in a label with a note.
//@param this The matrix to display.
//@param barIndex The `bar_index` to display the label at.
//@param bgColor The background color of the label.
//@param textColor The color of the label's text.
//@param note The text to display above the rows.
method debugLabel(
    matrix<float> this, int barIndex = bar_index, color bgColor = color.blue,
    color textColor = color.white, string note = ""
) =>
    labelText = note + "\n" + str.toString(this)
    if barstate.ishistory
        label.new(
            barIndex, 0, labelText, color = bgColor, style = label.style_label_center,
            textcolor = textColor, size = size.huge
        )

//@variable A 2x2 `float` matrix.
matrix<float> myMatrix = matrix.new<float>()
```

```

myMatrix.add_col(0, array.from(1.0, 3.0))
myMatrix.add_col(1, array.from(2.0, 4.0))

//@variable A shallow copy of `myMatrix`.
matrix<float> myCopy = myMatrix.copy()
// Add a row to the last index of `myCopy`.
myCopy.add_row(myCopy.rows(), array.from(5.0, 6.0))

if bar_index == last_bar_index - 1
    // Display the rows of both matrices in separate labels.
    myMatrix.debugLabel(note = "Original")
    myCopy.debugLabel(bar_index + 10, color.green, note = "Shallow Copy")

```

It's important to note that the elements within shallow copies of a matrix point to the same values as the original matrix. When matrices contain special types (line, linefill, box, polyline, label, table, or chart.point) or user-defined types, the elements of a shallow copy reference the same objects as the original.

This script declares a `myMatrix` variable with a `newLabel` as the initial value. It then copies `myMatrix` to a `myCopy` variable via `myMatrix.copy()` and plots the number of labels. As we see below, there's only one label on the chart, as the element in `myCopy` references the same object as the element in `myMatrix`. Consequently, changes to the element values in `myCopy` affect the values in both matrices:

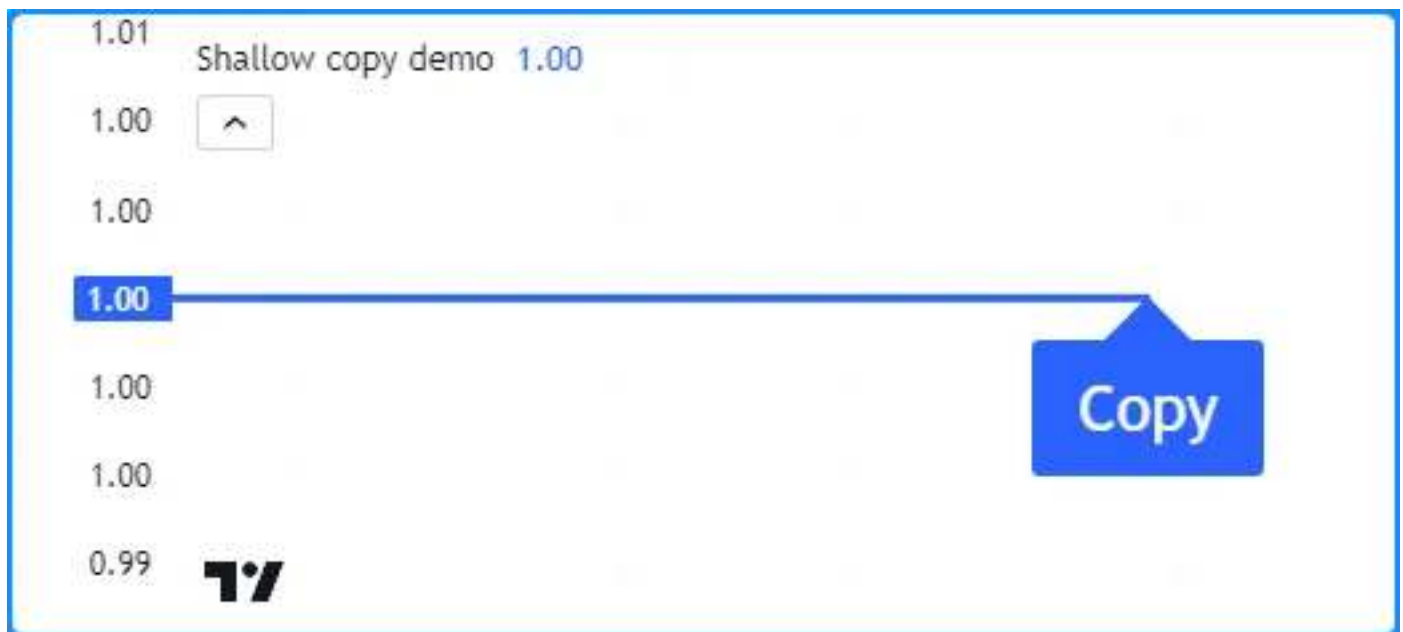


Figure 66: image

```

//@version=6
indicator("Shallow copy demo")

//@variable Initial value of the original matrix elements.
var label newLabel = label.new(
    bar_index, 1, "Original", color = color.blue, textcolor = color.white, size = size.huge
)

//@variable A 1x1 matrix containing a new `label` instance.
var matrix<label> myMatrix = matrix.new<label>(1, 1, newLabel)
//@variable A shallow copy of `myMatrix`.
var matrix<label> myCopy = myMatrix.copy()

//@variable The first label from the `myCopy` matrix.
label testLabel = myCopy.get(0, 0)

```



```
// Change the `text`, `style`, and `x` values of `testLabel`. Also affects the `newLabel`.
testLabel.set_text("Copy")
testLabel.set_style(label.style_label_up)
testLabel.set_x(bar_index)

// Plot the total number of labels.
plot(label.all.size(), linewidth = 3)
```

Deep copies

One can produce a *deep copy* of a matrix (i.e., a matrix whose elements point to copies of the original values) by explicitly copying each object the matrix references.

Here, we've added a `deepCopy()` user-defined method to our previous script. The method creates a new matrix and uses nested for loops to assign all elements to copies of the originals. When the script calls this method instead of the built-in `copy()`, we see that there are now two labels on the chart, and any changes to the label from `myCopy` do not affect the one from `myMatrix`:



Figure 67: image

```
//@version=6
indicator("Deep copy demo")

//@function Returns a deep copy of a label matrix.
method deepCopy(matrix<label> this) =>
  //@variable A deep copy of `this` matrix.
  matrix<label> that = this.copy()
  for row = 0 to that.rows() - 1
    for column = 0 to that.columns() - 1
      // Assign the element at each `row` and `column` of `that` matrix to a copy of the retrieved label
      that.set(row, column, that.get(row, column).copy())
  that

//@variable Initial value of the original matrix.
var label newLabel = label.new(
  bar_index, 2, "Original", color = color.blue, textcolor = color.white, size = size.huge
)

//@variable A 1x1 matrix containing a new `label` instance.
var matrix<label> myMatrix = matrix.new<label>(1, 1, newLabel)
//@variable A deep copy of `myMatrix`.
var matrix<label> myCopy = myMatrix.deepCopy()
```



```
//@variable The first label from the `myCopy` matrix.
label testLabel = myCopy.get(0, 0)

// Change the `text`, `style`, and `x` values of `testLabel`. Does not affect the `newLabel`.
testLabel.set_text("Copy")
testLabel.set_style(label.style_label_up)
testLabel.set_x(bar_index)

// Change the `x` value of `newLabel`.
newLabel.set_x(bar_index)

// Plot the total number of labels.
plot(label.all.size(), linewidth = 3)
```

Submatrices

In Pine, a *submatrix* is a shallow copy of an existing matrix that only includes the rows and columns specified by the `from_row/column` and `to_row/column` parameters. In essence, it is a sliced copy of a matrix.

For example, the script below creates an `mSub` matrix from the `m` matrix via the `m.submatrix()` method, then calls our user-defined `debugLabel()` function to display the rows of both matrices in labels:

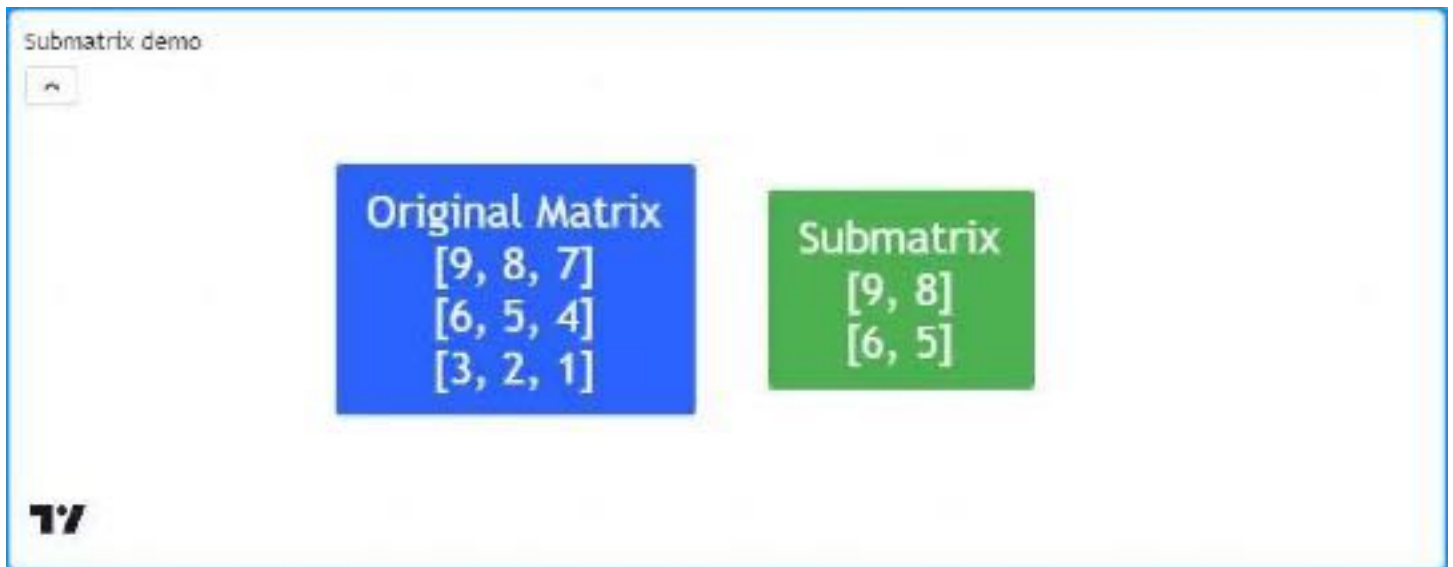


Figure 68: image

```
//@version=6
indicator("Submatrix demo")

//@function Displays the rows of a matrix in a label with a note.
//@param this The matrix to display.
//@param barIndex The `bar_index` to display the label at.
//@param bgColor The background color of the label.
//@param textColor The color of the label's text.
//@param note The text to display above the rows.
method debugLabel(
    matrix<float> this, int barIndex = bar_index, color bgColor = color.blue,
    color textColor = color.white, string note = ""
) =>
    labelText = note + "\n" + str.toString(this)
    if barstate.ishistory
        label.new(
            barIndex, 0, labelText, color = bgColor, style = label.style_label_center,
```

```

        textcolor = textColor, size = size.huge
    )

//@variable A 3x3 matrix of values.
var m = matrix.new<float>()

if bar_index == last_bar_index - 1
    // Add columns to `m`.
    m.add_col(0, array.from(9, 6, 3))
    m.add_col(1, array.from(8, 5, 2))
    m.add_col(2, array.from(7, 4, 1))
    // Display the rows of `m`.
    m.debugLabel(note = "Original Matrix")

    //@variable A 2x2 submatrix of `m` containing the first two rows and columns.
    matrix<float> mSub = m.submatrix(from_row = 0, to_row = 2, from_column = 0, to_column = 2)
    // Display the rows of `mSub`
    debugLabel(mSub, bar_index + 10, bgColor = color.green, note = "Submatrix")

```

Scope and history

Matrix variables leave historical trails on each bar, allowing scripts to use the history-referencing operator `[]` to interact with past matrix instances previously assigned to a variable. Additionally, scripts can modify matrices assigned to global variables from within the scopes of functions, methods, and conditional structures.

This script calculates the average ratios of body and wick distances relative to the bar range over `length` bars. It displays the data along with values from `length` bars ago in a table. The user-defined `addData()` function adds columns of current and historical ratios to the `globalMatrix`, and the `calcAvg()` function references `previous` matrices assigned to `globalMatrix` using the `[]` operator to calculate a matrix of averages:



Figure 69: image

```

//@version=6
indicator("Scope and history demo", "Bar ratio comparison")

int length = input.int(10, "Length", 1)

```

```

//@variable A global matrix.
matrix<float> globalMatrix = matrix.new<float>()

//@function Calculates the ratio of body range to candle range.
bodyRatio() =>
    math.abs(close - open) / (high - low)

//@function Calculates the ratio of upper wick range to candle range.
upperWickRatio() =>
    (high - math.max(open, close)) / (high - low)

//@function Calculates the ratio of lower wick range to candle range.
lowerWickRatio() =>
    (math.min(open, close) - low) / (high - low)

//@function Adds data to the `globalMatrix`.
addData() =>
    // Add a new column of data at `column` 0.
    globalMatrix.add_col(0, array.from(bodyRatio(), upperWickRatio(), lowerWickRatio()))
    //@variable The column of `globalMatrix` from index 0 `length` bars ago.
    array<float> pastValues = globalMatrix.col(0)[length]
    // Add `pastValues` to the `globalMatrix`, or an array of `na` if `pastValues` is `na`.
    if na(pastValues)
        globalMatrix.add_col(1, array.new<float>(3))
    else
        globalMatrix.add_col(1, pastValues)

//@function Returns the `length`-bar average of matrices assigned to `globalMatrix` on historical bars.
calcAvg() =>
    //@variable The sum historical `globalMatrix` matrices.
    matrix<float> sums = matrix.new<float>(globalMatrix.rows(), globalMatrix.columns(), 0.0)
    for i = 0 to length - 1
        //@variable The `globalMatrix` matrix `i` bars before the current bar.
        matrix<float> previous = globalMatrix[i]
        // Break the loop if `previous` is `na`.
        if na(previous)
            sums.fill(na)
            break
        // Assign the sum of `sums` and `previous` to `sums`.
        sums := matrix.sum(sums, previous)
    // Divide the `sums` matrix by the `length`.
    result = sums.mult(1.0 / length)

// Add data to the `globalMatrix`.
addData()

//@variable The historical average of the `globalMatrix` matrices.
globalAvg = calcAvg()

//@variable A `table` displaying information from the `globalMatrix`.
var table infoTable = table.new(
    position.middle_center, globalMatrix.columns() + 1, globalMatrix.rows() + 1, bgcolor = color.navy
)

// Define value cells.
for [i, row] in globalAvg
    for [j, value] in row
        color textColor = value > 0.333 ? color.orange : color.gray
        infoTable.cell(j + 1, i + 1, str.toString(value), text_color = textColor, text_size = size.huge)

```

```
// Define header cells.
infoTable.cell(0, 1, "Body ratio", text_color = color.white, text_size = size.huge)
infoTable.cell(0, 2, "Upper wick ratio", text_color = color.white, text_size = size.huge)
infoTable.cell(0, 3, "Lower wick ratio", text_color = color.white, text_size = size.huge)
infoTable.cell(1, 0, "Current average", text_color = color.white, text_size = size.huge)
infoTable.cell(2, 0, str.format("{0} bars ago", length), text_color = color.white, text_size = size.huge)
```

Note that:

- The `addData()` and `calcAvg()` functions have no parameters, as they directly interact with the `globalMatrix` and `length` variables declared in the outer scope.
- `calcAvg()` calculates the average by adding `previous` matrices using `matrix.sum()` and multiplying all elements by `1 / length` using `matrix.mult()`. We discuss these and other specialized functions in our Matrix calculations section below.

Inspecting a matrix

The ability to inspect the shape of a matrix and patterns within its elements is crucial, as it helps reveal important information about a matrix and its compatibility with various calculations and transformations. Pine Script™ includes several built-ins for matrix inspection, including `matrix.is_square()`, `matrix.is_identity()`, `matrix.is_diagonal()`, `matrix.is_antidiagonal()`, `matrix.is_symmetric()`, `matrix.is_antisymmetric()`, `matrix.is_triangular()`, `matrix.is_stochastic()`, `matrix.is_binary()`, and `matrix.is_zero()`.

To demonstrate these features, this example contains a custom `inspect()` method that uses conditional blocks with `matrix.is_*`() functions to return information about a matrix. It displays a string representation of an `m` matrix and the description returned from `m.inspect()` in labels on the chart:

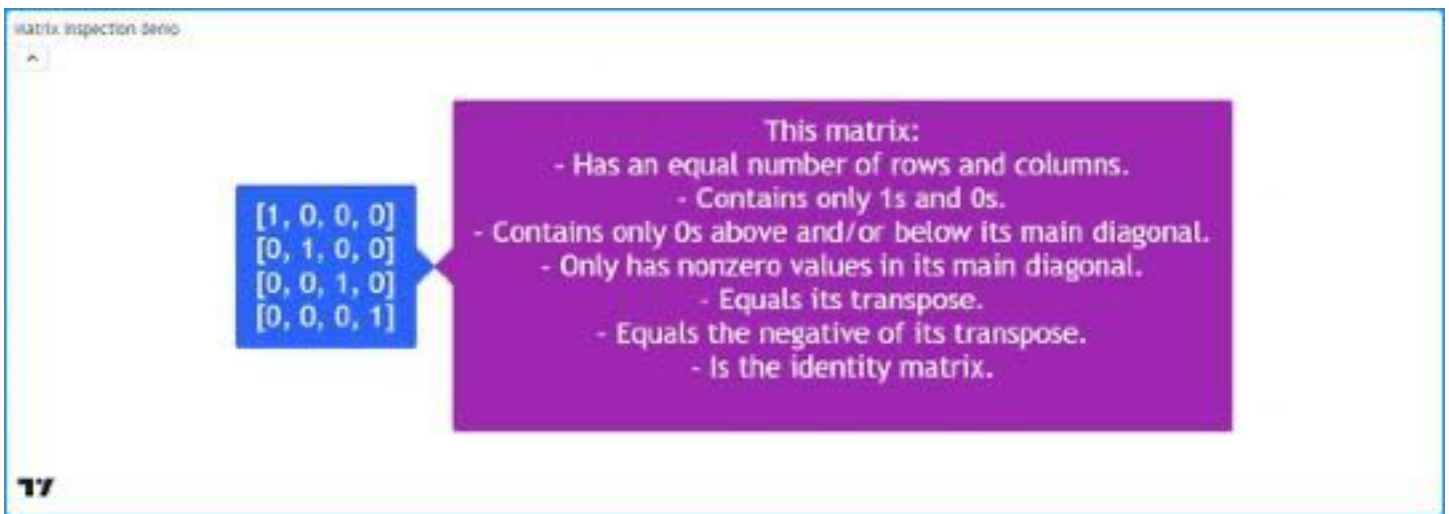


Figure 70: image

```
//@version=6
indicator("Matrix inspection demo")

//@function Inspects a matrix using `matrix.is_*( )` functions and returns a `string` describing some of its fe
method inspect(matrix<int> this)=>
    //@variable A string describing `this` matrix.
    string result = "This matrix:\n"
    if this.is_square()
        result += "- Has an equal number of rows and columns.\n"
    if this.is_binary()
        result += "- Contains only 1s and 0s.\n"
    if this.is_zero()
        result += "- Is filled with 0s.\n"
    if this.is_triangular()
        result += "- Contains only 0s above and/or below its main diagonal.\n"
    if this.is_diagonal()
```

```

        result += "- Only has nonzero values in its main diagonal.\n"
    if this.is_antidiagonal()
        result += "- Only has nonzero values in its main antidiagonal.\n"
    if this.is_symmetric()
        result += "- Equals its transpose.\n"
    if this.is_antisymmetric()
        result += "- Equals the negative of its transpose.\n"
    if this.is_identity()
        result += "- Is the identity matrix.\n"
    result

//@variable A 4x4 identity matrix.
matrix<int> m = matrix.new<int>()

// Add rows to the matrix.
m.add_row(0, array.from(1, 0, 0, 0))
m.add_row(1, array.from(0, 1, 0, 0))
m.add_row(2, array.from(0, 0, 1, 0))
m.add_row(3, array.from(0, 0, 0, 1))

if bar_index == last_bar_index - 1
    // Display the `m` matrix in a blue label.
    label.new(
        bar_index, 0, str.toString(m), color = color.blue, style = label.style_label_right,
        textcolor = color.white, size = size.huge
    )
    // Display the result of `m.inspect()` in a purple label.
    label.new(
        bar_index, 0, m.inspect(), color = color.purple, style = label.style_label_left,
        textcolor = color.white, size = size.huge
    )

```

Manipulating a matrix

Reshaping

The shape of a matrix can determine its compatibility with various matrix operations. In some cases, it is necessary to change the dimensions of a matrix without affecting the number of elements or the values they reference, otherwise known as *reshaping*. To reshape a matrix in Pine, use the `matrix.reshape()` function.

This example demonstrates the results of multiple reshaping operations on a matrix. The initial `m` matrix has a 1x8 shape (one row and eight columns). Through successive calls to the `m.reshape()` method, the script changes the shape of `m` to 2x4, 4x2, and 8x1. It displays each reshaped matrix in a label on the chart using the custom `debugLabel()` method:

```

//@version=6
indicator("Reshaping example")

//@function Displays the rows of a matrix in a label with a note.
//@param    this The matrix to display.
//@param    barIndex The `bar_index` to display the label at.
//@param    bgColor The background color of the label.
//@param    textColor The color of the label's text.
//@param    note The text to display above the rows.
method debugLabel(
    matrix<float> this, int barIndex = bar_index, color bgColor = color.blue,
    color textColor = color.white, string note = ""
) =>
    labelText = note + "\n" + str.toString(this)
    if barstate.ishistory
        label.new(
            barIndex, 0, labelText, color = bgColor, style = label.style_label_center,
            textcolor = textColor, size = size.huge
        )

```

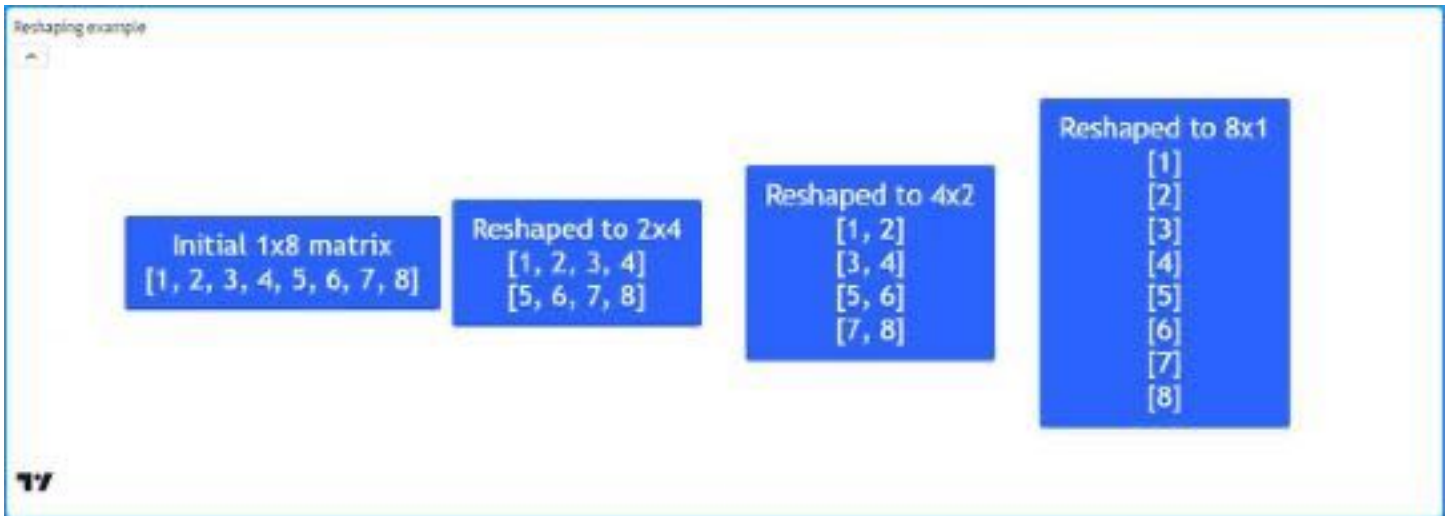


Figure 71: image

)

```

//@variable A matrix containing the values 1-8.
matrix<int> m = matrix.new<int>()

if bar_index == last_bar_index - 1
    // Add the initial vector of values.
    m.add_row(0, array.from(1, 2, 3, 4, 5, 6, 7, 8))
    m.debugLabel(note = "Initial 1x8 matrix")

    // Reshape. `m` now has 2 rows and 4 columns.
    m.reshape(2, 4)
    m.debugLabel(bar_index + 10, note = "Reshaped to 2x4")

    // Reshape. `m` now has 4 rows and 2 columns.
    m.reshape(4, 2)
    m.debugLabel(bar_index + 20, note = "Reshaped to 4x2")

    // Reshape. `m` now has 8 rows and 1 column.
    m.reshape(8, 1)
    m.debugLabel(bar_index + 30, note = "Reshaped to 8x1")
  
```

Note that:

- The order of elements in `m` does not change with each `m.reshape()` call.
- When reshaping a matrix, the product of the `rows` and `columns` arguments must equal the `matrix.elements_count()` value, as `matrix.reshape()` cannot change the number of elements in a matrix.

Reversing

One can reverse the order of all elements in a matrix using `matrix.reverse()`. This function moves the references of an `m`-by-`n` matrix `id` at the `i`-th row and `j`-th column to the `m - 1 - i` row and `n - 1 - j` column.

For example, this script creates a 3x3 matrix containing the values 1-9 in ascending order, then uses the `reverse()` method to reverse its contents. It displays the original and modified versions of the matrix in labels on the chart via `m.debugLabel()`:

```

//@version=6
indicator("Reversing demo")

//@function Displays the rows of a matrix in a label with a note.
//@param this The matrix to display.
//@param barIndex The `bar_index` to display the label at.
  
```



Figure 72: image

```
//@param    bgColor The background color of the label.
//@param    textColor The color of the label's text.
//@param    note The text to display above the rows.
method debugLabel(
    matrix<float> this, int barIndex = bar_index, color bgColor = color.blue,
    color textColor = color.white, string note = ""
) =>
    labelText = note + "\n" + str.toString(this)
    if barstate.ishistory
        label.new(
            barIndex, 0, labelText, color = bgColor, style = label.style_label_center,
            textcolor = textColor, size = size.huge
        )

//@variable A 3x3 matrix.
matrix<float> m = matrix.new<float>()

// Add rows to `m`.
m.add_row(0, array.from(1, 2, 3))
m.add_row(1, array.from(4, 5, 6))
m.add_row(2, array.from(7, 8, 9))

if bar_index == last_bar_index - 1
    // Display the contents of `m`.
    m.debugLabel(note = "Original")
    // Reverse `m`, then display its contents.
    m.reverse()
    m.debugLabel(bar_index + 10, color.red, note = "Reversed")
```

Transposing

Transposing a matrix is a fundamental operation that flips all rows and columns in a matrix about its *main diagonal* (the diagonal vector of all values in which the row index equals the column index). This process produces a new matrix with reversed row and column dimensions, known as the *transpose*. Scripts can calculate the transpose of a matrix using `matrix.transpose()`.

For any m-row, n-column matrix, the matrix returned from `matrix.transpose()` will have n rows and m columns. All elements in a matrix at the i-th row and j-th column correspond to the elements in its transpose at the j-th row and i-th column.

This example declares a 2x4 m matrix, calculates its transpose using the `m.transpose()` method, and displays both matrices

on the chart using our custom `debugLabel()` method. As we can see below, the transposed matrix has a 4x2 shape, and the rows of the transpose match the columns of the original:

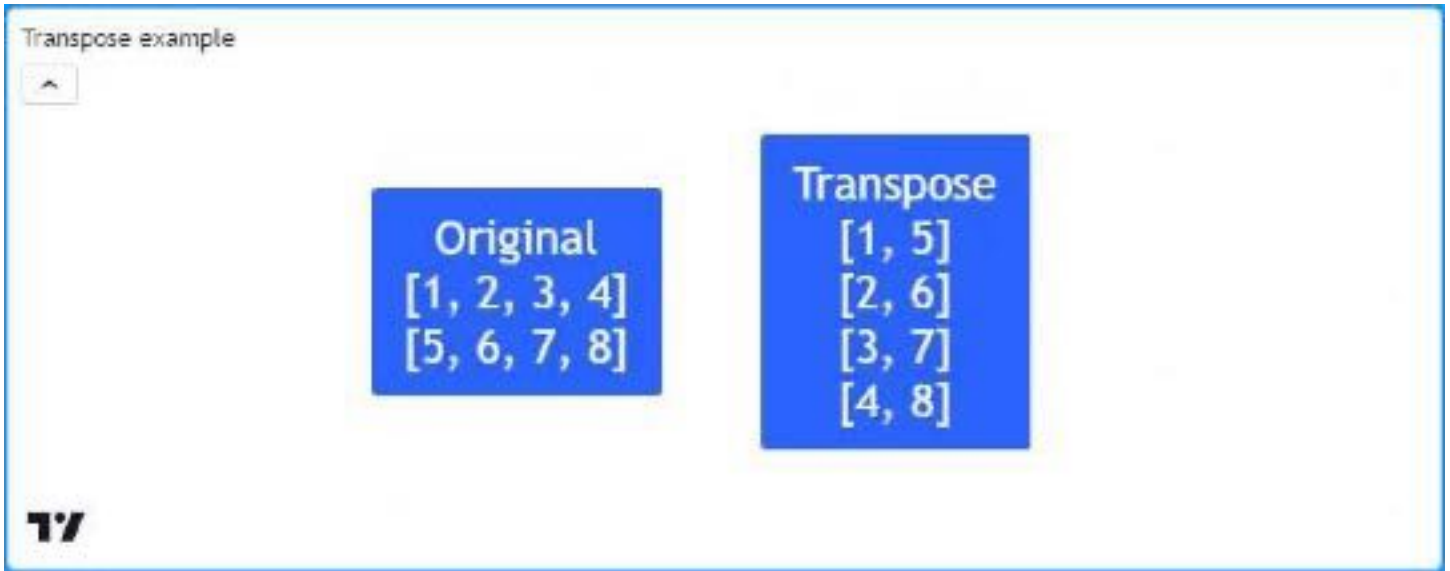


Figure 73: image

```
//@version=6
indicator("Transpose example")

//@function Displays the rows of a matrix in a label with a note.
//@param this The matrix to display.
//@param barIndex The `bar_index` to display the label at.
//@param bgColor The background color of the label.
//@param textColor The color of the label's text.
//@param note The text to display above the rows.
method debugLabel(
    matrix<float> this, int barIndex = bar_index, color bgColor = color.blue,
    color textColor = color.white, string note = ""
) =>
    labelText = note + "\n" + str.toString(this)
    if barstate.ishistory
        label.new(
            barIndex, 0, labelText, color = bgColor, style = label.style_label_center,
            textcolor = textColor, size = size.huge
        )

//@variable A 2x4 matrix.
matrix<int> m = matrix.new<int>()

// Add columns to `m`.
m.add_col(0, array.from(1, 5))
m.add_col(1, array.from(2, 6))
m.add_col(2, array.from(3, 7))
m.add_col(3, array.from(4, 8))

//@variable The transpose of `m`. Has a 4x2 shape.
matrix<int> mt = m.transpose()

if bar_index == last_bar_index - 1
    m.debugLabel(note = "Original")
    mt.debugLabel(bar_index + 10, note = "Transpose")
```


Sorting

Scripts can sort the contents of a matrix via `matrix.sort()`. Unlike `array.sort()`, which sorts *elements*, this function organizes all *rows* in a matrix in a specified **order** (order.ascending by default) based on the values in a specified **column**.

This script declares a 3x3 m matrix, sorts the rows of the m1 copy in ascending order based on the first column, then sorts the rows of the m2 copy in descending order based on the second column. It displays the original matrix and sorted copies in labels using our `debugLabel()` method:

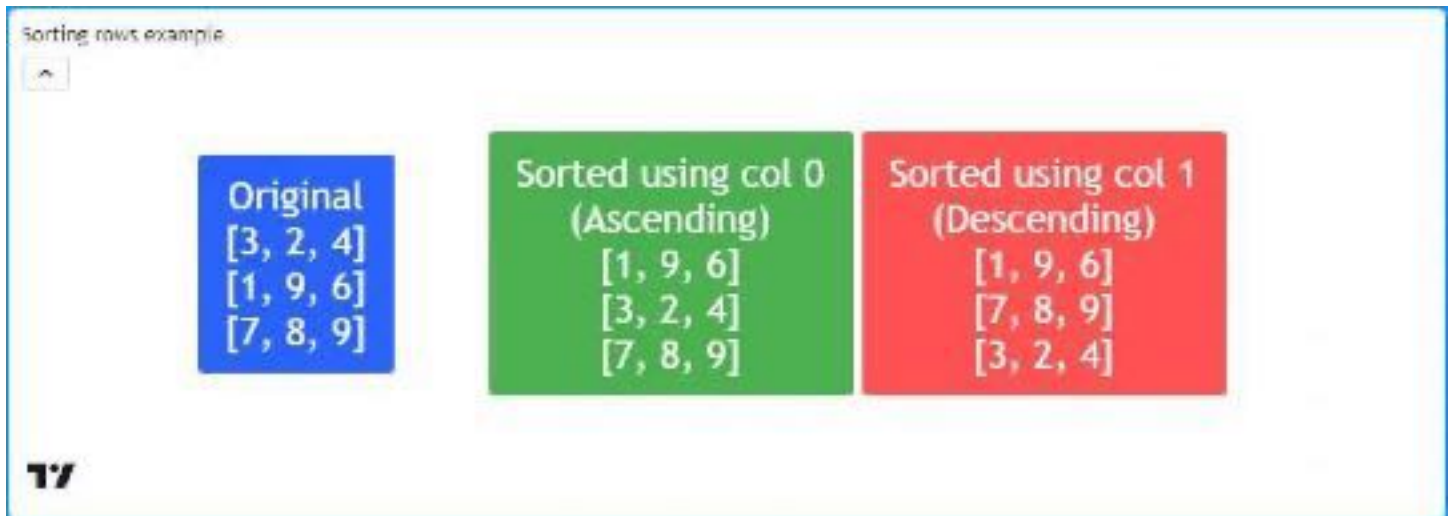


Figure 74: image

```
//@version=6
indicator("Sorting rows example")

//@function Displays the rows of a matrix in a label with a note.
//@param this The matrix to display.
//@param barIndex The `bar_index` to display the label at.
//@param bgColor The background color of the label.
//@param textColor The color of the label's text.
//@param note The text to display above the rows.
method debugLabel(
    matrix<float> this, int barIndex = bar_index, color bgColor = color.blue,
    color textColor = color.white, string note = ""
) =>
    labelText = note + "\n" + str.toString(this)
    if barstate.ishistory
        label.new(
            barIndex, 0, labelText, color = bgColor, style = label.style_label_center,
            textcolor = textColor, size = size.huge
        )

//@variable A 3x3 matrix.
matrix<int> m = matrix.new<int>()

if bar_index == last_bar_index - 1
    // Add rows to `m`.
    m.add_row(0, array.from(3, 2, 4))
    m.add_row(1, array.from(1, 9, 6))
    m.add_row(2, array.from(7, 8, 9))
    m.debugLabel(note = "Original")

    // Copy `m` and sort rows in ascending order based on the first column (default).
    matrix<int> m1 = m.copy()
    m1.sort()
```

```

m1.debugLabel(bar_index + 10, color.green, note = "Sorted using col 0\n(Ascending)")

// Copy `m` and sort rows in descending order based on the second column.
matrix<int> m2 = m.copy()
m2.sort(1, order.descending)
m2.debugLabel(bar_index + 20, color.red, note = "Sorted using col 1\n(Descending)")

```

It's important to note that `matrix.sort()` does not sort the columns of a matrix. However, one *can* use this function to sort matrix columns with the help of `matrix.transpose()`.

As an example, this script contains a `sortColumns()` method that uses the `sort()` method to sort the transpose of a matrix using the column corresponding to the row of the original matrix. The script uses this method to sort the `m` matrix based on the contents of its first row:

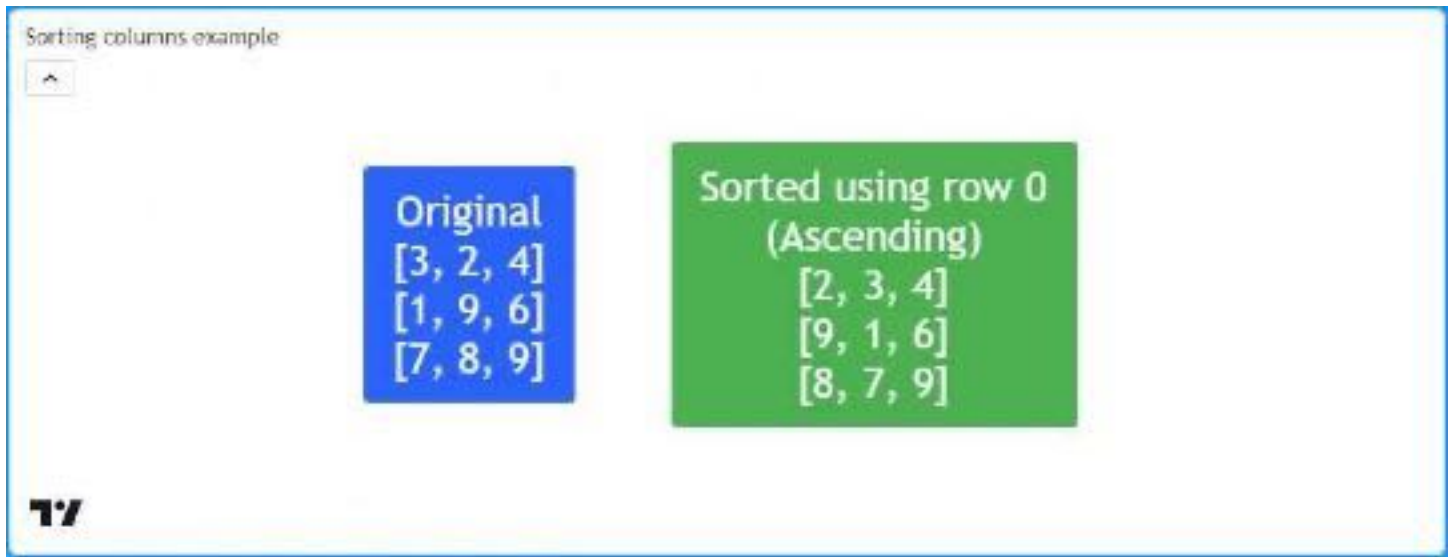


Figure 75: image

```

//@version=6
indicator("Sorting columns example")

//@function Displays the rows of a matrix in a label with a note.
//@param this The matrix to display.
//@param barIndex The `bar_index` to display the label at.
//@param bgColor The background color of the label.
//@param textColor The color of the label's text.
//@param note The text to display above the rows.
method debugLabel(
    matrix<float> this, int barIndex = bar_index, color bgColor = color.blue,
    color textColor = color.white, string note = ""
) =>
    labelText = note + "\n" + str.toString(this)
    if barstate.ishistory
        label.new(
            barIndex, 0, labelText, color = bgColor, style = label.style_label_center,
            textcolor = textColor, size = size.huge
        )

//@function Sorts the columns of `this` matrix based on the values in the specified `row`.
method sortColumns(matrix<int> this, int row = 0, bool ascending = true) =>
    //@variable The transpose of `this` matrix.
    matrix<int> thisT = this.transpose()
    //@variable Is `order.ascending` when `ascending` is `true`, `order.descending` otherwise.
    order = ascending ? order.ascending : order.descending
    // Sort the rows of `thisT` using the `row` column.

```

```

thisT.sort(row, order)
//@variable A copy of `this` matrix with sorted columns.
result = thisT.transpose()

//@variable A 3x3 matrix.
matrix<int> m = matrix.new<int>()

if bar_index == last_bar_index - 1
    // Add rows to `m`.
    m.add_row(0, array.from(3, 2, 4))
    m.add_row(1, array.from(1, 9, 6))
    m.add_row(2, array.from(7, 8, 9))
    m.debugLabel(note = "Original")

    // Sort the columns of `m` based on the first row and display the result.
    m.sortColumns(0).debugLabel(bar_index + 10, note = "Sorted using row 0\n(Ascending)")

```

Concatenating

Scripts can *concatenate* two matrices using `matrix.concat()`. This function appends the rows of an `id2` matrix to the end of an `id1` matrix with the same number of columns.

To create a matrix with elements representing the *columns* of a matrix appended to another, transpose both matrices, use `matrix.concat()` on the transposed matrices, then `transpose()` the result.

For example, this script appends the rows of the `m2` matrix to the `m1` matrix and appends their columns using *transposed copies* of the matrices. It displays the `m1` and `m2` matrices and the results after concatenating their rows and columns in labels using the custom `debugLabel()` method:

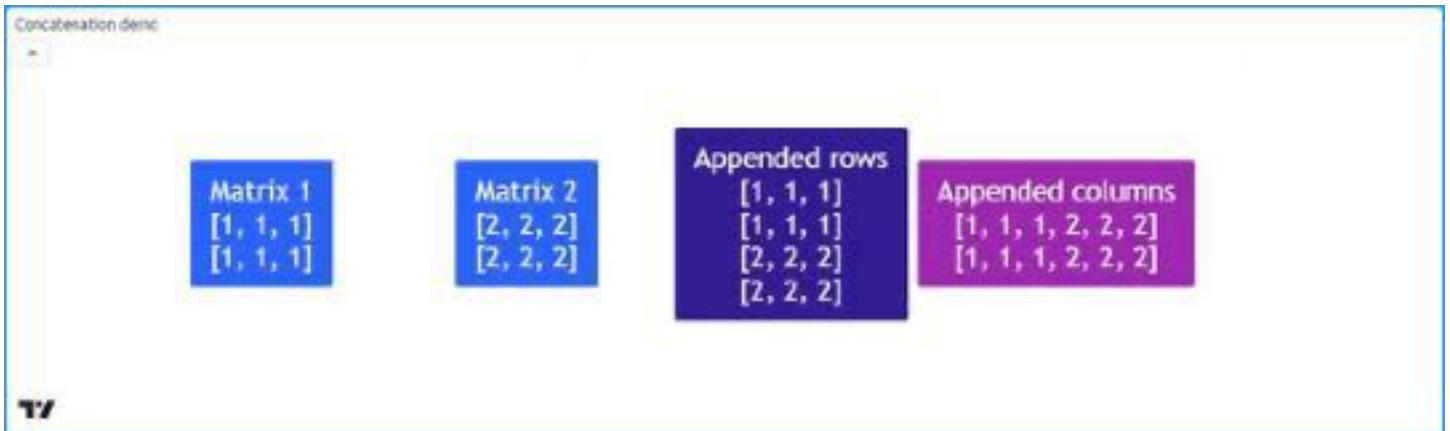


Figure 76: image

```

//@version=6
indicator("Concatenation demo")

//@function Displays the rows of a matrix in a label with a note.
//@param this The matrix to display.
//@param barIndex The `bar_index` to display the label at.
//@param bgColor The background color of the label.
//@param textColor The color of the label's text.
//@param note The text to display above the rows.
method debugLabel(
    matrix<float> this, int barIndex = bar_index, color bgColor = color.blue,
    color textColor = color.white, string note = ""
) =>
    labelText = note + "\n" + str.toStringing(this)
    if barstate.ishistory
        label.new(

```

```

        barIndex, 0, labelText, color = bgColor, style = label.style_label_center,
        textcolor = textColor, size = size.huge
    )

//@variable A 2x3 matrix filled with 1s.
matrix<int> m1 = matrix.new<int>(2, 3, 1)
//@variable A 2x3 matrix filled with 2s.
matrix<int> m2 = matrix.new<int>(2, 3, 2)

//@variable The transpose of `m1`.
t1 = m1.transpose()
//@variable The transpose of `m2`.
t2 = m2.transpose()

if bar_index == last_bar_index - 1
    // Display the original matrices.
    m1.debugLabel(note = "Matrix 1")
    m2.debugLabel(bar_index + 10, note = "Matrix 2")
    // Append the rows of `m2` to the end of `m1` and display `m1`.
    m1.concat(m2)
    m1.debugLabel(bar_index + 20, color.blue, note = "Appended rows")
    // Append the rows of `t2` to the end of `t1`, then display the transpose of `t1`.
    t1.concat(t2)
    t1.transpose().debugLabel(bar_index + 30, color.purple, note = "Appended columns")

```

Matrix calculations

Element-wise calculations

Pine scripts can calculate the *average*, *minimum*, *maximum*, and *mode* of all elements within a matrix via `matrix.avg()`, `matrix.min()`, `matrix.max()`, and `matrix.mode()`. These functions operate the same as their `array.*` equivalents, allowing users to run element-wise calculations on a matrix, its submatrices, and its rows and columns using the same syntax. For example, the built-in `*.avg()` functions called on a 3x3 matrix with values 1-9 and an array with the same nine elements will both return a value of 5.

The script below uses `*.avg()`, `*.max()`, and `*.min()` methods to calculate developing averages and extremes of OHLC data in a period. It adds a new column of open, high, low, and close values to the end of the `ohlcData` matrix whenever `queueColumn` is `true`. When `false`, the script uses the `get()` and `set()` matrix methods to adjust the elements in the last column for developing HLC values in the current period. It uses the `ohlcData` matrix, a `submatrix()`, and `row()` and `col()` arrays to calculate the developing OHLC4 and HL2 averages over `length` periods, the maximum high and minimum low over `length` periods, and the current period's developing OHLC4 price:

```

//@version=6
indicator("Element-wise calculations example", "Developing values", overlay = true)

//@variable The number of data points in the averages.
int length = input.int(3, "Length", 1)
//@variable The timeframe of each reset period.
string timeframe = input.timeframe("D", "Reset Timeframe")

//@variable A 4x`length` matrix of OHLC values.
var matrix<float> ohlcData = matrix.new<float>(4, length)

//@variable Is `true` at the start of a new bar at the `timeframe`.
bool queueColumn = timeframe.change(timeframe)

if queueColumn
    // Add new values to the end column of `ohlcData`.
    ohlcData.add_col(length, array.from(open, high, low, close))
    // Remove the oldest column from `ohlcData`.
    ohlcData.remove_col(0)
else

```



Figure 77: image

```
// Adjust the last element of column 1 for new highs.
if high > ohlcData.get(1, length - 1)
    ohlcData.set(1, length - 1, high)
// Adjust the last element of column 2 for new lows.
if low < ohlcData.get(2, length - 1)
    ohlcData.set(2, length - 1, low)
// Adjust the last element of column 3 for the new closing price.
ohlcData.set(3, length - 1, close)

//@variable The `matrix.avg()` of all elements in `ohlcData`.
avgOHLC4 = ohlcData.avg()
//@variable The `matrix.avg()` of all elements in rows 1 and 2, i.e., the average of all `high` and `low` values.
avgHL2 = ohlcData.submatrix(from_row = 1, to_row = 3).avg()
//@variable The `matrix.max()` of all values in `ohlcData`. Equivalent to `ohlcData.row(1).max()`.
maxHigh = ohlcData.max()
//@variable The `array.min()` of all `low` values in `ohlcData`. Equivalent to `ohlcData.min()`.
minLow = ohlcData.row(2).min()
//@variable The `array.avg()` of the last column in `ohlcData`, i.e., the current OHLC4.
ohlc4Value = ohlcData.col(length - 1).avg()

plot(avgOHLC4, "Average OHLC4", color.purple, 2)
plot(avgHL2, "Average HL2", color.navy, 2)
plot(maxHigh, "Max High", color.green)
plot(minLow, "Min Low", color.red)
plot(ohlc4Value, "Current OHLC4", color.blue)
```

Note that:

- In this example, we used `array.*()` and `matrix.*()` methods interchangeably to demonstrate their similarities in syntax and behavior.
- Users can calculate the matrix equivalent of `array.sum()` by multiplying the `matrix.avg()` by the `matrix.elements_count()`.

Special calculations

Pine Script™ features several built-in functions for performing essential matrix arithmetic and linear algebra operations, including `matrix.sum()`, `matrix.diff()`, `matrix.mult()`, `matrix.pow()`, `matrix.det()`, `matrix.inv()`, `matrix.pinv()`, `matrix.rank()`, `matrix.trace()`, `matrix.eigenvalues()`, `matrix.eigenvectors()`, and `matrix.kron()`. These functions are advanced features that facilitate a variety of matrix calculations and transformations.

Below, we explain a few fundamental functions with some basic examples.

`matrix.sum()` and `matrix.diff()` Scripts can perform addition and subtraction of two matrices with the same shape or a matrix and a scalar value using the `matrix.sum()` and `matrix.diff()` functions. These functions use the values from the `id2` matrix or scalar to add to or subtract from the elements in `id1`.

This script demonstrates a simple example of matrix addition and subtraction in Pine. It creates a 3x3 matrix, calculates its transpose, then calculates the `matrix.sum()` and `matrix.diff()` of the two matrices. This example displays the original matrix, its transpose, and the resulting sum and difference matrices in labels on the chart:

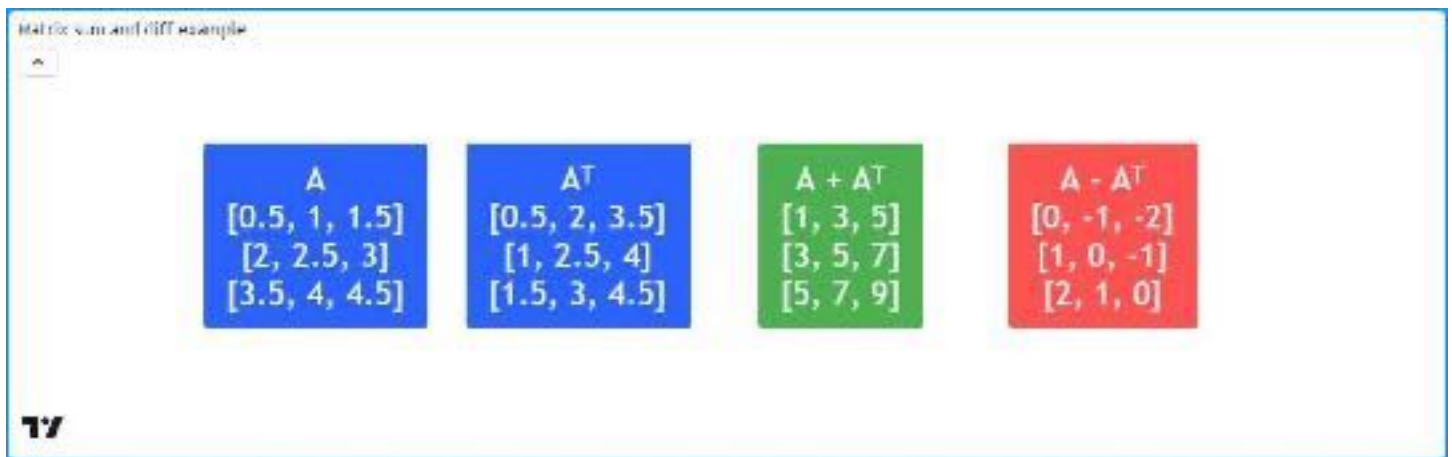


Figure 78: image

```
//@version=6
indicator("Matrix sum and diff example")

//@function Displays the rows of a matrix in a label with a note.
//@param this The matrix to display.
//@param barIndex The `bar_index` to display the label at.
//@param bgColor The background color of the label.
//@param textColor The color of the label's text.
//@param note The text to display above the rows.
method debugLabel(
    matrix<float> this, int barIndex = bar_index, color bgColor = color.blue,
    color textColor = color.white, string note = ""
) =>
    labelText = note + "\n" + str.toString(this)
    if barstate.ishistory
        label.new(
            barIndex, 0, labelText, color = bgColor, style = label.style_label_center,
            textcolor = textColor, size = size.huge
        )

//@variable A 3x3 matrix.
m = matrix.new<float>()

// Add rows to `m`.
m.add_row(0, array.from(0.5, 1.0, 1.5))
m.add_row(1, array.from(2.0, 2.5, 3.0))
m.add_row(2, array.from(3.5, 4.0, 4.5))
```

```

if bar_index == last_bar_index - 1
    // Display `m`.
    m.debugLabel(note = "A")
    // Get and display the transpose of `m`.
    matrix<float> t = m.transpose()
    t.debugLabel(bar_index + 10, note = "A ")
    // Calculate the sum of the two matrices. The resulting matrix is symmetric.
    matrix.sum(m, t).debugLabel(bar_index + 20, color.green, note = "A + A ")
    // Calculate the difference between the two matrices. The resulting matrix is antisymmetric.
    matrix.diff(m, t).debugLabel(bar_index + 30, color.red, note = "A - A ")

```

Note that:

- In this example, we've labeled the original matrix as "A" and the transpose as "A ".
- Adding "A" and "A " produces a symmetric matrix, and subtracting them produces an antisymmetric matrix.

matrix.mult() Scripts can multiply two matrices via the `matrix.mult()` function. This function also facilitates the multiplication of a matrix by an array or a scalar value.

In the case of multiplying two matrices, unlike addition and subtraction, matrix multiplication does not require two matrices to share the same shape. However, the number of columns in the first matrix must equal the number of rows in the second one. The resulting matrix returned by `matrix.mult()` will contain the same number of rows as `id1` and the same number of columns as `id2`. For instance, a 2x3 matrix multiplied by a 3x4 matrix will produce a matrix with two rows and four columns, as shown below. Each value within the resulting matrix is the dot product of the corresponding row in `id1` and column in `id2`:

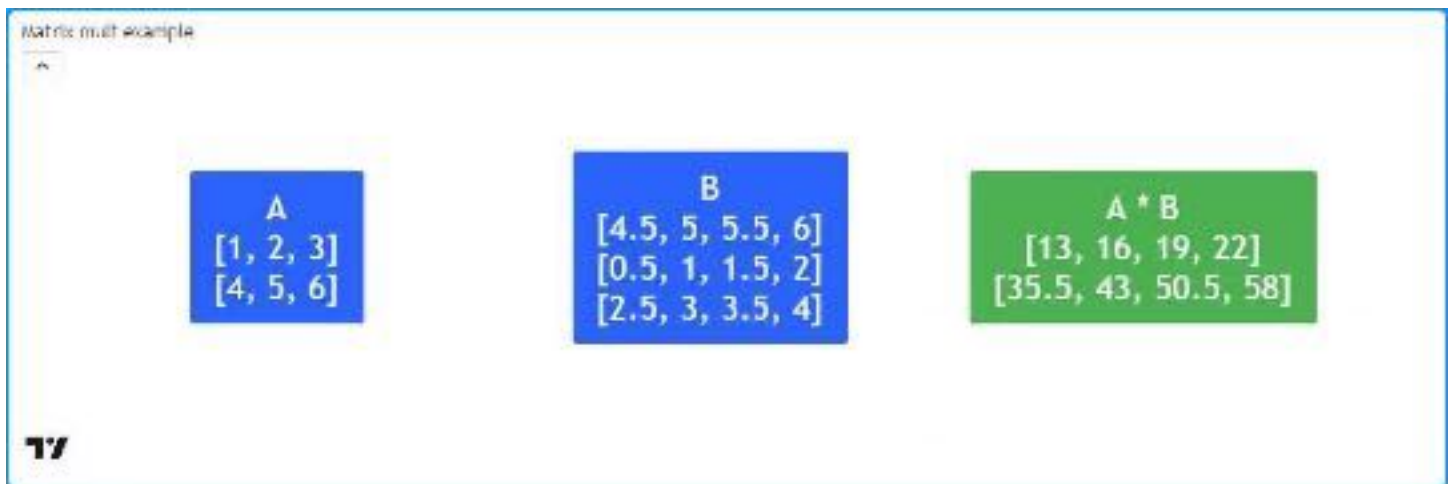


Figure 79: image

```

//@version=6
indicator("Matrix mult example")

//@function Displays the rows of a matrix in a label with a note.
//@param this The matrix to display.
//@param barIndex The `bar_index` to display the label at.
//@param bgColor The background color of the label.
//@param textColor The color of the label's text.
//@param note The text to display above the rows.
method debugLabel(
    matrix<float> this, int barIndex = bar_index, color bgColor = color.blue,
    color textColor = color.white, string note = ""
) =>
    labelText = note + "\n" + str.toStringing(this)
    if barstate.ishistory
        label.new(

```

```

        barIndex, 0, labelText, color = bgColor, style = label.style_label_center,
        textcolor = textColor, size = size.huge
    )

//@variable A 2x3 matrix.
a = matrix.new<float>()
//@variable A 3x4 matrix.
b = matrix.new<float>()

// Add rows to `a`.
a.add_row(0, array.from(1, 2, 3))
a.add_row(1, array.from(4, 5, 6))

// Add rows to `b`.
b.add_row(0, array.from(0.5, 1.0, 1.5, 2.0))
b.add_row(1, array.from(2.5, 3.0, 3.5, 4.0))
b.add_row(2, array.from(4.5, 5.0, 5.5, 6.0))

if bar_index == last_bar_index - 1
    //@variable The result of `a` * `b`.
    matrix<float> ab = a.mult(b)
    // Display `a`, `b`, and `ab` matrices.
    debugLabel(a, note = "A")
    debugLabel(b, bar_index + 10, note = "B")
    debugLabel(ab, bar_index + 20, color.green, note = "A * B")

```

Note that:

- In contrast to the multiplication of scalars, matrix multiplication is *non-commutative*, i.e., `matrix.mult(a, b)` does not necessarily produce the same result as `matrix.mult(b, a)`. In the context of our example, the latter will raise a runtime error because the number of columns in `b` doesn't equal the number of rows in `a`.

When multiplying a matrix and an array, this function treats the operation the same as multiplying `id1` by a single-column matrix, but it returns an array with the same number of elements as the number of rows in `id1`. When `matrix.mult()` passes a scalar as its `id2` value, the function returns a new matrix whose elements are the elements in `id1` multiplied by the `id2` value.

matrix.det() A *determinant* is a scalar value associated with a square matrix that describes some of its characteristics, namely its invertibility. If a matrix has an inverse, its determinant is nonzero. Otherwise, the matrix is *singular* (non-invertible). Scripts can calculate the determinant of a matrix via `matrix.det()`.

Programmers can use determinants to detect similarities between matrices, identify *full-rank* and *rank-deficient* matrices, and solve systems of linear equations, among other applications.

For example, this script utilizes determinants to solve a system of linear equations with a matching number of unknown values using Cramer's rule. The user-defined `solve()` function returns an array containing solutions for each unknown value in the system, where the *n*-th element of the array is the determinant of the coefficient matrix with the *n*-th column replaced by the column of constants divided by the determinant of the original coefficients.

In this script, we've defined the matrix `m` that holds coefficients and constants for these three equations:

$$3 * x_0 + 4 * x_1 - 1 * x_2 = 85 \quad 3 * x_0 - 2 * x_1 + 1 * x_2 = 42 \quad 3 * x_0 - 2 * x_1 + 1 * x_2 = 1$$

The solution to this system is ($x_0 = 1$, $x_1 = 2$, $x_2 = 3$). The script calculates these values from `m` via `m.solve()` and plots them on the chart:

```

//@version=6
indicator("Determinants example", "Cramer's Rule")

//@function Solves a system of linear equations with a matching number of unknowns using Cramer's rule.
//@param this An augmented matrix containing the coefficients for each unknown and the results of
// the equations. For example, a row containing the values 2, -1, and 3 represents the equation
// `2 * x0 + (-1) * x1 = 3`, where `x0` and `x1` are the unknown values in the system.
//@returns An array containing solutions for each variable in the system.

```

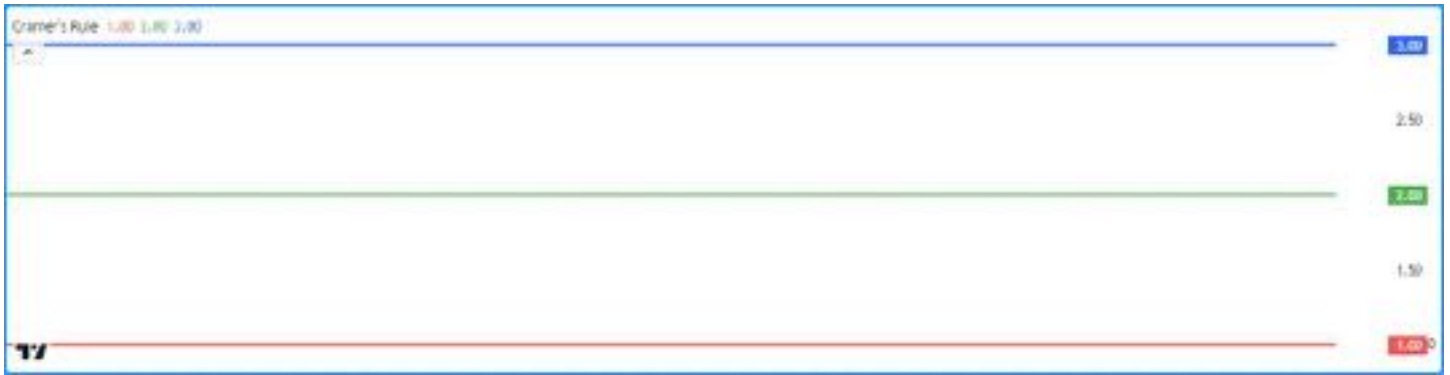



Figure 80: image

```

solve(matrix<float> this) =>
    //@variable The coefficient matrix for the system of equations.
    matrix<float> coefficients = this.submatrix(from_column = 0, to_column = this.columns() - 1)
    //@variable The array of resulting constants for each equation.
    array<float> constants = this.col(this.columns() - 1)
    //@variable An array containing solutions for each unknown in the system.
    array<float> result = array.new<float>()

    //@variable The determinant value of the coefficient matrix.
    float baseDet = coefficients.det()
    matrix<float> modified = na
    for col = 0 to coefficients.columns() - 1
        modified := coefficients.copy()
        modified.add_col(col, constants)
        modified.remove_col(col + 1)

        // Calculate the solution for the column's unknown by dividing the determinant of `modified` by the `baseDet`
        result.push(modified.det() / baseDet)

    result

//@variable A 3x4 matrix containing coefficients and results for a system of three equations.
m = matrix.new<float>()

// Add rows for the following equations:
// Equation 1: 3 * x0 + 4 * x1 - 1 * x2 = 8
// Equation 2: 5 * x0 - 2 * x1 + 1 * x2 = 4
// Equation 3: 2 * x0 - 2 * x1 + 1 * x2 = 1
m.add_row(0, array.from(3.0, 4.0, -1.0, 8.0))
m.add_row(1, array.from(5.0, -2.0, 1.0, 4.0))
m.add_row(2, array.from(2.0, -2.0, 1.0, 1.0))

//@variable An array of solutions to the unknowns in the system of equations represented by `m`.
solutions = solve(m)

plot(solutions.get(0), "x0", color.red, 3) // Plots 1.
plot(solutions.get(1), "x1", color.green, 3) // Plots 2.
plot(solutions.get(2), "x2", color.blue, 3) // Plots 3.

```

Note that:

- Solving systems of equations is particularly useful for *regression analysis*, e.g., linear and polynomial regression.
- Cramer's rule works fine for small systems of equations. However, it's computationally inefficient on larger systems. Other methods, such as Gaussian elimination, are often preferred for such use cases.

matrix.inv() and **matrix.pinv()** For any non-singular square matrix, there is an inverse matrix that yields the identity matrix when multiplied by the original. Inverses have utility in various matrix transformations and solving systems of equations. Scripts can calculate the inverse of a matrix **when one exists** via the `matrix.inv()` function.

For singular (non-invertible) matrices, one can calculate a generalized inverse (pseudoinverse), regardless of whether the matrix is square or has a nonzero determinant, via the `matrix.pinv()` function. Keep in mind that unlike a true inverse, the product of a pseudoinverse and the original matrix does not necessarily equal the identity matrix unless the original matrix *is invertible*.

The following example forms a 2x2 `m` matrix from user inputs, then uses the `m.inv()` and `m.pinv()` methods to calculate the inverse or pseudoinverse of `m`. The script displays the original matrix, its inverse or pseudoinverse, and their product in labels on the chart:



Figure 81: image

```
//@version=6
indicator("Inverse example")

// Element inputs for the 2x2 matrix.
float r0c0 = input.float(4.0, "Row 0, Col 0")
float r0c1 = input.float(3.0, "Row 0, Col 1")
float r1c0 = input.float(2.0, "Row 1, Col 0")
float r1c1 = input.float(1.0, "Row 1, Col 1")

//@function Displays the rows of a matrix in a label with a note.
//@param this The matrix to display.
//@param barIndex The `bar_index` to display the label at.
//@param bgColor The background color of the label.
//@param textColor The color of the label's text.
//@param note The text to display above the rows.
method debugLabel(
    matrix<float> this, int barIndex = bar_index, color bgColor = color.blue,
    color textColor = color.white, string note = ""
) =>
    labelText = note + "\n" + str.toString(this)
    if barstate.ishistory
        label.new(
            barIndex, 0, labelText, color = bgColor, style = label.style_label_center,
            textcolor = textColor, size = size.huge
        )

//@variable A 2x2 matrix of input values.
m = matrix.new<float>()
```

```
// Add input values to `m`.
m.add_row(0, array.from(r0c0, r0c1))
m.add_row(1, array.from(r1c0, r1c1))

//@variable Is `true` if `m` is square with a nonzero determinant, indicating invertibility.
bool isInvertible = m.is_square() and m.det() != 0

//@variable The inverse or pseudoinverse of `m`.
mInverse = isInvertible ? m.inv() : m.pinv()

//@variable The product of `m` and `mInverse`. Returns the identity matrix when `isInvertible` is `true`.
matrix<float> product = m.mult(mInverse)

if bar_index == last_bar_index - 1
    // Display `m`, `mInverse`, and their `product`.
    m.debugLabel(note = "Original")
    mInverse.debugLabel(bar_index + 10, color.purple, note = isInvertible ? "Inverse" : "Pseudoinverse")
    product.debugLabel(bar_index + 20, color.green, note = "Product")
```

Note that:

- This script will only call `m.inv()` when `isInvertible` is `true`, i.e., when `m` is square and has a nonzero determinant. Otherwise, it uses `m.pinv()` to calculate the generalized inverse.

matrix.rank() The *rank* of a matrix represents the number of linearly independent vectors (rows or columns) it contains. In essence, matrix rank measures the number of vectors one cannot express as a linear combination of others, or in other words, the number of vectors that contain **unique** information. Scripts can calculate the rank of a matrix via `matrix.rank()`.

This script identifies the number of linearly independent vectors in two 3x3 matrices (`m1` and `m2`) and plots the values in a separate pane. As we see on the chart, the `m1.rank()` value is 3 because each vector is unique. The `m2.rank()` value, on the other hand, is 1 because it has just one unique vector:



Figure 82: image

```
//@version=6
indicator("Matrix rank example")

//@variable A 3x3 full-rank matrix.
m1 = matrix.new<float>()
//@variable A 3x3 rank-deficient matrix.
m2 = matrix.new<float>()

// Add linearly independent vectors to `m1`.
```

```

m1.add_row(0, array.from(3, 2, 3))
m1.add_row(1, array.from(4, 6, 6))
m1.add_row(2, array.from(7, 4, 9))

// Add linearly dependent vectors to `m2`.
m2.add_row(0, array.from(1, 2, 3))
m2.add_row(1, array.from(2, 4, 6))
m2.add_row(2, array.from(3, 6, 9))

// Plot `matrix.rank()` values.
plot(m1.rank(), color = color.green, linewidth = 3)
plot(m2.rank(), color = color.red, linewidth = 3)

```

Note that:

- The highest rank value a matrix can have is the minimum of its number of rows and columns. A matrix with the maximum possible rank is known as a *full-rank* matrix, and any matrix without full rank is known as a *rank-deficient* matrix.
- The determinants of full-rank square matrices are nonzero, and such matrices have inverses. Conversely, the determinant of a rank-deficient matrix is always 0.
- For any matrix that contains nothing but the same value in each of its elements (e.g., a matrix filled with 0), the rank is always 0 since none of the vectors hold unique information. For any other matrix with distinct values, the minimum possible rank is 1.

Error handling

In addition to usual **compiler** errors, which occur during a script's compilation due to improper syntax, scripts using matrices can raise specific **runtime** errors during their execution. When a script raises a runtime error, it displays a red exclamation point next to the script title. Users can view the error message by clicking this icon.

In this section, we discuss runtime errors that users may encounter while utilizing matrices in their scripts.

The row/column index (xx) is out of bounds, row/column size is (yy).

This runtime error occurs when trying to access indices outside the matrix dimensions with functions including `matrix.get()`, `matrix.set()`, `matrix.fill()`, and `matrix.submatrix()`, as well as some of the functions relating to the rows and columns of a matrix.

For example, this code contains two lines that will produce this runtime error. The `m.set()` method references a `row` index that doesn't exist (2). The `m.submatrix()` method references all column indices up to `to_column - 1`. A `to_column` value of 4 results in a runtime error because the last column index referenced (3) does not exist in `m`:

```

//@version=6
indicator("Out of bounds demo")

//@variable A 2x3 matrix with a max row index of 1 and max column index of 2.
matrix<float> m = matrix.new<float>(2, 3, 0.0)

m.set(row = 2, column = 0, value = 1.0)      // The `row` index is out of bounds on this line. The max value is
m.submatrix(from_column = 1, to_column = 4) // The `to_column` index is invalid on this line. The max value is

if bar_index == last_bar_index - 1
    label.new(bar_index, 0, str.tostring(m), color = color.navy, textcolor = color.white, size = size.huge)

```

Users can avoid this error in their scripts by ensuring their function calls do not reference indices greater than or equal to the number of rows/columns.

The array size does not match the number of rows/columns in the matrix.

When using `matrix.add_row()` and `matrix.add_col()` functions to insert rows and columns into a non-empty matrix, the size of the inserted array must align with the matrix dimensions. The size of an inserted row must match the number of columns, and the size of an inserted column must match the number of rows. Otherwise, the script will raise this runtime error. For example:

```
//@version=6
indicator("Invalid array size demo")

// Declare an empty matrix.
m = matrix.new<float>()

m.add_col(0, array.from(1, 2)) // Add a column. Changes the shape of `m` to 2x1.
m.add_col(1, array.from(1, 2, 3)) // Raises a runtime error because `m` has 2 rows, not 3.

plot(m.col(0).get(1))
```

Note that:

- When `m` is empty, one can insert a row or column array of *any* size, as shown in the first `m.add_col()` line.

Cannot call matrix methods when the ID of matrix is 'na'.

When a matrix variable is assigned to `na`, it means that the variable doesn't reference an existing object. Consequently, one cannot use built-in `matrix.*()` functions and methods with it. For example:

```
//@version=6
indicator("na matrix methods demo")

//@variable A `matrix` variable assigned to `na`.
matrix<float> m = na

mCopy = m.copy() // Raises a runtime error. You can't copy a matrix that doesn't exist.

if bar_index == last_bar_index - 1
    label.new(bar_index, 0, str.tostring(mCopy), color = color.navy, textcolor = color.white, size = size.huge)
```

To resolve this error, assign `m` to a valid matrix instance before using `matrix.*()` functions.

Matrix is too large. Maximum size of the matrix is 100,000 elements.

The total number of elements in a matrix (`matrix.elements_count()`) cannot exceed **100,000**, regardless of its shape. For example, this script will raise an error because it inserts 1000 rows with 101 elements into the `m` matrix:

```
//@version=6
indicator("Matrix too large demo")

var matrix<float> m = matrix.new<float>()

if bar_index == 0
    for i = 1 to 1000
        // This raises an error because the script adds 101 elements on each iteration.
        // 1000 rows * 101 elements per row = 101000 total elements. This is too large.
        m.add_row(m.rows(), array.new<float>(101, i))

plot(m.get(0, 0))
```

The row/column index must be 0 <= from_row/column < to_row/column.

When using `matrix.*()` functions with `from_row/column` and `to_row/column` indices, the `from_*` values must be less than the corresponding `to_*` values, with the minimum possible value being 0. Otherwise, the script will raise a runtime error.

For example, this script shows an attempt to declare a submatrix from a 4x4 `m` matrix with a `from_row` value of 2 and a `to_row` value of 2, which will result in an error:

```
//@version=6
indicator("Invalid from_row, to_row demo")

//@variable A 4x4 matrix filled with a random value.
matrix<float> m = matrix.new<float>(4, 4, math.random())
```

```
matrix<float> mSub = m.submatrix(from_row = 2, to_row = 2) // Raises an error. `from_row` can't equal `to_row`

plot(mSub.get(0, 0))
```

Matrices ‘id1’ and ‘id2’ must have an equal number of rows and columns to be added.

When using `matrix.sum()` and `matrix.diff()` functions, the `id1` and `id2` matrices must have the same number of rows and the same number of columns. Attempting to add or subtract two matrices with mismatched dimensions will raise an error, as demonstrated by this code:

```
//@version=6
indicator("Invalid sum dimensions demo")

//@variable A 2x3 matrix.
matrix<float> m1 = matrix.new<float>(2, 3, 1)
//@variable A 3x4 matrix.
matrix<float> m2 = matrix.new<float>(3, 4, 2)

mSum = matrix.sum(m1, m2) // Raises an error. `m1` and `m2` don't have matching dimensions.

plot(mSum.get(0, 0))
```

The number of columns in the ‘id1’ matrix must equal the number of rows in the matrix (or the number of elements in the array) ‘id2’.

When using `matrix.mult()` to multiply an `id1` matrix by an `id2` matrix or array, the `matrix.rows()` or `array.size()` of `id2` must equal the `matrix.columns()` in `id1`. If they don’t align, the script will raise this error.

For example, this script tries to multiply two 2x3 matrices. While *adding* these matrices is possible, *multiplying* them is not:

```
//@version=6
indicator("Invalid mult dimensions demo")

//@variable A 2x3 matrix.
matrix<float> m1 = matrix.new<float>(2, 3, 1)
//@variable A 2x3 matrix.
matrix<float> m2 = matrix.new<float>(2, 3, 2)

mSum = matrix.mult(m1, m2) // Raises an error. The number of columns in `m1` and rows in `m2` aren't equal.

plot(mSum.get(0, 0))
```

Operation not available for non-square matrices.

Some matrix operations, including `matrix.inv()`, `matrix.det()`, `matrix.eigenvalues()`, and `matrix.eigenvectors()` only work with **square** matrices, i.e., matrices with the same number of rows and columns. When attempting to execute such functions on non-square matrices, the script will raise an error stating the operation isn’t available or that it cannot calculate the result for the matrix `id`. For example:

```
//@version=6
indicator("Non-square demo")

//@variable A 3x5 matrix.
matrix<float> m = matrix.new<float>(3, 5, 1)

plot(m.det()) // Raises a runtime error. You can't calculate the determinant of a 3x5 matrix.
```

[Previous

Arrays](#arrays)[Next

Maps

Introduction

Pine Script™ Maps are collections that store elements in *key-value pairs*. They allow scripts to collect multiple value references associated with unique identifiers (keys).

Unlike arrays and matrices, maps are *unordered* collections. Scripts quickly access a map's values by referencing the keys from the key-value pairs put into them rather than traversing an internal index.

A map's keys can be of any fundamental type or enum type, and its values can be of any available type. Maps cannot directly use other collections (maps, arrays, or matrices) as values, but they can hold UDT instances containing these data structures within their fields. See this section for more information.

As with other collections, maps can contain up to 100,000 elements in total. Since each key-value pair in a map consists of two elements (a unique *key* and its associated *value*), the maximum number of key-value pairs a map can hold is 50,000.

Declaring a map

Pine Script™ uses the following syntax to declare maps:

```
[var/varip ][map<keyType, valueType> ]<identifier> = <expression>
```

Where **<keyType, valueType>** is the map's type template that declares the types of keys and values it will contain, and the **<expression>** returns either a map instance or **na**.

When declaring a map variable assigned to **na**, users must include the **map** keyword followed by a type template to tell the compiler that the variable can accept maps with **keyType** keys and **valueType** values.

For example, this line of code declares a new **myMap** variable that can accept map instances holding pairs of string keys and float values:

```
map<string, float> myMap = na
```

When the **<expression>** is not **na**, the compiler does not require explicit type declaration, as it will infer the type information from the assigned map object.

This line declares a **myMap** variable assigned to an empty map with string keys and float values. Any maps assigned to this variable later must have the same key and value types:

```
myMap = map.new<string, float>()
```

Using var and varip keywords

Users can include the **var** or **varip** keywords to instruct their scripts to declare map variables only on the first chart bar. Variables that use these keywords point to the same map instances on each script iteration until explicitly reassigned.

For example, this script declares a **colorMap** variable assigned to a map that holds pairs of string keys and color values on the first chart bar. The script displays an **oscillator** on the chart and uses the values it put into the **colorMap** on the *first* bar to color the plots on *all* bars:

```
//@version=6
indicator("var map demo")

//@variable A map associating color values with string keys.
var colorMap = map.new<string, color>()

// Put `<string, color>` pairs into `colorMap` on the first bar.
if bar_index == 0
    colorMap.put("Bull", color.green)
    colorMap.put("Bear", color.red)
    colorMap.put("Neutral", color.gray)
```

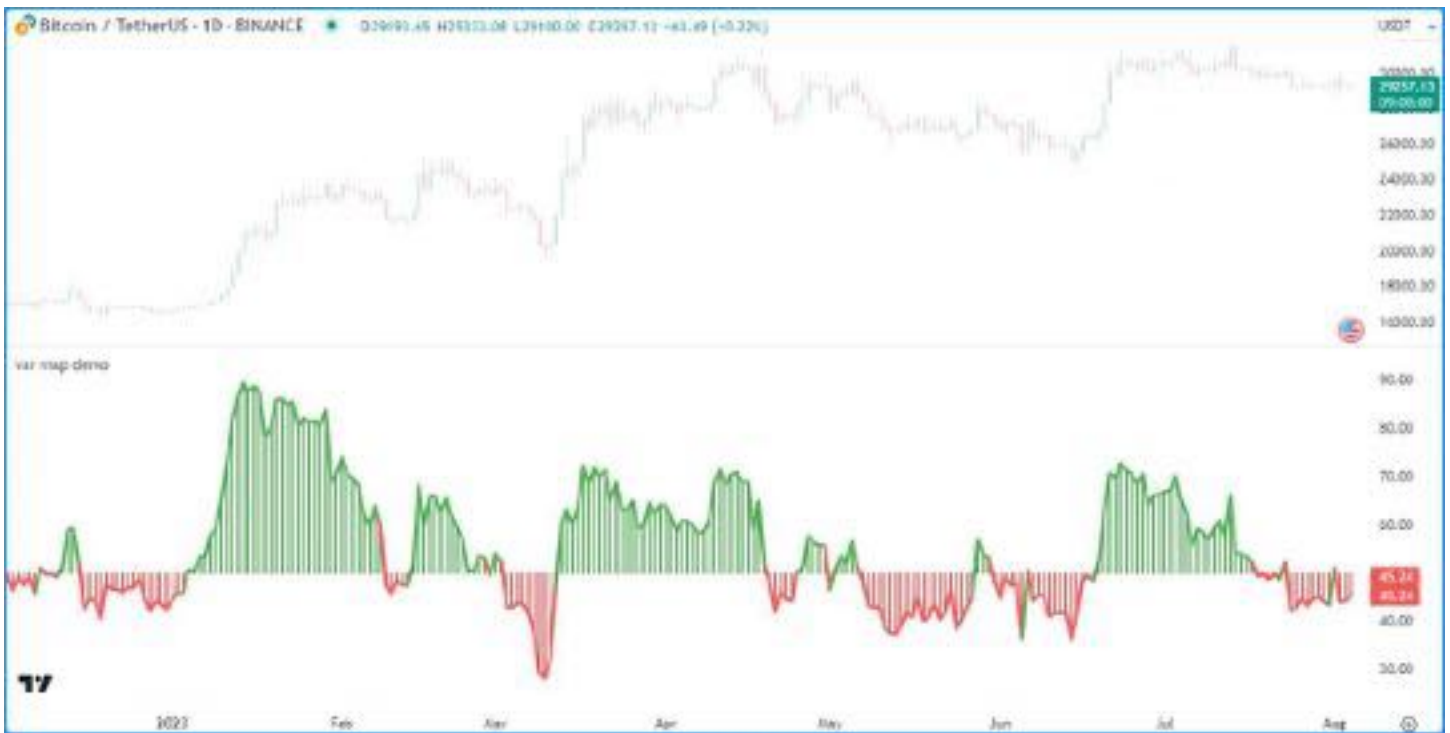


Figure 83: image

```
//@variable The 14-bar RSI of `close`.
float oscillator = ta.rsi(close, 14)

//@variable The color of the `oscillator`.
color oscColor = switch
    oscillator > 50 => colorMap.get("Bull")
    oscillator < 50 => colorMap.get("Bear")
    => colorMap.get("Neutral")

// Plot the `oscillator` using the `oscColor` from our `colorMap`.
plot(oscillator, "Histogram", oscColor, 2, plot.style_histogram, histbase = 50)
plot(oscillator, "Line", oscColor, 3)
```

Reading and writing

Putting and getting key-value pairs

The `map.put()` function is one that map users will utilize quite often, as it's the primary method to put a new key-value pair into a map. It associates the `key` argument with the `value` argument in the call and adds the pair to the map `id`.

If the `key` argument in the `map.put()` call already exists in the map's keys, the new pair passed into the function will **replace** the existing one.

To retrieve the value from a map `id` associated with a given `key`, use `map.get()`. This function returns the value if the `id` map contains the `key`. Otherwise, it returns `na`.

The following example calculates the difference between the `bar_index` values from when `close` was last rising and falling over a given `length` with the help of `map.put()` and `map.get()` methods. The script puts a `("Rising", bar_index)` pair into the `data` map when the price is rising and puts a `("Falling", bar_index)` pair into the map when the price is falling. It then puts a pair containing the "Difference" between the "Rising" and "Falling" values into the map and plots its value on the chart:

```
//@version=6
indicator("Putting and getting demo")

//@variable The length of the `ta.rising()` and `ta.falling()` calculation.
```




Figure 84: image

```
int length = input.int(2, "Length")

//@variable A map associating `string` keys with `int` values.
var data = map.new<string, int>()

// Put a new ("Rising", `bar_index`) pair into the `data` map when `close` is rising.
if ta.rising(close, length)
    data.put("Rising", bar_index)
// Put a new ("Falling", `bar_index`) pair into the `data` map when `close` is falling.
if ta.falling(close, length)
    data.put("Falling", bar_index)

// Put the "Difference" between current "Rising" and "Falling" values into the `data` map.
data.put("Difference", data.get("Rising") - data.get("Falling"))

//@variable The difference between the last "Rising" and "Falling" `bar_index`.
int index = data.get("Difference")

//@variable Returns `color.green` when `index` is positive, `color.red` when negative, and `color.gray` otherwise.
color indexColor = index > 0 ? color.green : index < 0 ? color.red : color.gray

plot(index, color = indexColor, style = plot.style_columns)
```

Note that:

- This script replaces the values associated with the “Rising”, “Falling”, and “Difference” keys on successive `data.put()` calls, as each of these keys is unique and can only appear once in the `data` map.
- Replacing the pairs in a map does not change the internal *insertion order* of its keys. We discuss this further in the next section.

Similar to working with other collections, when putting a value of a *special type* (line, linefill, box, polyline, label, table, or `chart.point`) or a user-defined type into a map, it’s important to note the inserted pair’s **value** points to that same object without copying it. Modifying the value referenced by a key-value pair will also affect the original object.

For example, this script contains a custom `ChartData` type with `o`, `h`, `l`, and `c` fields. On the first chart bar, the script

declares a `myMap` variable and adds the pair ("A", `myData`), where `myData` is a `ChartData` instance with initial field values of `na`. It adds the pair ("B", `myData`) to `myMap` and updates the object from this pair on every bar via the user-defined `update()` method.

Each change to the object with the "B" key affects the one referenced by the "A" key, as shown by the candle plot of the "A" object's fields:



Figure 85: image

```
//@version=6
indicator("Putting and getting objects demo")

//@type A custom type to hold OHLC data.
type ChartData
    float o
    float h
    float l
    float c

//@function Updates the fields of a `ChartData` object.
method update(ChartData this) =>
    this.o := open
    this.h := high
    this.l := low
    this.c := close

//@variable A new `ChartData` instance declared on the first bar.
var myData = ChartData.new()
//@variable A map associating `string` keys with `ChartData` instances.
var myMap = map.new<string, ChartData>()

// Put a new pair with the "A" key into `myMap` only on the first bar.
if bar_index == 0
    myMap.put("A", myData)

// Put a pair with the "B" key into `myMap` on every bar.
myMap.put("B", myData)

//@variable The `ChartData` value associated with the "A" key in `myMap`.
ChartData oldest = myMap.get("A")
//@variable The `ChartData` value associated with the "B" key in `myMap`.
ChartData newest = myMap.get("B")
```

```
// Update `newest`. Also affects `oldest` and `myData` since they all reference the same `ChartData` object.
newest.update()

// Plot the fields of `oldest` as candles.
plotcandle(oldest.o, oldest.h, oldest.l, oldest.c)
```

Note that:

- This script would behave differently if it passed a copy of `myData` into each `myMap.put()` call. For more information, see this section of our User Manual's page on objects.

Inspecting keys and values

map.keys() and **map.values()** To retrieve all keys and values put into a map, use `map.keys()` and `map.values()`. These functions copy all key/value references within a map `id` to a new array object. Modifying the array returned from either of these functions does not affect the `id` map.

Although maps are *unordered* collections, Pine Script™ internally maintains the *insertion order* of a map's key-value pairs. As a result, the `map.keys()` and `map.values()` functions always return arrays with their elements ordered based on the `id` map's insertion order.

The script below demonstrates this by displaying the key and value arrays from an `m` map in a label once every 50 bars. As we see on the chart, the order of elements in each array returned by `m.keys()` and `m.values()` aligns with the insertion order of the key-value pairs in `m`:



Figure 86: image

```
//@version=6
indicator("Keys and values demo")

if bar_index % 50 == 0
    //@variable A map containing pairs of `string` keys and `float` values.
    m = map.new<string, float>()

    // Put pairs into `m`. The map will maintain this insertion order.
    m.put("First", math.round(math.random(0, 100)))
    m.put("Second", m.get("First") + 1)
    m.put("Third", m.get("Second") + 1)

    //@variable An array containing the keys of `m` in their insertion order.
    array<string> keys = m.keys()
    //@variable An array containing the values of `m` in their insertion order.
    array<float> values = m.values()

    //@variable A label displaying the `size` of `m` and the `keys` and `values` arrays.
    label debugLabel = label.new(
```

```

    bar_index, 0,
    str.format("Pairs: {0}\nKeys: {1}\nValues: {2}", m.size(), keys, values),
    color = color.navy, style = label.style_label_center,
    textcolor = color.white, size = size.huge
)

```

Note that:

- The value with the “First” key is a random whole number between 0 and 100. The “Second” value is one greater than the “First”, and the “Third” value is one greater than the “Second”.

It’s important to note a map’s internal insertion order **does not** change when replacing its key-value pairs. The locations of the new elements in the `keys()` and `values()` arrays will be the same as the old elements in such cases. The only exception is if the script completely removes the key beforehand.

Below, we’ve added a line of code to put a new value with the “Second” key into the `m` map, overwriting the previous value associated with that key. Although the script puts this new pair into the map *after* the one with the “Third” key, the pair’s key and value are still second in the `keys` and `values` arrays since the key was already present in *mbefore* the change:

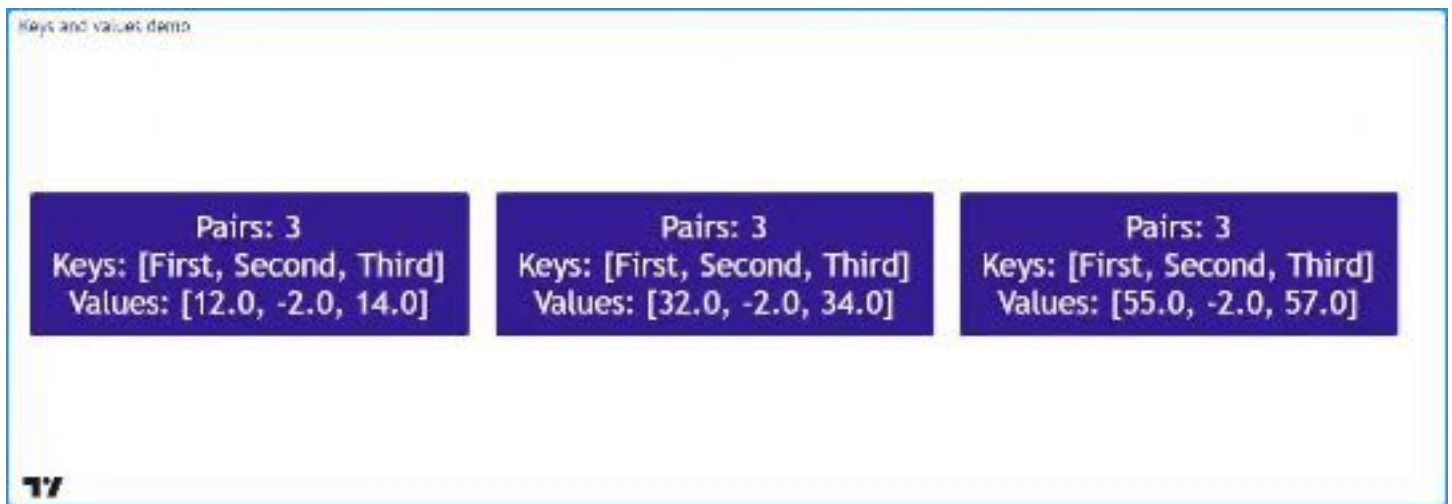


Figure 87: image

```

//@version=6
indicator("Keys and values demo")

if bar_index % 50 == 0
    //@variable A map containing pairs of `string` keys and `float` values.
    m = map.new<string, float>()

    // Put pairs into `m`. The map will maintain this insertion order.
    m.put("First", math.round(math.random(0, 100)))
    m.put("Second", m.get("First") + 1)
    m.put("Third", m.get("Second") + 1)

    // Overwrite the "Second" pair in `m`. This will NOT affect the insertion order.
    // The key and value will still appear second in the `keys` and `values` arrays.
    m.put("Second", -2)

    //@variable An array containing the keys of `m` in their insertion order.
    array<string> keys = m.keys()
    //@variable An array containing the values of `m` in their insertion order.
    array<float> values = m.values()

    //@variable A label displaying the `size` of `m` and the `keys` and `values` arrays.
    label debugLabel = label.new(
        bar_index, 0,

```

```

        str.format("Pairs: {0}\nKeys: {1}\nValues: {2}", m.size(), keys, values),
        color = color.navy, style = label.style_label_center,
        textcolor = color.white, size = size.huge
    )
)

```

map.contains() To check if a specific key exists within a map id, use `map.contains()`. This function is a convenient alternative to calling `array.includes()` on the array returned from `map.keys()`.

For example, this script checks if various keys exist within an `m` map, then displays the results in a label:

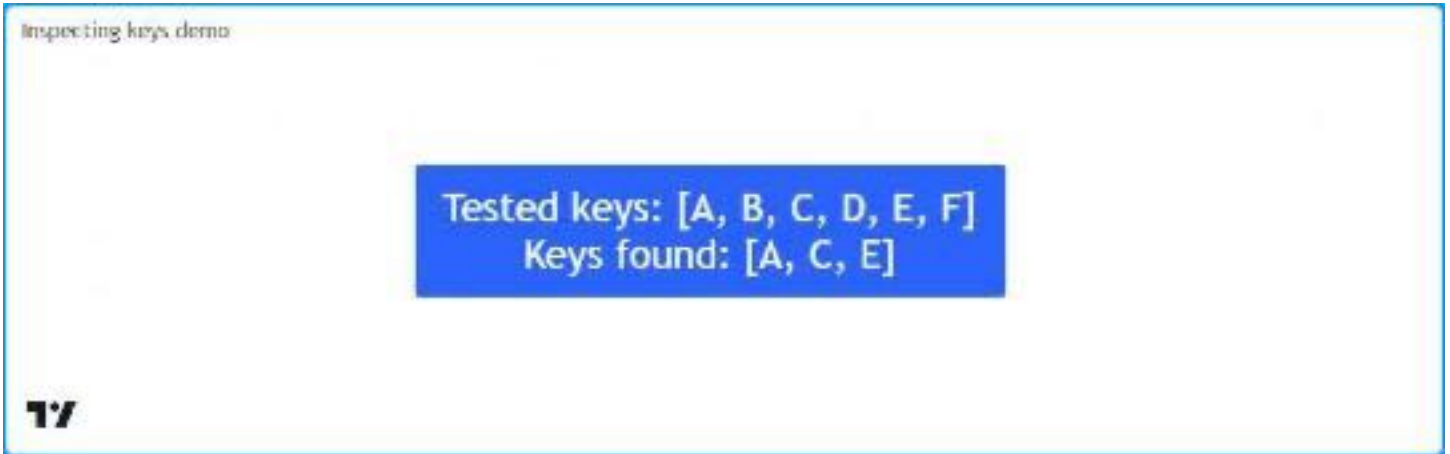


Figure 88: image

```

//@version=6
indicator("Inspecting keys demo")

//@variable A map containing `string` keys and `string` values.
m = map.new<string, string>()

// Put key-value pairs into the map.
m.put("A", "B")
m.put("C", "D")
m.put("E", "F")

//@variable An array of keys to check for in `m`.
array<string> testKeys = array.from("A", "B", "C", "D", "E", "F")

//@variable An array containing all elements from `testKeys` found in the keys of `m`.
array<string> mappedKeys = array.new<string>()

for key in testKeys
    // Add the `key` to `mappedKeys` if `m` contains it.
    if m.contains(key)
        mappedKeys.push(key)

//@variable A string representing the `testKeys` array and the elements found within the keys of `m`.
string testText = str.format("Tested keys: {0}\nKeys found: {1}", testKeys, mappedKeys)

if bar_index == last_bar_index - 1
    //@variable Displays the `testText` in a label at the `bar_index` before the last.
    label debugLabel = label.new(
        bar_index, 0, testText, style = label.style_label_center,
        textcolor = color.white, size = size.huge
    )
)

```

Removing key-value pairs

To remove a specific key-value pair from a map `id`, use `map.remove()`. This function removes the **key** and its associated value from the map while preserving the insertion order of other key-value pairs. It returns the removed value if the map contained the **key**. Otherwise, it returns `na`.

To remove all key-value pairs from a map `id` at once, use `map.clear()`.

The following script creates a new `m` map, puts key-value pairs into the map, uses `m.remove()` within a loop to remove each valid key listed in the `removeKeys` array, then calls `m.clear()` to remove all remaining key-value pairs. It uses a custom `debugLabel()` method to display the size, keys, and values of `m` after each change:



Figure 89: image

```
//@version=6
indicator("Removing key-value pairs demo")

//@function Returns a label to display the keys and values from a map.
method debugLabel(
    map<string, int> this, int barIndex = bar_index,
    color bgColor = color.blue, string note = ""
) =>
    //@variable A string representing the size, keys, and values in `this` map.
    string repr = str.format(
        "{0}\nSize: {1}\nKeys: {2}\nValues: {3}",
        note, this.size(), str.toString(this.keys()), str.toString(this.values())
    )
    label.new(
        barIndex, 0, repr, color = bgColor, style = label.style_label_center,
        textcolor = color.white, size = size.huge
    )

if bar_index == last_bar_index - 1
    //@variable A map containing `string` keys and `int` values.
    m = map.new<string, int>()

    // Put key-value pairs into `m`.
    for [i, key] in array.from("A", "B", "C", "D", "E")
        m.put(key, i)
    m.debugLabel(bar_index, color.green, "Added pairs")

    //@variable An array of keys to remove from `m`.
    array<string> removeKeys = array.from("B", "B", "D", "F", "a")

    // Remove each `key` in `removeKeys` from `m`.
    for key in removeKeys
        m.remove(key)
    m.debugLabel(bar_index + 10, color.red, "Removed pairs")
```

```
// Remove all remaining keys from `m`.
m.clear()
m.debugLabel(bar_index + 20, color.purple, "Cleared the map")
```

Note that:

- Not all strings in the `removeKeys` array were present in the keys of `m`. Attempting to remove non-existent keys (“F”, “a”, and the second “B” in this example) has no effect on a map’s contents.

Combining maps

Scripts can combine two maps via `map.put_all()`. This function puts *all* key-value pairs from the `id2` map, in their insertion order, into the `id1` map. As with `map.put()`, if any keys in `id2` are also present in `id1`, this function **replaces** the key-value pairs that contain those keys without affecting their initial insertion order.

This example contains a user-defined `hexMap()` function that maps decimal int keys to string representations of their hexadecimal forms. The script uses this function to create two maps, `mapA` and `mapB`, then uses `mapA.put_all(mapB)` to put all key-value pairs from `mapB` into `mapA`.

The script uses a custom `debugLabel()` function to display labels showing the keys and values of `mapA` and `mapB`, then another label displaying the contents of `mapA` after putting all key-value pairs from `mapB` into it:

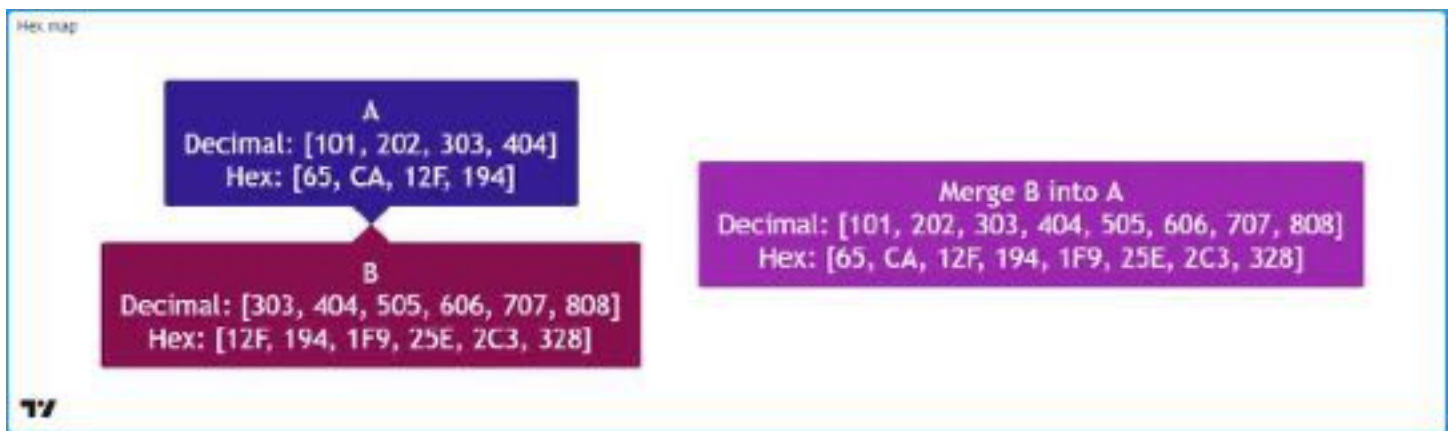


Figure 90: image

```
//@version=6
indicator("Combining maps demo", "Hex map")

//@variable An array of string hex digits.
var array<string> hexDigits = str.split("0123456789ABCDEF", "")

//@function Returns a hexadecimal string for the specified `value`.
hex(int value) =>
    //@variable A string representing the hex form of the `value`.
    string result = ""
    //@variable A temporary value for digit calculation.
    int tempValue = value
    while tempValue > 0
        //@variable The next integer digit.
        int digit = tempValue % 16
        // Add the hex form of the `digit` to the `result`.
        result := hexDigits.get(digit) + result
        // Divide the `tempValue` by the base.
        tempValue := int(tempValue / 16)
    result

//@function Returns a map holding the `numbers` as keys and their `hex` strings as values.
hexMap(array<int> numbers) =>
```



```

//@variable A map associating `int` keys with `string` values.
result = map.new<int, string>()
for number in numbers
    // Put a pair containing the `number` and its `hex()` representation into the `result`.
    result.put(number, hex(number))
result

//@function Returns a label to display the keys and values of a hex map.
debugLabel(
    map<int, string> this, int barIndex = bar_index, color bgColor = color.blue,
    string style = label.style_label_center, string note = ""
) =>
    string repr = str.format(
        "{0}\nDecimal: {1}\nHex: {2}",
        note, str.toString(this.keys()), str.toString(this.values())
    )
    label.new(
        barIndex, 0, repr, color = bgColor, style = style,
        textcolor = color.white, size = size.huge
    )

if bar_index == last_bar_index - 1
    //@variable A map with decimal `int` keys and hexadecimal `string` values.
    map<int, string> mapA = hexMap(array.from(101, 202, 303, 404))
    debugLabel(mapA, bar_index, color.navy, label.style_label_down, "A")

    //@variable A map containing key-value pairs to add to `mapA`.
    map<int, string> mapB = hexMap(array.from(303, 404, 505, 606, 707, 808))
    debugLabel(mapB, bar_index, color.maroon, label.style_label_up, "B")

    // Put all pairs from `mapB` into `mapA`.
    mapA.put_all(mapB)
    debugLabel(mapA, bar_index + 10, color.purple, note = "Merge B into A")

```

Looping through a map

There are several ways scripts can iteratively access the keys and values in a map. For example, one could loop through a map's `keys()` array and `get()` the value for each **key**, like so:

```

for key in thisMap.keys()
    value = thisMap.get(key)

```

However, we recommend using a `for...in` loop directly on a map, as it iterates over the map's key-value pairs in their insertion order, returning a tuple containing the next pair's key and value on each iteration.

For example, this line of code loops through each **key** and **value** in `thisMap`, starting from the first key-value pair put into it:

```

for [key, value] in thisMap

```

Let's use this structure to write a script that displays a map's key-value pairs in a table. In the example below, we've defined a custom `toTable()` method that creates a table, then uses a `for...in` loop to iterate over the map's key-value pairs and populate the table's cells. The script uses this method to visualize a map containing **length-bar averages** of price and volume data:

```

//@version=6
indicator("Looping through a map demo", "Table of averages")

//@variable The length of the moving average.
int length = input.int(20, "Length")
//@variable The size of the table text.
string txtSize = input.string(
    size.huge, "Text size",

```




Figure 91: image

```

options = [size.auto, size.tiny, size.small, size.normal, size.large, size.huge]
)

//@function Displays the pairs of `this` map within a table.
//@param    this A map with `string` keys and `float` values.
//@param    position The position of the table on the chart.
//@param    header The string to display on the top row of the table.
//@param    textSize The size of the text in the table.
//@returns  A new `table` object with cells displaying each pair in `this`.
method toTable(
    map<string, float> this, string position = position.middle_center, string header = na,
    string textSize = size.huge
) =>
    // Color variables
    borderColor = #000000
    headerColor = color.rgb(1, 88, 80)
    pairColor    = color.maroon
    textColor    = color.white

    //@variable A table that displays the key-value pairs of `this` map.
    table result = table.new(
        position, this.size() + 1, 3, border_width = 2, border_color = borderColor
    )
    // Initialize top and side header cells.
    result.cell(1, 0, header, bgcolor = headerColor, text_color = textColor, text_size = textSize)
    result.merge_cells(1, 0, this.size(), 0)
    result.cell(0, 1, "Key", bgcolor = headerColor, text_color = textColor, text_size = textSize)
    result.cell(0, 2, "Value", bgcolor = headerColor, text_color = textColor, text_size = textSize)

    //@variable The column index of the table. Updates on each loop iteration.
    int col = 1

    // Loop over each `key` and `value` from `this` map in the insertion order.
    for [key, value] in this

```

```

// Initialize a `key` cell in the `result` table on row 1.
result.cell(
    col, 1, str.toString(key), bgcolor = color.maroon,
    text_color = color.white, text_size = textSize
)
// Initialize a `value` cell in the `result` table on row 2.
result.cell(
    col, 2, str.toString(value), bgcolor = color.maroon,
    text_color = color.white, text_size = textSize
)
// Move to the next column index.
col += 1
result // Return the `result` table.

//@variable A map with `string` keys and `float` values to hold `length`-bar averages.
averages = map.new<string, float>()

// Put key-value pairs into the `averages` map.
averages.put("Open", ta.sma(open, length))
averages.put("High", ta.sma(high, length))
averages.put("Low", ta.sma(low, length))
averages.put("Close", ta.sma(close, length))
averages.put("Volume", ta.sma(volume, length))

//@variable The text to display at the top of the table.
string headerText = str.format("{0} {1}-bar averages", "" + syminfo.tickerid + "", length)
// Display the `averages` map in a `table` with the `headerText`.
averages.toTable(header = headerText, textSize = txtSize)

```

Copying a map

Shallow copies

Scripts can make a *shallow copy* of a map id using the `map.copy()` function. Modifications to a shallow copy do not affect the original id map or its internal insertion order.

For example, this script constructs an `m` map with the keys “A”, “B”, “C”, and “D” assigned to four random values between 0 and 10. It then creates an `mCopy` map as a shallow copy of `m` and updates the values associated with its keys. The script displays the key-value pairs in `m` and `mCopy` on the chart using our custom `debugLabel()` method:

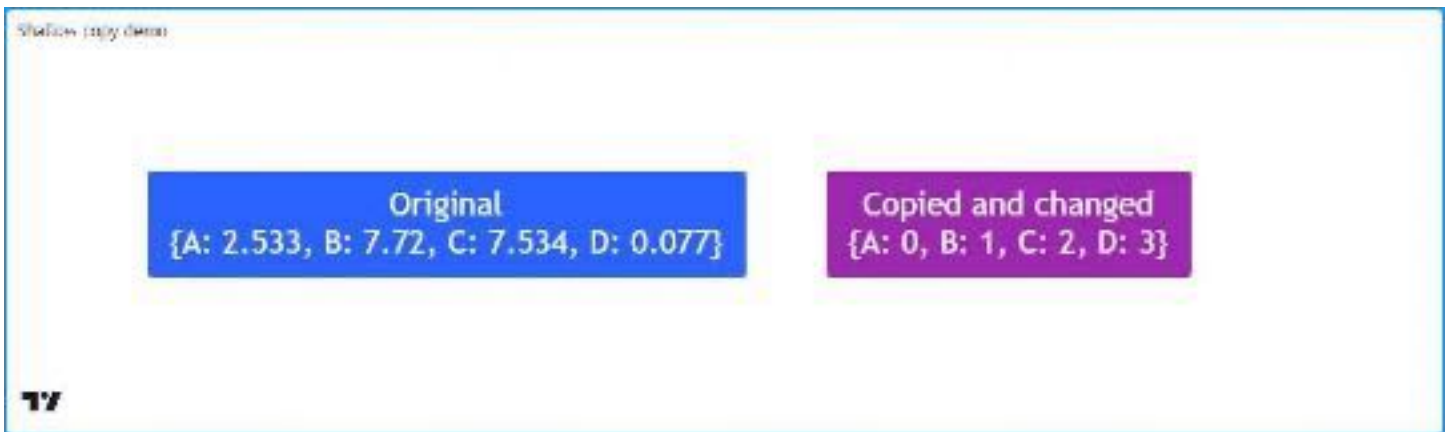


Figure 92: image

```

//@version=6
indicator("Shallow copy demo")

//@function Displays the key-value pairs of `this` map in a label.
method debugLabel(

```

```

    map<string, float> this, int barIndex = bar_index, color bgColor = color.blue,
    color textColor = color.white, string note = ""
) =>
    //@variable The text to display in the label.
    labelText = note + "\n{"
    for [key, value] in this
        labelText += str.format("{0}: {1}, ", key, value)
    labelText := str.replace(labelText, ", ", "}", this.size() - 1)

    if barstate.ishistory
        label result = label.new(
            barIndex, 0, labelText, color = bgColor, style = label.style_label_center,
            textcolor = textColor, size = size.huge
        )

if bar_index == last_bar_index - 1
    //@variable A map of `string` keys and random `float` values.
    m = map.new<string, float>()

    // Assign random values to an array of keys in `m`.
    for key in array.from("A", "B", "C", "D")
        m.put(key, math.random(0, 10))

    //@variable A shallow copy of `m`.
    mCopy = m.copy()

    // Assign the insertion order value `i` to each `key` in `mCopy`.
    for [i, key] in mCopy.keys()
        mCopy.put(key, i)

    // Display the labels.
    m.debugLabel(bar_index, note = "Original")
    mCopy.debugLabel(bar_index + 10, color.purple, note = "Copied and changed")

```

Deep copies

While a shallow copy will suffice when copying maps that have values of a fundamental type or enum type, it's crucial to understand that shallow copies of a map holding values of a *special type* (line, linefill, box, polyline, label, table, chart.point or a UDT) point to the same objects as the original. Modifying the objects referenced by a shallow copy will affect the instances referenced by the original map and vice versa.

To ensure changes to objects referenced by a copied map do not affect instances referenced in other locations, one can make a *deep copy* by creating a new map with key-value pairs containing copies of each value in the original map.

This example creates an **original** map of string keys and label values and puts a key-value pair into it. The script copies the map to a **shallow** variable via the built-in copy() method, then to a **deep** variable using a custom deepCopy() method.

As we see from the chart, changes to the label retrieved from the **shallow** copy also affect the instance referenced by the **original** map, but changes to the one from the **deep** copy do not:

```

//@version=6
indicator("Deep copy demo")

//@function Returns a deep copy of `this` map.
method deepCopy(map<string, label> this) =>
    //@variable A deep copy of `this` map.
    result = map.new<string, label>()
    // Add key-value pairs with copies of each `value` to the `result`.
    for [key, value] in this
        result.put(key, value.copy())
    result //Return the `result`.

```

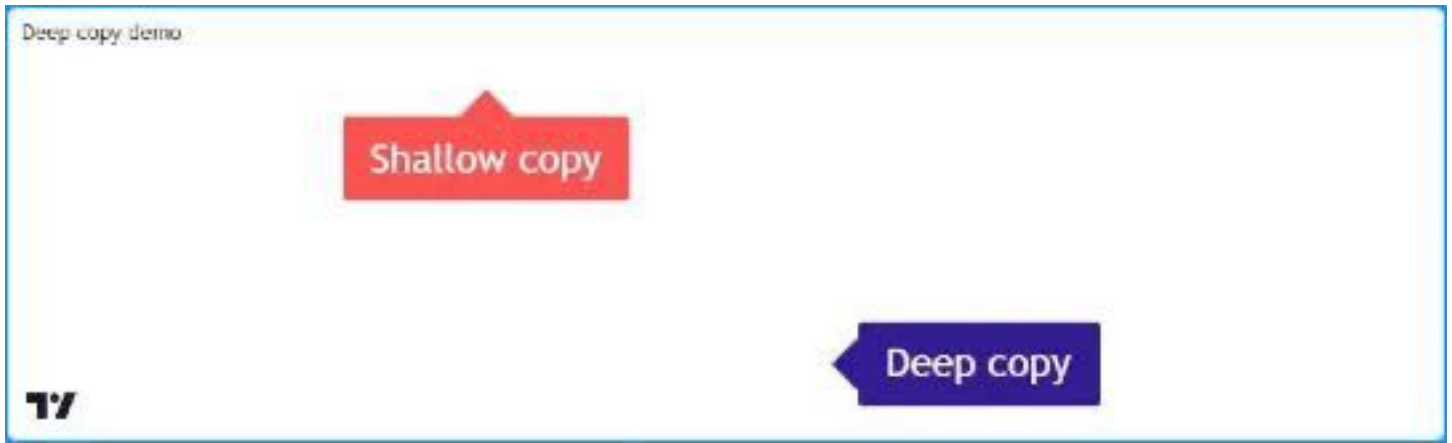


Figure 93: image

```
//@variable A map containing `string` keys and `label` values.
var original = map.new<string, label>()

if bar_index == last_bar_index - 1
  // Put a new key-value pair into the `original` map.
  map.put(
    original, "Test",
    label.new(bar_index, 0, "Original", textcolor = color.white, size = size.huge)
  )

  //@variable A shallow copy of the `original` map.
  map<string, label> shallow = original.copy()
  //@variable A deep copy of the `original` map.
  map<string, label> deep = original.deepCopy()

  //@variable The "Test" label from the `shallow` copy.
  label shallowLabel = shallow.get("Test")
  //@variable The "Test" label from the `deep` copy.
  label deepLabel = deep.get("Test")

  // Modify the "Test" label's `y` attribute in the `original` map.
  // This also affects the `shallowLabel`.
  original.get("Test").set_y(label.all.size())

  // Modify the `shallowLabel`. Also modifies the "Test" label in the `original` map.
  shallowLabel.set_text("Shallow copy")
  shallowLabel.set_color(color.red)
  shallowLabel.set_style(label.style_label_up)

  // Modify the `deepLabel`. Does not modify any other label instance.
  deepLabel.set_text("Deep copy")
  deepLabel.set_color(color.navy)
  deepLabel.set_style(label.style_label_left)
  deepLabel.set_x(bar_index + 5)
```

Note that:

- The `deepCopy()` method loops through the `original` map, copying each `value` and putting key-value pairs containing the copies into a new map instance.

Scope and history

As with other collections in Pine, map variables leave historical trails on each bar, allowing a script to access past map instances assigned to a variable using the history-referencing operator `[]`. Scripts can also assign maps to global variables and

interact with them from the scopes of functions, methods, and conditional structures.

As an example, this script uses a global map and its history to calculate an aggregate set of EMAs. It declares a `globalData` map of int keys and float values, where each key in the map corresponds to the length of each EMA calculation. The user-defined `update()` function calculates each `key`-length EMA by mixing the values from the `previous` map assigned to `globalData` with the current `source` value.

The script plots the maximum and minimum values in the global map's `values()` array and the value from `globalData.get(50)` (i.e., the 50-bar EMA):



Figure 94: image

```
//@version=6
indicator("Scope and history demo", overlay = true)

//@variable The source value for EMA calculation.
float source = input.source(close, "Source")

//@variable A map containing global key-value pairs.
globalData = map.new<int, float>()

//@function Calculates a set of EMAs and updates the key-value pairs in `globalData`.
update() =>
    //@variable The previous map instance assigned to `globalData`.
    map<int, float> previous = globalData[1]

    // Put key-value pairs with keys 10-200 into `globalData` if `previous` is `na`.
    if na(previous)
        for i = 10 to 200
            globalData.put(i, source)
    else
        // Iterate each `key` and `value` in the `previous` map.
        for [key, value] in previous
            //@variable The smoothing parameter for the `key`-length EMA.
            float alpha = 2.0 / (key + 1.0)
            //@variable The `key`-length EMA value.
            float ema = (1.0 - alpha) * value + alpha * source
            // Put the `key`-length `ema` into the `globalData` map.
            globalData.put(key, ema)

    // Update the `globalData` map.
    update()
```

```
//@variable The array of values from `globalData` in their insertion order.
array<float> values = globalData.values()

// Plot the max EMA, min EMA, and 50-bar EMA values.
plot(values.max(), "Max EMA", color.green, 2)
plot(values.min(), "Min EMA", color.red, 2)
plot(globalData.get(50), "50-bar EMA", color.orange, 3)
```

Maps of other collections

Maps cannot directly use other maps, arrays, or matrices as values, but they can hold values of a user-defined type that contains collections within its fields.

For example, suppose we want to create a “2D” map that uses string keys to access *nested maps* that hold pairs of string keys and float values. Since maps cannot use other collections as values, we will first create a *wrapper type* with a field to hold a `map<string, float>` instance, like so:

```
//@type A wrapper type for maps with `string` keys and `float` values.
type Wrapper
    map<string, float> data
```

With our `Wrapper` type defined, we can create maps of string keys and `Wrapper` values, where the `data` field of each value in the map points to a `map<string, float>` instance:

```
mapOfMaps = map.new<string, Wrapper>()
```

The script below uses this concept to construct a map containing maps that hold OHLCV data requested from multiple tickers. The user-defined `requestData()` function requests price and volume data from a ticker, creates a `<string, float>` map, puts the data into it, then returns a `Wrapper` instance containing the new map.

The script puts the results from each call to `requestData()` into the `mapOfMaps`, then creates a string representation of the nested maps with a user-defined `toString()` method, which it displays on the chart in a label:

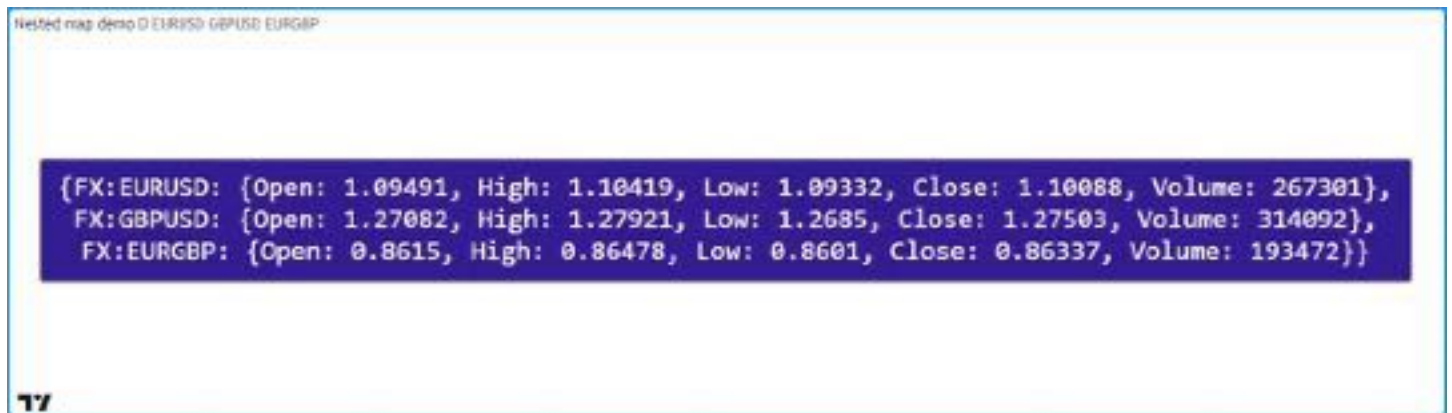


Figure 95: image

```
//@version=6
indicator("Nested map demo")

//@variable The timeframe of the requested data.
string tf = input.timeframe("D", "Timeframe")
// Symbol inputs.
string symbol1 = input.symbol("EURUSD", "Symbol 1")
string symbol2 = input.symbol("GBPUSD", "Symbol 2")
string symbol3 = input.symbol("EURGBP", "Symbol 3")

//@type A wrapper type for maps with `string` keys and `float` values.
type Wrapper
    map<string, float> data
```

```

//@function Returns a wrapped map containing OHLCV data from the `tickerID` at the `timeframe`.
requestData(string tickerID, string timeframe) =>
    // Request a tuple of OHLCV values from the specified ticker and timeframe.
    [o, h, l, c, v] = request.security(
        tickerID, timeframe,
        [open, high, low, close, volume]
    )
    //@variable A map containing requested OHLCV data.
    result = map.new<string, float>()
    // Put key-value pairs into the `result`.
    result.put("Open", o)
    result.put("High", h)
    result.put("Low", l)
    result.put("Close", c)
    result.put("Volume", v)
    //Return the wrapped `result`.
    Wrapper.new(result)

//@function Returns a string representing `this` map of `string` keys and `Wrapper` values.
method toString(map<string, Wrapper> this) =>
    //@variable A string representation of `this` map.
    string result = "{"

    // Iterate over each `key1` and associated `wrapper` in `this`.
    for [key1, wrapper] in this
        // Add `key1` to the `result`.
        result += key1

        //@variable A string representation of the `wrapper.data` map.
        string innerStr = ": {"
        // Iterate over each `key2` and associated `value` in the wrapped map.
        for [key2, value] in wrapper.data
            // Add the key-value pair's representation to `innerStr`.
            innerStr += str.format("{0}: {1}, ", key2, str.toString(value))

        // Replace the end of `innerStr` with `"}"` and add to `result`.
        result += str.replace(innerStr, ", ", "},\n", wrapper.data.size() - 1)

    // Replace the blank line at the end of `result` with `"}"`.
    result := str.replace(result, ",\n", "}", this.size() - 1)
    result

//@variable A map of wrapped maps containing OHLCV data from multiple tickers.
var mapOfMaps = map.new<string, Wrapper>()

//@variable A label showing the contents of the `mapOfMaps`.
var debugLabel = label.new(
    bar_index, 0, color = color.navy, textcolor = color.white, size = size.huge,
    style = label.style_label_center, text_font_family = font.family_monospace
)

// Put wrapped maps into `mapOfMaps`.
mapOfMaps.put(symbol1, requestData(symbol1, tf))
mapOfMaps.put(symbol2, requestData(symbol2, tf))
mapOfMaps.put(symbol3, requestData(symbol3, tf))

// Update the label.
debugLabel.set_text(mapOfMaps.toString())
debugLabel.set_x(bar_index)

```

Alerts

Introduction

TradingView alerts run 24x7 on our servers and do not require users to be logged in to execute. Alerts are created from the charts user interface (*UI*). You will find all the information necessary to understand how alerts work and how to create them from the charts UI in the Help Center's About TradingView alerts page.

Some of the alert types available on TradingView (*generic alerts*, *drawing alerts* and *script alerts* on order fill events) are created from symbols or scripts loaded on the chart and do not require specific coding. Any user can create these types of alerts from the charts UI.

Other types of alerts (*script alerts* triggering on *alert()* function calls, and *alertcondition()* alerts) require specific Pine Script™ code to be present in a script to create an *alert event* before script users can create alerts from them using the charts UI. Additionally, while script users can create *script alerts* triggering on *order fill events* from the charts UI on any strategy loaded on their chart, Programmers can specify explicit order fill alert messages in their script for each type of order filled by the broker emulator.

This page covers the different ways Pine Script™ programmers can code their scripts to create alert events from which script users will in turn be able to create alerts from the charts UI. We will cover:

- How to use the `alert()` function to *alert()* function calls in indicators or strategies, which can then be included in *script alerts* created from the charts UI.
- How to add custom alert messages to be included in *script alerts* triggering on the *order fill events* of strategies.
- How to use the `alertcondition()` function to generate, in indicators only, *alertcondition()* events which can then be used to create *alertcondition()* alerts from the charts UI.

Keep in mind that:

- No alert-related Pine Script™ code can create a running alert in the charts UI; it merely creates alert events which can then be used by script users to create running alerts from the charts UI.
- Alerts only trigger in the realtime bar. The operational scope of Pine Script™ code dealing with any type of alert is therefore restricted to realtime bars only.
- When an alert is created in the charts UI, TradingView saves a mirror image of the script and its inputs, along with the chart's main symbol and timeframe to run the alert on its servers. Subsequent changes to your script's inputs or the chart will thus not affect running alerts previously created from them. If you want any changes to your context to be reflected in a running alert's behavior, you will need to delete the alert and create a new one in the new context.

Background

The different methods Pine programmers can use today to create alert events in their script are the result of successive enhancements deployed throughout Pine Script™'s evolution. The `alertcondition()` function, which works in indicators only, was the first feature allowing Pine Script™ programmers to create alert events. Then came order fill alerts for strategies, which trigger when the broker emulator creates *order fill events*. *Order fill events* require no special code for script users to create alerts on them, but by way of the `alert_message` parameter for order-generating `strategy.*()` functions, programmers can customize the message of alerts triggering on *order fill events* by defining a distinct alert message for any number of order fulfillment events.

The `alert()` function is the most recent addition to Pine Script™. It more or less supersedes `alertcondition()`, and when used in strategies, provides a useful complement to alerts on *order fill events*.

Which type of alert is best?

For Pine Script™ programmers, the `alert()` function will generally be easier and more flexible to work with. Contrary to `alertcondition()`, it allows for dynamic alert messages, works in both indicators and strategies and the programmer decides on the frequency of `alert()` events.

While `alert()` calls can be generated on any logic programmable in Pine, including when orders are **sent** to the broker emulator in strategies, they cannot be coded to trigger when orders are **executed** (or *filled*) because after orders are sent to the broker emulator, the emulator controls their execution and does not report fill events back to the script directly.

When a script user wants to generate an alert on a strategy's order fill events, he must include those events when creating a *script alert* on the strategy in the "Create Alert" dialog box. No special code is required in scripts for users to be able to do this. The message sent with order fill events can, however, be customized by programmers through use of the `alert_message` parameter in order-generating `strategy.*()` function calls. A combination of `alert()` calls and the use of custom `alert_message` arguments in order-generating `strategy.*()` calls should allow programmers to generate alert events on most conditions occurring in their script's execution.

The `alertcondition()` function remains in Pine Script™ for backward compatibility, but it can also be used advantageously to generate distinct alerts available for selection as individual items in the "Create Alert" dialog box's "Condition" field.

Script alerts

When a script user creates a *script alert* using the "Create Alert" dialog box, the events able to trigger the alert will vary depending on whether the alert is created from an indicator or a strategy.

A *script alert* created from an **indicator** will trigger when:

- The indicator contains `alert()` calls.
- The code's logic allows a specific `alert()` call to execute.
- The frequency specified in the `alert()` call allows the alert to trigger.

A *script alert* created from a **strategy** can trigger on *alert() function calls*, on *order fill events*, or both. The script user creating an alert on a strategy decides which type of events he wishes to include in his *script alert*. While users can create a *script alert* on *order fill events* without the need for a strategy to include special code, it must contain `alert()` calls for users to include *alert() function calls* in their *script alert*.

`alert()` function events

The `alert()` function has the following signature:

`alert(message, freq)`

`message`

A "series string" representing the message text sent when the alert triggers. Because this argument allows "series" values, it can be generated at runtime and differ bar to bar, making it dynamic.

`freq`

An "input string" specifying the triggering frequency of the alert. Valid arguments are:

- `alert.freq_once_per_bar`: Only the first call per realtime bar triggers the alert (default value).
- `alert.freq_once_per_bar_close`: An alert is only triggered when the realtime bar closes and an `alert()` call is executed during that script iteration.
- `alert.freq_all`: All calls during the realtime bar trigger the alert.

The `alert()` function can be used in both indicators and strategies. For an `alert()` call to trigger a *script alert* configured on *alert() function calls*, the script's logic must allow the `alert()` call to execute, **and** the frequency determined by the `freq` parameter must allow the alert to trigger.

Note that by default, strategies are recalculated at the bar's close, so if the `alert()` function with the frequency `alert.freq_all` or `alert.freq_once_per_bar` is used in a strategy, then it will be called no more often than once at the bar's close. In order to enable the `alert()` function to be called during the bar construction process, you need to enable the `calc_on_every_tick` option.

Using all `alert()` calls Let's look at an example where we detect crosses of the RSI centerline:

```
//@version=6
indicator("All `alert()` calls")
r = ta.rsi(close, 20)

// Detect crosses.
xUp = ta.crossover( r, 50)
xDn = ta.crossunder(r, 50)
// Trigger an alert on crosses.
if xUp
    alert("Go long (RSI is " + str.tostring(r, "#.00"))")
```

```

else if xDn
    alert("Go short (RSI is " + str.tostring(r, "#.00"))

plotchar(xUp, "Go Long", " ", location.bottom, color.lime, size = size.tiny)
plotchar(xDn, "Go Short", " ", location.top, color.red, size = size.tiny)
hline(50)
plot(r)

```

If a *script alert* is created from this script:

- When RSI crosses the centerline up, the *script alert* will trigger with the “Go long...” message. When RSI crosses the centerline down, the *script alert* will trigger with the “Go short...” message.
- Because no argument is specified for the `freq` parameter in the `alert()` call, the default value of `alert.freq_once_per_bar` will be used, so the alert will only trigger the first time each of the `alert()` calls is executed during the realtime bar.
- The message sent with the alert is composed of two parts: a constant string and then the result of the `str.tostring()` call which will include the value of RSI at the moment where the `alert()` call is executed by the script. An alert message for a cross up would look like: “Go long (RSI is 53.41)”.
- Because a *script alert* always triggers on any occurrence of a call to `alert()`, as long as the frequency used in the call allows for it, this particular script does not allow a script user to restrict his *script alert* to longs only, for example.

Note that:

- Contrary to an `alertcondition()` call which is always placed at column 0 (in the script’s global scope), the `alert()` call is placed in the local scope of an `if` branch so it only executes when our triggering condition is met. If an `alert()` call was placed in the script’s global scope at column 0, it would execute on all bars, which would likely not be the desired behavior.
- An `alertcondition()` could not accept the same string we use for our alert’s message because of its use of the `str.tostring()` call. `alertcondition()` messages must be constant strings.

Lastly, because `alert()` messages can be constructed dynamically at runtime, we could have used the following code to generate our alert events:

```

// Trigger an alert on crosses.
if xUp or xDn
    firstPart = (xUp ? "Go long" : "Go short") + " (RSI is "
    alert(firstPart + str.tostring(r, "#.00"))

```

Using selective `alert()` calls When users create a *script alert* on *alert()* function calls, the alert will trigger on any call the script makes to the `alert()` function, provided its frequency constraints are met. If you want to allow your script’s users to select which `alert()` function call in your script will trigger a *script alert*, you will need to provide them with the means to indicate their preference in your script’s inputs, and code the appropriate logic in your script. This way, script users will be able to create multiple *script alerts* from a single script, each behaving differently as per the choices made in the script’s inputs prior to creating the alert in the charts UI.

Suppose, for our next example, that we want to provide the option of triggering alerts on only longs, only shorts, or both. You could code your script like this:

```

//@version=6
indicator("Selective `alert()` calls")
detectLongsInput  = input.bool(true,  "Detect Longs")
detectShortsInput = input.bool(true,  "Detect Shorts")
repaintInput      = input.bool(false, "Allow Repainting")

r = ta.rsi(close, 20)
// Detect crosses.
xUp = ta.crossover( r, 50)
xDn = ta.crossunder(r, 50)
// Only generate entries when the trade's direction is allowed in inputs.
enterLong  = detectLongsInput and xUp and (repaintInput or barstate.isconfirmed)
enterShort = detectShortsInput and xDn and (repaintInput or barstate.isconfirmed)
// Trigger the alerts only when the compound condition is met.
if enterLong
    alert("Go long (RSI is " + str.tostring(r, "#.00"))
else if enterShort

```

```

    alert("Go short (RSI is " + str.tostring(r, "%.00"))

plotchar(enterLong, "Go Long", " ", location.bottom, color.lime, size = size.tiny)
plotchar(enterShort, "Go Short", " ", location.top, color.red, size = size.tiny)
hline(50)
plot(r)

```

Note how:

- We create a compound condition that is met only when the user's selection allows for an entry in that direction. A long entry on a crossover of the centerline only triggers the alert when long entries have been enabled in the script's Inputs.
- We offer the user to indicate his repainting preference. When he does not allow the calculations to repaint, we wait until the bar's confirmation to trigger the compound condition. This way, the alert and the marker only appear at the end of the realtime bar.
- If a user of this script wanted to create two distinct script alerts from this script, i.e., one triggering only on longs, and one only on shorts, then he would need to:
 - Select only "Detect Longs" in the inputs and create a first *script alert* on the script.
 - Select only "Detect Shorts" in the Inputs and create another *script alert* on the script.

In strategies `alert()` function calls can be used in strategies also, with the provision that strategies, by default, only execute on the close of realtime bars. Unless `calc_on_every_tick = true` is used in the `strategy()` declaration statement, all `alert()` calls will use the `alert.freq_once_per_bar_close` frequency, regardless of the argument used for `freq`.

While *script alerts* on strategies will use *order fill events* to trigger alerts when the broker emulator fills orders, `alert()` can be used advantageously to generate other alert events in strategies.

This strategy creates *alert() function calls* when RSI moves against the trade for three consecutive bars:

```

//@version=6
strategy("Strategy with selective `alert()` calls")
r = ta.rsi(close, 20)

// Detect crosses.
xUp = ta.crossover( r, 50)
xDn = ta.crossunder(r, 50)
// Place orders on crosses.
if xUp
    strategy.entry("Long", strategy.long)
else if xDn
    strategy.entry("Short", strategy.short)

// Trigger an alert when RSI diverges from our trade's direction.
divInLongTrade = strategy.position_size > 0 and ta.falling(r, 3)
divInShortTrade = strategy.position_size < 0 and ta.rising( r, 3)
if divInLongTrade
    alert("WARNING: Falling RSI", alert.freq_once_per_bar_close)
if divInShortTrade
    alert("WARNING: Rising RSI", alert.freq_once_per_bar_close)

plotchar(xUp, "Go Long", " ", location.bottom, color.lime, size = size.tiny)
plotchar(xDn, "Go Short", " ", location.top, color.red, size = size.tiny)
plotchar(divInLongTrade, "WARNING: Falling RSI", "•", location.top, color.red, size = size.tiny)
plotchar(divInShortTrade, "WARNING: Rising RSI", "•", location.bottom, color.lime, size = size.tiny)
hline(50)
plot(r)

```

If a user created a *script alert* from this strategy and included both *order fill events* and *alert() function calls* in his alert, the alert would trigger whenever an order is executed, or when one of the `alert()` calls was executed by the script on the realtime bar's closing iteration, i.e., when `barstate.isrealtime` and `barstate.isconfirmed` are both true. The *alert() function events* in the script would only trigger the alert when the realtime bar closes because `alert.freq_once_per_bar_close` is the argument used for the `freq` parameter in the `alert()` calls.

Order fill events

When a *script alert* is created from an indicator, it can only trigger on *alert()* function calls. However, when a *script alert* is created from a strategy, the user can specify that *order fill events* also trigger the *script alert*. An *order fill event* is any event generated by the broker emulator which causes a simulated order to be executed. It is the equivalent of a trade order being filled by a broker/exchange. Orders are not necessarily executed when they are placed. In a strategy, the execution of orders can only be detected indirectly and after the fact, by analyzing changes in built-in variables such as `strategy.opentrades` or `strategy.position_size`. *Script alerts* configured on *order fill events* are thus useful in that they allow the triggering of alerts at the precise moment of an order's execution, before a script's logic can detect it.

Pine Script™ programmers can customize the alert message sent when specific orders are executed. While this is not a pre-requisite for *order fill events* to trigger, custom alert messages can be useful because they allow custom syntax to be included with alerts in order to route actual orders to a third-party execution engine, for example. Specifying custom alert messages for specific *order fill events* is done by means of the `alert_message` parameter in functions which can generate orders: `strategy.close()`, `strategy.entry()`, `strategy.exit()` and `strategy.order()`.

The argument used for the `alert_message` parameter is a “series string”, so it can be constructed dynamically using any variable available to the script, as long as it is converted to string format.

Let's look at a strategy where we use the `alert_message` parameter in both our `strategy.entry()` calls:

```
//@version=6
strategy("Strategy using `alert_message`")
r = ta.rsi(close, 20)

// Detect crosses.
xUp = ta.crossover( r, 50)
xDn = ta.crossunder(r, 50)
// Place order on crosses using a custom alert message for each.
if xUp
    strategy.entry("Long", strategy.long, stop = high, alert_message = "Stop-buy executed (stop was " + str.to
else if xDn
    strategy.entry("Short", strategy.short, stop = low, alert_message = "Stop-sell executed (stop was " + str.

plotchar(xUp, "Go Long", " ", location.bottom, color.lime, size = size.tiny)
plotchar(xDn, "Go Short", " ", location.top, color.red, size = size.tiny)
hline(50)
plot(r)
```

Note that:

- We use the `stop` parameter in our `strategy.entry()` calls, which creates stop-buy and stop-sell orders. This entails that buy orders will only execute once price is higher than the `high` on the bar where the order is placed, and sell orders will only execute once price is lower than the `[low]` on the bar where the order is placed.
- The up/down arrows which we plot with `plotchar()` are plotted when orders are **placed**. Any number of bars may elapse before the order is actually executed, and in some cases the order will never be executed because price does not meet the required condition.
- Because we use the same `id` argument for all buy orders, any new buy order placed before a previous order's condition is met will replace that order. The same applies to sell orders.
- Variables included in the `alert_message` argument are evaluated when the order is executed, so when the alert triggers.

When the `alert_message` parameter is used in a strategy's order-generating `strategy.*()` function calls, script users must include the `{{strategy.order.alert_message}}` placeholder in the “Create Alert” dialog box's “Message” field when creating *script alerts* on *order fill events*. This is required so the `alert_message` argument used in the order-generating `strategy.*()` function calls is used in the message of alerts triggering on each *order fill event*. When only using the `{{strategy.order.alert_message}}` placeholder in the “Message” field and the `alert_message` parameter is present in only some of the order-generating `strategy.*()` function calls in your strategy, an empty string will replace the placeholder in the message of alerts triggered by any order-generating `strategy.*()` function call not using the `alert_message` parameter.

While other placeholders can be used in the “Create Alert” dialog box's “Message” field by users creating alerts on *order fill events*, they cannot be used in the argument of `alert_message`.

alertcondition() events

The `alertcondition()` function allows programmers to create individual *alertcondition events* in their indicators. One indicator may contain more than one `alertcondition()` call. Each call to `alertcondition()` in a script will create a corresponding alert selectable in the “Condition” dropdown menu of the “Create Alert” dialog box.

While the presence of `alertcondition()` calls in a **strategy** script will not cause a compilation error, alerts cannot be created from them.

The `alertcondition()` function has the following signature:

`alertcondition(condition, title, message)`

condition

A “series bool” value (**true** or **false**) which determines when the alert will trigger. It is a required argument. When the value is **true** the alert will trigger. When the value is **false** the alert will not trigger. Contrary to `alert()` function calls, `alertcondition()` calls must start at column zero of a line, so cannot be placed in conditional blocks.

title

A “const string” optional argument that sets the name of the alert condition as it will appear in the “Create Alert” dialog box’s “Condition” field in the charts UI. If no argument is supplied, “Alert” will be used.

message

A “const string” optional argument that specifies the text message to display when the alert triggers. The text will appear in the “Message” field of the “Create Alert” dialog box, from where script users can then modify it when creating an alert. **As this argument must be a “const string”, it must be known at compilation time and thus cannot vary bar to bar.** It can, however, contain placeholders which will be replaced at runtime by dynamic values that may change bar to bar. See this page’s Placeholders section for a list.

The `alertcondition()` function does not include a **freq** parameter. The frequency of *alertcondition()* alerts is determined by users in the “Create Alert” dialog box.

Using one condition

Here is an example of code creating *alertcondition()* events:

```
//@version=6
indicator("`alertcondition()` on single condition")
r = ta.rsi(close, 20)

xUp = ta.crossover( r, 50)
xDn = ta.crossunder(r, 50)

plot(r, "RSI")
hline(50)
plotchar(xUp, "Long", " ", location.bottom, color.lime, size = size.tiny)
plotchar(xDn, "Short", " ", location.top, color.red, size = size.tiny)

alertcondition(xUp, "Long Alert", "Go long")
alertcondition(xDn, "Short Alert", "Go short ")
```

Because we have two `alertcondition()` calls in our script, two different alerts will be available in the “Create Alert” dialog box’s “Condition” field: “Long Alert” and “Short Alert”.

If we wanted to include the value of RSI when the cross occurs, we could not simply add its value to the **message** string using `str.toString(r)`, as we could in an `alert()` call or in an `alert_message` argument in a strategy. We can, however, include it using a placeholder. This shows two alternatives:

```
alertcondition(xUp, "Long Alert", "Go long. RSI is {{plot_0}}")
alertcondition(xDn, "Short Alert", 'Go short. RSI is {{plot("RSI")}}')
```

Note that:

- The first line uses the `{{plot_0}}` placeholder, where the plot number corresponds to the order of the plot in the script.

- The second line uses the `{{plot("[plot_title]")}}` type of placeholder, which must include the `title` of the `plot()` call used in our script to plot RSI. Double quotes are used to wrap the plot's title inside the `{{plot("RSI")}}` placeholder. This requires that we use single quotes to wrap the `message` string.
- Using one of these methods, we can include any numeric value that is plotted by our indicator, but as strings cannot be plotted, no string variable can be used.

Using compound conditions

If we want to offer script users the possibility of creating a single alert from an indicator using multiple `alertcondition()` calls, we will need to provide options in the script's inputs through which users will indicate the conditions they want to trigger their alert before creating it.

This script demonstrates one way to do it:

```
//@version=6
indicator("`alertcondition()` on multiple conditions")
detectLongsInput = input.bool(true, "Detect Longs")
detectShortsInput = input.bool(true, "Detect Shorts")

r = ta.rsi(close, 20)
// Detect crosses.
xUp = ta.crossover( r, 50)
xDn = ta.crossunder(r, 50)
// Only generate entries when the trade's direction is allowed in inputs.
enterLong  = detectLongsInput  and xUp
enterShort = detectShortsInput and xDn

plot(r)
plotchar(enterLong,  "Go Long",  " ", location.bottom, color.lime, size = size.tiny)
plotchar(enterShort, "Go Short", " ", location.top,    color.red,   size = size.tiny)
hline(50)
// Trigger the alert when one of the conditions is met.
alertcondition(enterLong or enterShort, "Compound alert", "Entry")
```

Note how the `alertcondition()` call is allowed to trigger on one of two conditions. Each condition can only trigger the alert if the user enables it in the script's inputs before creating the alert.

Placeholders

These placeholders can be used in the `message` argument of `alertcondition()` calls. They will be replaced with dynamic values when the alert triggers. They are the only way to include dynamic values (values that can vary bar to bar) in `alertcondition()` messages.

Note that users creating *alertcondition()* alerts from the “Create Alert” dialog box in the charts UI are also able to use these placeholders in the dialog box's “Message” field.

`{{exchange}}`

Exchange of the symbol used in the alert (NASDAQ, NYSE, MOEX, etc.). Note that for delayed symbols, the exchange will end with “_DL” or “_DLY.” For example, “NYMEX_DL.”

`{{interval}}`

Returns the timeframe of the chart the alert is created on. Note that Range charts are calculated based on 1m data, so the placeholder will always return “1” on any alert created on a Range chart.

`{{open}}`, `{{high}}`, `{{low}}`, `{{close}}`, `{{volume}}`

Corresponding values of the bar on which the alert has been triggered.

`{{plot_0}}`, `{{plot_1}}`, [...], `{{plot_19}}`

Value of the corresponding plot number. Plots are numbered from zero to 19 in order of appearance in the script, so only one of the first 20 plots can be used. For example, the built-in “Volume” indicator has two output series: Volume and Volume MA, so you could use the following:

```
alertcondition(volume > ta.sma(volume,20), "Volume alert", "Volume ({{plot_0}}) > average ({{plot_1}})")
```

```
{{plot("[plot_title]")}}
```

This placeholder can be used when one needs to refer to a plot using the `title` argument used in a `plot()` call. Note that double quotation marks (") **must** be used inside the placeholder to wrap the `title` argument. This requires that a single quotation mark (') be used to wrap the `message` string:

```
//@version=6
indicator("")
r = ta.rsi(close, 14)
xUp = ta.crossover(r, 50)
plot(r, "RSI", display = display.none)
alertcondition(xUp, "xUp alert", message = 'RSI is bullish at: {{plot("RSI")}}')

{{ticker}}
```

Ticker of the symbol used in the alert (AAPL, BTCUSD, etc.).

```
{{time}}
```

Returns the time at the beginning of the bar. Time is UTC, formatted as `yyyy-MM-ddTHH:mm:ssZ`, so for example: `2019-08-27T09:56:00Z`.

```
{{timenow}}
```

Current time when the alert triggers, formatted in the same way as `{{time}}`. The precision is to the nearest second, regardless of the chart's timeframe.

Avoiding repainting with alerts

The most common instances of repainting traders want to avoid with alerts are ones where they must prevent an alert from triggering at some point during the realtime bar when it would **not** have triggered at its close. This can happen when these conditions are met:

- The calculations used in the condition triggering the alert can vary during the realtime bar. This will be the case with any calculation using `high`, `low` or `close`, for example, which includes almost all built-in indicators. It will also be the case with the result of any `request.security()` call using a higher timeframe than the chart's, when the higher timeframe's current bar has not closed yet.
- The alert can trigger before the close of the realtime bar, so with any frequency other than "Once Per Bar Close".

The simplest way to avoid this type of repainting is to configure the triggering frequency of alerts so they only trigger on the close of the realtime bar. There is no panacea; avoiding this type of repainting **always** entails waiting for confirmed information, which means the trader must sacrifice immediacy to achieve reliability.

Note that other types of repainting such as those documented in our Repainting section may not be preventable by simply triggering alerts on the close of realtime bars.

[Next

Backgrounds](#backgrounds) User Manual/Concepts/Backgrounds

Backgrounds

The `bgcolor()` function changes the color of the script's background. If the script is running in `overlay = true` mode, then it will color the chart's background.

The function's signature is:

```
bgcolor(color, offset, editable, show_last, title, force_overlay) → void
```

Its `color` parameter allows a "series color" to be used for its argument, so it can be dynamically calculated in an expression.

If the correct transparency is not part of the color to be used, it can be generated using the `color.new()` function.

Here is a script that colors the background of trading sessions (try it on 30min EURUSD, for example):

```
//@version=6
indicator("Session backgrounds", overlay = true)
```

```
// Default color constants using transparency of 25.
BLUE_COLOR   = #0050FF40
PURPLE_COLOR  = #0000FF40
PINK_COLOR    = #5000FF40
NO_COLOR      = color(na)

// Allow user to change the colors.
preMarketColor = input.color(BLUE_COLOR, "Pre-market")
regSessionColor = input.color(PURPLE_COLOR, "Pre-market")
postMarketColor = input.color(PINK_COLOR, "Pre-market")

// Function returns `true` when the bar's time is
timeInRange(tf, session) =>
    time(tf, session) != 0

// Function prints a message at the bottom-right of the chart.
f_print(_text) =>
    var table _t = table.new(position.bottom_right, 1, 1)
    table.cell(_t, 0, 0, _text, bgcolor = color.yellow)

var chartIs30MinOrLess = timeframe.isseconds or (timeframe.isintraday and timeframe.multiplier <=30)
sessionColor = if chartIs30MinOrLess
    switch
        timeInRange(timeframe.period, "0400-0930") => preMarketColor
        timeInRange(timeframe.period, "0930-1600") => regSessionColor
        timeInRange(timeframe.period, "1600-2000") => postMarketColor
        => NO_COLOR
else
    f_print("No background is displayed.\nChart timeframe must be <= 30min.")
    NO_COLOR

bgcolor(sessionColor)
```



Figure 96: image

Note that:

- The script only works on chart timeframes of 30min or less. It prints an error message when the chart's timeframe is higher than 30min.
- When the if structure's `else` branch is used because the chart's timeframe is incorrect, the local block returns the `NO_COLOR` color so that no background is displayed in that case.
- We first initialize constants using our base colors, which include the 40 transparency in hex notation at the end. 40 in the hexadecimal notation on the reversed 00-FF scale for transparency corresponds to 75 in Pine Script™'s 0-100

decimal scale for transparency.

- We provide color inputs allowing script users to change the default colors we propose.

In our next example, we generate a gradient for the background of a CCI line:

```
//@version=6
indicator("CCI Background")

bullColor = input.color(color.lime, " ", inline = "1")
bearColor = input.color(color.fuchsia, " ", inline = "1")

// Calculate CCI.
myCCI = ta.cci(hlc3, 20)
// Get relative position of CCI in last 100 bars, on a 0-100% scale.
myCCIPosition = ta.percentrank(myCCI, 100)
// Generate a bull gradient when position is 50-100%, bear gradient when position is 0-50%.
backgroundColor = if myCCIPosition >= 50
    color.from_gradient(myCCIPosition, 50, 100, color.new(bullColor, 75), bullColor)
else
    color.from_gradient(myCCIPosition, 0, 50, bearColor, color.new(bearColor, 75))

// Wider white line background.
plot(myCCI, "CCI", color.white, 3)
// Think black line.
plot(myCCI, "CCI", color.black, 1)
// Zero level.
hline(0)
// Gradient background.
bgcolor(backgroundColor)
```



Figure 97: image

Note that:

- We use the `ta.cci()` built-in function to calculate the indicator value.
- We use the `ta.percentrank()` built-in function to calculate `myCCIPosition`, i.e., the percentage of past `myCCI` values in the last 100 bars that are below the current value of `myCCI`.

- To calculate the gradient, we use two different calls of the `color.from_gradient()` built-in: one for the bull gradient when `myCCIPosition` is in the 50-100% range, which means that more past values are below its current value, and another for the bear gradient when `myCCIPosition` is in the 0-49.99% range, which means that more past values are above it.
- We provide inputs so the user can change the bull/bear colors, and we place both color input widgets on the same line using `inline = "1"` in both `input.color()` calls.
- We plot the CCI signal using two `plot()` calls to achieve the best contrast over the busy background: the first plot is a 3-pixel wide white background, the second `plot()` call plots the thin, 1-pixel wide black line.

See the Colors page for more examples of backgrounds.

[\[Previous](#)

[Alerts\]\(#alerts\)\[Next](#)

[Bar coloring\]\(#bar-coloring\)](#) User Manual/Concepts/Bar coloring

Bar coloring

The `barcolor()` function colors bars on the main chart, regardless of whether the script is running in the main chart pane or a separate pane.

The function's signature is:

`barcolor(color, offset, editable, show_last, title, display) → void`

The coloring can be conditional because the `color` parameter accepts “series color” arguments.

The following script renders *inside* and *outside* bars in different colors:



Figure 98: image

```
//@version=6
indicator("barcolor example", overlay = true)
isUp = close > open
isDown = close <= open
isOutsideUp = high > high[1] and low < low[1] and isUp
isOutsideDown = high > high[1] and low < low[1] and isDown
isInside = high < high[1] and low > low[1]
barcolor(isInside ? color.yellow : isOutsideUp ? color.aqua : isOutsideDown ? color.purple : na)
```

Note that:

- The `na` value leaves bars as is.
- In the `barcolor()` call, we use embedded `?:` ternary operator expressions to select the color.

[\[Previous](#)

[Backgrounds\]\(#backgrounds\)\[Next](#)

Bar plotting

Introduction

The `plotcandle()` built-in function is used to plot candles. `plotbar()` is used to plot conventional bars.

Both functions require four arguments that will be used for the OHLC prices (open, high, low, close) of the bars they will be plotting. If one of those is `na`, no bar is plotted.

Plotting candles with `plotcandle()`

The signature of `plotcandle()` is:

`plotcandle(open, high, low, close, title, color, wickcolor, editable, show_last, bordercolor, display) → void`

This plots simple candles, all in blue, using the habitual OHLC values, in a separate pane:

```
//@version=6
indicator("Single-color candles")
plotcandle(open, high, low, close)
```

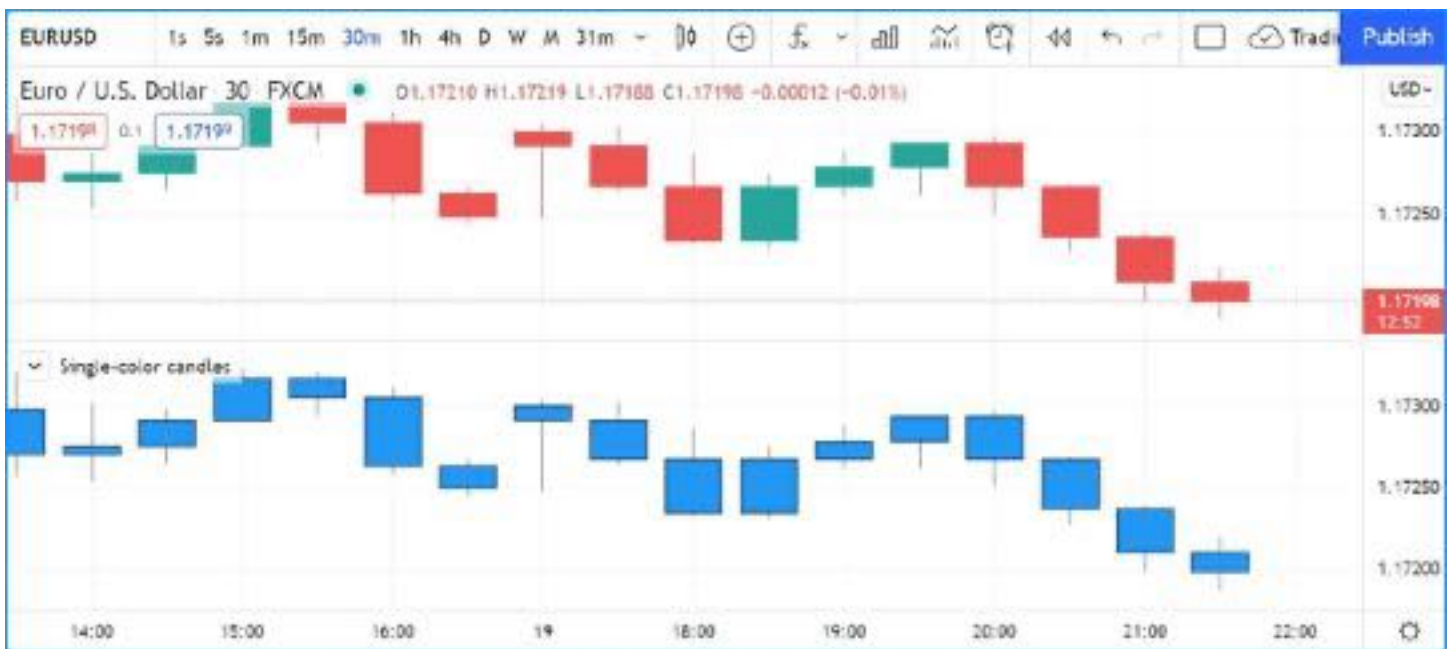


Figure 99: image

To color them green or red, we can use the following code:

```
//@version=6
indicator("Example 2")
paletteColor = close >= open ? color.lime : color.red
plotbar(open, high, low, close, color = paletteColor)
```

Note that the `color` parameter accepts “series color” arguments, so constant values such as `color.red`, `color.lime`, `“#FF9090”`, as well as expressions that calculate colors at runtime, as is done with the `paletteColor` variable here, will all work.

You can build bars or candles using values other than the actual OHLC values. For example you could calculate and plot smoothed candles using the following code, which also colors wicks depending on the position of close relative to the smoothed close (`c`) of our indicator:

```
//@version=6
indicator("Smoothed candles", overlay = true)
lenInput = input.int(9)
```



Figure 100: image

```
smooth(source, length) =>
    ta.sma(source, length)
o = smooth(open, lenInput)
h = smooth(high, lenInput)
l = smooth(low, lenInput)
c = smooth(close, lenInput)
ourWickColor = close > c ? color.green : color.red
plotcandle(o, h, l, c, wickcolor = ourWickColor)
```



Figure 101: image

You may find it useful to plot OHLC values taken from a higher timeframe. You can, for example, plot daily bars on an intraday chart:

```
// NOTE: Use this script on an intraday chart.
//@version=6
```